

Relatório de Auditoria de Segurança, Estrutura e Performance

— FTTH Manager (ftth_tiiv)

Data da auditoria	18/09/2026
Versão auditada	e446c49 (HEAD) + working tree com 108 arquivos modificados/novos (auditado o working tree)
Escopo auditado	backend/ (FastAPI, 110 handlers), frontend/ (Next.js), compose.yaml, Caddyfile, Dockerfiles, CI, migrations, scripts, docs, histórico git (8 commits)
Modo de trabalho	Somente leitura no projeto. Medições apenas em PostgreSQL/PostGIS descartável local. Nenhum exploit executado; nada foi rodado contra produção.
Resultado	53 achados — 0 críticos, 10 altos, 26 médios, 17 baixos — e 21 pontos fortes verificados. Nenhum achado crítico.

Nota metodológica — como cada categoria foi mapeada para a stack

Stack detectada	Python 3.12 · FastAPI 0.141 (Starlette 1.6) · SQLAlchemy 2 síncrono + psycopg3 + GeoAlchemy2/Shapely · PostgreSQL 16/PostGIS 3.4 · Alembic · Argon2id · Pillow · Next.js 15.5 (React 19, TanStack Query/Table, MapLibre GL 5) · Docker Compose (db, migrate, backend, worker, frontend, Caddy 2.8) · CI GitHub Actions.
Autenticação / autorização	Sessão opaca (cookie ftth_session ou Bearer) → get_current_user → require_permission(perm) (matriz por papel). CSRF por double-submit + Origin. Sem JWT.
Isolamento de tenant	Inexistente por design (organização única): categorias 'banco sem tranca' e 'IDOR' foram mapeadas para (a) rotas sem autenticação/permissão (SEC-01/02/03/04, EST-02) e (b) IDOR vertical — permissão por tipo de objeto (anexos, clientes, jobs, exportações).
Mapeamento das categorias	Chaves expostas → código, compose, .env.example, CI, scripts, docs, histórico git (8 commits) e bundle .next. XSS → sinks React/DOM e HTML/e-mail no backend (N/A). Fundamentos → sessão/CSRF/CORS/headers/upload/SQLi/logs. Performance → N+1 do ORM, índices das migrações × WHERE/ORDER, código síncrono em async def (equivalente a event loop bloqueado), pool/threadpool, memória, escala horizontal.
Método de verificação	Leitura integral do núcleo de segurança e dos módulos de risco; introspecção do app para o inventário (110 handlers); pip-audit e pnpm audit; varredura do histórico git; medições em PostgreSQL/PostGIS descartável (127.0.0.1:55432, contêiner --rm) com dataset sintético do próprio repositório (10.001 estruturas, 103.200 fibras).
O que foi executado	Somente leitura no projeto. Nenhum exploit, nenhuma requisição a ambientes reais. Único código do projeto importado: app.main (introspecção), serviços e schemas em ambiente descartável. Venv em /tmp/audit-env; PYTHONDONTWRITEBYTECODE=1.

AVISO DE CONFIDENCIALIDADE. Este documento descreve vulnerabilidades reais e trechos de código do sistema. Destina-se exclusivamente à equipe responsável pelo projeto. Não o publique em repositório público nem o encaminhe a terceiros sem autorização. Segredos foram mascarados (4 primeiros caracteres + ...). Sugere-se manter docs/security-audit/ fora do controle de versão público (adicionar ao .gitignore).

Sumário

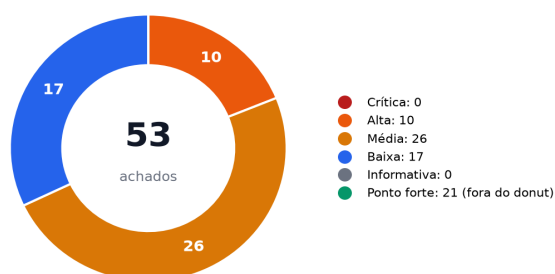
1. Resumo executivo	3
2. Mapa do sistema	4
3. Pontos fortes verificados	5
4. Pontos fracos (riscos centrais)	7
5. Categorias não aplicáveis à stack	7
6. Achados detalhados por categoria	8
6.1 Segurança (SEC)	8
6.2 Estrutura (EST)	10
6.3 Performance (PERF)	13
7. Top 10 gargalos de performance	17
8. Matriz esforço × impacto (todos os achados)	18
9. Recomendações priorizadas	19
Onda 1 — imediato (dias)	19
Onda 2 — curto prazo (2-4 semanas)	19
Onda 3 — médio prazo (1-3 meses)	21
Backlog	22
10. Limitações do trabalho	22
11. ISSUES PARA O GITHUB	23

1. Resumo executivo

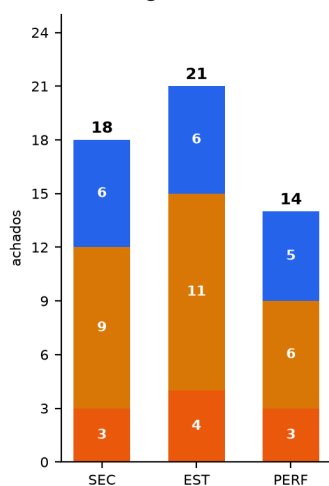
Foram verificados **53 achados** no código real (arquivo, linha e trecho conferidos por script de fidelidade): **10 altos**, **26 médios** e **17 baixos**; nenhum crítico e nenhum informativo. Há 21 controles corretos documentados. Por categoria: 18 de segurança, 21 estruturais e 14 de performance. A aplicação é de **organização única** (sem tenant): o principal risco de segurança não é o vazamento entre clientes, e sim a **autorização por rota** — endpoints públicos sem intenção e permissões declaradas mas nunca aplicadas.

Severidade	Segurança	Estrutura	Performance	Total
CRÍTICA	0	0	0	0
ALTA	3	4	3	10
MÉDIA	9	11	6	26
BAIXA	6	6	5	17
INFORMATIVA	0	0	0	0
Total	18	21	14	53
Pontos fortes				21

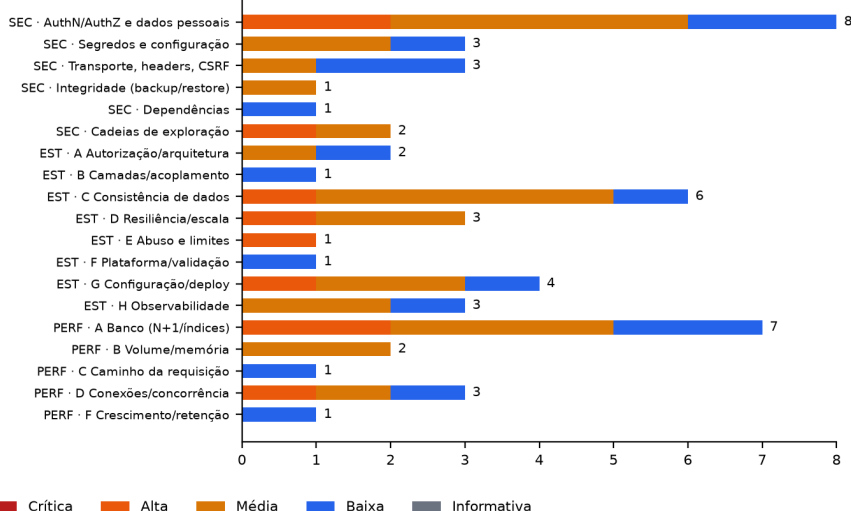
Achados por severidade



Por categoria



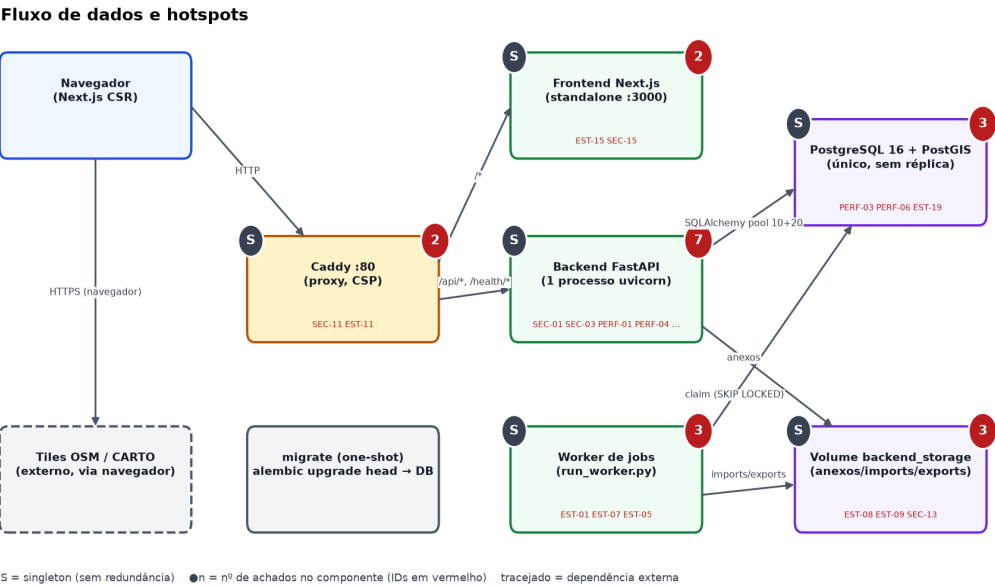
Por subcategoria



Três conclusões para decisão

- **Fechar hoje (Onda 1):** endpoints anônimos (/search, /dashboard/summary), customers:* não aplicado, token de métricas padrão, worker que não inicia, build do frontend, teto de fiber_count e do upload de imagem. São correções de horas a poucos dias.
- **Autorização precisa virar padrão, não opt-in:** dependência global de autenticação + teste que percorre todas as rotas + teste de matriz papel×rota evita que a classe de erro volte (EST-02, SEC-03).
- **Performance está no caminho de leitura pesado:** o dashboard (2.514 queries com 10 mil estruturas), o impacto/rastreo (62 queries por vínculo) e o upload de imagem (+339 MiB) foram **medidos**; o restante está rotulado 'verificado por leitura, não medido'.

2. Mapa do sistema



Componentes, fronteiras e dependências

Componente	Singleton?	Dependências / observações
Navegador (Next.js CSR)	não	Next.js (CSR), TanStack Query; sessão em cookie HttpOnly; busca tiles diretamente em OSM/CARTO (CSP).
Caddy :80 (proxy, CSP)	sim (SPOF)	Proxy único em :80 (auto_https off); encaminha /api/*, /health/* ao backend e o resto ao Next.js. Aplica CSP/headers.
Frontend Next.js (standalone :3000)	sim (SPOF)	Next.js standalone :3000; rewrite /api/v1 só usado sem Caddy. Imagem não builda (EST-15).
Backend FastAPI (1 processo uvicorn)	sim (SPOF)	FastAPI + SQLAlchemy síncrono; 1 processo uvicorn; pool 10+20; métricas e lru_cache em memória do processo.
Worker de jobs (run_worker.py)	sim (SPOF)	Consumidor de async_jobs (SKIP LOCKED); hoje crash-loop (EST-01); limpeza periódica de previews.
PostgreSQL 16 + PostGIS (único, sem réplica)	sim (SPOF)	PostgreSQL 16 + PostGIS; única instância; rede internal; senha default do compose (SEC-05). Fonte da verdade de sessões, rate limit e fila.
Volume backend_storage (anexos/imports/exports)	sim (SPOF)	Volume Docker local compartilhado por backend e worker; anexos, previews e exports; sem retenção de exports (PERF-11).
Tiles OSM / CARTO (externo, via navegador)	não	Serviços externos consultados pelo navegador; única integração de terceiros. O backend não faz chamadas HTTP de saída.
migrate (one-shot) alembic upgrade head -> DB	não	Serviço one-shot alembic upgrade head antes do backend (sem corrida entre réplicas).

Isolamento de tenant, autenticação e superfície

- **Tenant:** não existe. Nenhuma das 28 tabelas tem tenant_id/org_id/owner_id (justificativa: organização única; o controle é RBAC global). Detalhes na seção “Categorias não aplicáveis”.
- **AuthN:** sessão opaca (core/dependencies.py:117-142 → identity/service.py:119-157). **AuthZ:** require_permission (core/dependencies.py:181-195) + matriz (core/permissions.py:4-80). **CSRF:** core/dependencies.py:67-114.
- **Superfície:** 110 handlers (contagem por introspecção de app, ver inventario-rotas.md): 89 com require_permission, 7 públicos por design, 9 sem auth por engano/stub, 2 com auth opcional, 3 com sessão de qualquer papel.
- **Singletons / estado local:** Caddy, backend, worker, PostgreSQL e volume de arquivos são únicos; métricas/lru_cache locais ao processo; sessões, rate limit de login e fila estão no banco (escalam).
- **Modelo de dados (P0.5):** 28 tabelas em 10 migrações reversíveis; FKs RESTRICT, índices únicos parciais para atendimento ativo, GIST nas geometrias; ausência de índice em terminals(entity_type, entity_id) (PERF-03) e de trigram (PERF-06).

3. Pontos fortes verificados

Cada item abaixo foi conferido no código (arquivo:linha). Evidências completas com trechos em `findings.json`.

ID	Controle correto	Evidência (arquivo:linha)
OK-01	Senhas com Argon2id e mitigação de timing na enumeração de usuários Argon2id (time=2, mem=64 MiB) e verificação contra hash sintético quando o e-mail não existe.	backend/app/core/ security.py:10-15 backend/app/core/ security.py:36 backend/app/modules/identity/ service.py:85
OK-02	Sessões opacas de 256 bits, armazenadas como SHA-256, com expiração absoluta/por inatividade e revogação Token gerado com <code>secrets.token_urlsafe(48)</code> ; só o hash vai ao banco; expira em 7 dias (absoluto) e 24 h (inatividade); desativar usuário revoga todas as sessões; novo token a cada login.	backend/app/core/ security.py:48 backend/app/core/ security.py:55 backend/app/modules/identity/ service.py:138
OK-03	Cookie de sessão HttpOnly/SameSite=Lax/Secure em produção; nenhum token em localStorage O frontend só guarda a preferência de sidebar em <code>localStorage</code> ; a sessão nunca fica acessível a JavaScript.	backend/app/core/ dependencies.py:39 backend/app/core/ dependencies.py:41 frontend/src/components/layout/ app-shell.tsx:27
OK-04	RBAC aplicado no servidor em 89 dos 110 handlers via <code>require_permission</code>; papel resolvido a cada requisição O papel é lido do usuário no banco a cada request (mudança de papel/desativação vale imediatamente). Levantamento completo em <code>inventario-rotas.md</code> .	backend/app/core/ dependencies.py:181 backend/app/api/v1/ users.py:53
OK-05	CSRF exigido em todas as mutações reais (47 de 56 handlers mutantes; os 9 restantes são stubs ou cálculos somente-leitura) com comparação em tempo constante <code>validate_csrf</code> exige cabeçalho X-CSRF-Token igual ao cookie e usa <code>hmac.compare_digest</code> .	backend/app/core/ dependencies.py:67 backend/app/core/ security.py:67
OK-06	Sem SQL injection na aplicação: ORM com parâmetros vinculados (único SQL montado por f-string está no CLI de backup — SEC-10) Filtros <code>ilike</code> recebem o termo como parâmetro; <code>text()</code> só aparece com SQL estático.	backend/app/modules/customers/ service.py:86 backend/app/modules/gis/ service.py:37
OK-07	Upload de anexos: magic bytes, nome no disco = UUID, sanitização de nome, contenção de path traversal e Content-Disposition: attachment Formatos limitados a JPEG/PNG/WebP/PDF; SVG/HTML/script rejeitados; nome original nunca usado no caminho físico.	backend/app/modules/attachments/ service.py:25 backend/app/modules/attachments/ service.py:211 backend/app/modules/attachments/ service.py:294
OK-08	Importações/exportações: defusedxml (XXE), proteção contra zip bomb no KMZ e neutralização de injeção de fórmula em CSV KML/KMZ é parseado com <code>defusedxml</code> ; KMZ limitado a 50 MB descompactados; CSV de entrada e de saída rejeita/neutraliza <code>=</code> , <code>+</code> , <code>-</code> , <code>@</code> .	backend/app/modules/imports/ service.py:10 backend/app/modules/imports/ service.py:169 backend/app/modules/imports/ service.py:346
OK-09	Editor de fusão: locks pessimistas ordenados, revisão topológica esperada e bump atômico FOR UPDATE na estrutura e nos terminais em ordem determinística (sem deadlock), validação de <code>expected_topology_revision</code> e UPDATE ... RETURNING atômico da revisão.	backend/app/modules/connectivity/ service.py:439 backend/app/modules/connectivity/ service.py:448 backend/app/modules/gis/ service.py:36
OK-10	Fila de jobs em PostgreSQL com SKIP LOCKED, lease e limite de tentativas Claim seguro entre múltiplos workers e recuperação de crash por lease expirada (limitações em EST-07).	backend/app/modules/jobs/ service.py:102 backend/app/modules/jobs/ service.py:97
OK-11	Integridade referencial e unicidade no banco: FKs RESTRICT, índices únicos parciais para vínculo ativo, migrações reversíveis e executadas em serviço one-shot Uma porta/ONU só tem 1 atendimento ativo (índice único parcial); migrate roda uma vez antes do backend (sem corrida entre réplicas); todas as 10 migrações têm downgrade com operações.	backend/app/modules/customers/ models.py:73 compose.yaml:25 backend/migrations/versions/ 0006_connectivity_engine.py:256

ID	Controle correto	Evidência (arquivo:linha)
OK-12	Frontend sem sinks de XSS, redirecionamento pós-login sanitizado e links externos com rel=noopener grep -rE 'dangerouslySetInnerHTML innerHTML setHTML eval\(new Function javascript:' frontend/src só encontra a checagem defensiva de sanitizeReturnUrl; React escapa por padrão; cliente HTTP envia X-CSRF-Token.	frontend/src/features/auth/ utils.ts:10 frontend/src/components/ui/ copyable-coordinates.tsx:126 frontend/src/lib/api/ client.ts:72
OK-13	Contêineres não-root, banco em rede interna sem porta publicada e docs OpenAPI desativados em produção Backend roda como ftthuser, frontend como nextjs; a rede interna é internal: true; /docs, /redoc e /openapi.json só existem fora de produção.	backend/ Dockerfile:51 frontend/ Dockerfile:29 compose.yaml:122
OK-14	Logs JSON estruturados com redação de campos sensíveis e X-Request-ID propagado Chaves como password/token/secret/cookie/csrf são mascaradas; o request_id acompanha logs, resposta e erros Problem Details; a mensagem 500 não expõe stack trace.	backend/app/core/ logging.py:15 backend/app/core/ middleware.py:33 backend/app/core/ errors.py:269
OK-15	Dependências do backend sem vulnerabilidades conhecidas (pip-audit) e versões travadas por lockfile pip-audit em 2026-09-18: 55 dependências resolvidas, 0 vulnerabilidades (evidencias/pip-audit-backend.json); Dockerfile usa uv sync --frozen.	backend/ uv.lock:793 backend/ Dockerfile:18
OK-16	Referências por ID no corpo/query são validadas quanto à existência e ao tipo antes de gravar Ex.: create_service_link verifica cliente, ONU (kind='onu') e porta; create_measurement verifica terminal e vínculo; create_structure verifica site_id.	backend/app/modules/customers/ service.py:297 backend/app/modules/measurements/ service.py:128 backend/app/modules/inventory/ service.py:275
OK-17	Rate limit de login e sessões persistidos no banco (funcionam com várias réplicas) Contagem de tentativas por janela em login_attempts; sem estado em memória do processo (limitações de política em SEC-06).	backend/app/modules/identity/ service.py:53 backend/app/modules/identity/ models.py:58
OK-18	Proteção do último administrador e If-Match obrigatório (428) na gestão de usuários Impede desativar/rebaixar o único admin ativo; PATCH/DELETE /users/{id} exigem If-Match.	backend/app/modules/identity/ service.py:246 backend/app/api/v1/ users.py:98
OK-19	Performance do frontend: busca com debounce, mapa carregado sob demanda e cache de consultas data-table-filter-bar usa debounce de 350 ms; o mapa (MapLibre) é dynamic(); o mapa consulta por bbox com debounce e cancelamento; TanStack Query com staleTime.	frontend/src/components/ui/data-table/ data-table-filter-bar.tsx:27 frontend/src/features/map/components/ map-view.tsx:38 frontend/src/app/ providers.tsx:14
OK-20	Verificação negativa: histórico git e bundle sem segredos 8 commits varridos (git log --all -p): nenhum arquivo .env/.pem/.key rastreado e nenhum padrão AKIA/ghp/_xox*/sk-/PRIVATE KEY; só valores de teste. O bundle .next local não contém os defaults de segredo. Os defaults em código/compose estão em SEC-05.	git log --all --name-only --pretty=format: sort -u grep -E '(\.env \.pem \.key id_rsa credentials)\$' # → vazio
OK-21	Suíte de testes ampla e CI executando testes, tipos, lint e restore drill 142 funções de teste no backend, 19 arquivos de teste no frontend; o CI roda tudo e o restore drill (lacuna: nenhum teste verifica autorização por rota — EST-02/SEC-03).	.github/workflows/ ci.yml:79

4. Pontos fracos (riscos centrais)

1	Autorização é opt-in por rota e não há teste de matriz papel×rota 9 handlers ficaram públicos sem intenção, <code>customers:*</code> nunca é exigido e anexos/auditoria ignoram a permissão da entidade.	SEC-01, SEC-03, SEC-04, EST-02
2	Entrega/operação quebrada em pontos críticos Worker não inicia, build do frontend falha, importação descarta a geometria dos cabos e grava fora do volume configurado.	EST-01, EST-15, EST-05, EST-08
3	Sem defesas de abuso nem tetos de recurso Um único request pode esgotar memória (<code>fiber_count</code> , imagem grande) e um anônimo satura o processo (dashboard N+1).	EST-11, SEC-16, PERF-04, PERF-12
4	Integridade sob concorrência sem atomicidade If-Match é check-then-act, divisão de segmento sem lock, lease de job nunca renovada, uploads gravados antes do commit.	EST-03, EST-04, EST-07, EST-09
5	Rastreabilidade insuficiente Auditoria cobre 4 módulos; exportações de PII, papéis e login não geram evento; métricas por processo com cardinalidade ilimitada.	EST-10, SEC-13, SEC-17, EST-13
6	Configuração, segredos e supply chain Defaults públicos aceitos em produção, token de métricas efetivo é o default, CSP permissivo/sem TLS e CI sem scanners.	SEC-02, SEC-05, SEC-11, SEC-15, EST-16
7	N+1 e índices ausentes nos caminhos de leitura pesados Dashboard (2.514 queries), impacto/rastreio (62 queries por vínculo) e busca em <code>terminals</code> sem índice.	PERF-01, PERF-02, PERF-03, PERF-14

5. Categorias não aplicáveis à stack

Categoria	Por quê
Isolamento multi-tenant (RLS, <code>tenant_id</code> , filtro por organização)	Instância de organização única: nenhuma tabela possui coluna de tenant/dono (<code>grep -riE 'tenant org_id owner_id created_by' backend/app → 0</code>). Todas as tabelas (<code>users</code> , <code>sites</code> , <code>structures</code> , <code>devices</code> , <code>ports</code> , <code>cables</code> , <code>cable_segments</code> , <code>fibers</code> , <code>terminals</code> , <code>connections</code> , <code>customers</code> , <code>service_links</code> , <code>attachments</code> , <code>audit_events</code> , <code>async_jobs</code> ...) são globais por design. O controle é RBAC (SEC-03/SEC-04). IDOR 'horizontal' (acesso ao objeto de outro dono) não se aplica; avaliou-se IDOR vertical (permissão por objeto).
JWT (algoritmo, audience, expiração)	Não há JWT: sessões opacas persistidas no banco (OK-02).
E-mail, templates e HTML de e-mail (injeção)	Nenhum envio de e-mail nem template server-side (<code>grep -riE 'smtp jinja Template\'(' backend/app → 0</code>).
SSRF em URLs fornecidas pelo usuário	O backend não faz requisições HTTP de saída (<code>httpx</code> está declarado em <code>pyproject</code> mas nenhum módulo de <code>app/</code> o usa). Tiles do mapa são buscados pelo navegador (CSP limita os hosts).
Webhooks e pagamentos (idempotência)	Inexistentes. A idempotência de jobs de importação foi avaliada em EST-06.
Reset de senha por link/OTP e MFA	Inexistentes por design (ADR-0003: sem cadastro público; recuperação via CLI <code>bootstrap_admin --reset-password</code> , avaliada em SEC-08).
Cache de servidor (stampede, chave sem tenant, TTL)	Não há Redis/memcached nem cache de resposta; <code>lru_cache</code> só em <code>get_settings</code> . O cache existe no cliente (TanStack Query, OK-19). Ausência de cache no dashboard é tratada em PERF-01.
Goroutines, pool de conexões Go, event loop Node, lazy loading Hibernate	Stack Python/FastAPI/SQLAlchemy síncrono: equivalentes avaliados — código síncrono em <code>async def</code> (PERF-04/05), <code>pool/threadpool</code> (PERF-12), N+1 do ORM (PERF-01/02/10).
GraphQL / resolvers sem dataloader	API REST.
Cron duplicado entre réplicas	Não há cron externo; a limpeza periódica roda no loop do worker e é idempotente. O risco real é o oposto (worker não inicia — EST-01).
Migrações no startup com múltiplas réplicas	Migrações rodam num serviço one-shot <code>migrate</code> (OK-11), não no startup do backend.
Rotação de logs	Logs em <code>stdout/stderr</code> ; rotação delegada ao runtime do contêiner.
Sticky sessions	Sessões em banco; não há afinidade necessária (OK-17).

6. Achados detalhados por categoria

Tabelas: Severidade | Arquivo:linha | Descrição (PERF inclui gatilho de escala). O texto completo de cada achado — evidência com trecho, impacto, correção, validação e critérios de aceite — está na seção “Issues para o GitHub” (e em `issues.md`). Rótulos de prova: medido, reproduzido ou leitura (verificado por leitura, não medido).

6.1 Segurança (SEC)

Severid.	Arquivo:linha	Descrição
ALTA	backend/app/api/v1/ reports.py:35 backend/app/api/v1/ reports.py:51 backend/app/modules/reports/ service.py:280 (+2 âncoras em findings.json)	SEC-01 — Busca global e resumo do painel acessíveis sem autenticação dashboard/summary não declara nenhuma dependência de autenticação e search usa <code>get_optional_current_user</code> (anônimo permitido). Anônimos recebem sites, estruturas, cabos e dispositivos (inclui <code>serial_number</code>) por <code>ILIKE '%q%'</code> , além de contadores agregados (clientes, vínculos ativos, faixas de ocupação de CTOs). <i>Condições:</i> Sempre (nenhuma feature flag). GET <code>/api/v1/search</code> e GET <code>/api/v1/dashboard/summary</code> respondem 200 a qualquer cliente de rede que alcance a API; o Caddy publica <code>/api/*</code> (Caddyfile:19-21). Prova: medido
ALTA	backend/app/api/v1/ customers.py:30-31 backend/app/core/ permissions.py:33 backend/app/modules/reports/ service.py:305 (+1 âncoras em findings.json)	SEC-03 — Papel viewer lê dados pessoais de clientes: permissão <code>customers:*</code> nunca é exigida A matriz de permissões atribui <code>customers:read/write</code> apenas a <code>engineer/admin</code> , e a busca global usa <code>customers:read</code> para decidir se mostra clientes. Porém todas as rotas <code>/customers</code> e <code>/service-links</code> exigem apenas <code>network:read</code> (leitura) ou <code>network:write</code> (escrita); <code>customers:*</code> não é exigido por nenhuma rota. <i>Condições:</i> Qualquer conta autenticada (inclusive viewer, o papel mínimo). Sem feature flag. Prova: medido
ALTA	backend/app/api/v1/ reports.py:35 backend/app/modules/reports/ service.py:49 backend/app/db/ session.py:21 (+1 âncoras em findings.json)	SEC-16 — Cadeia: painel/busca anônimos + N+1 de 2.514 queries + pool pequeno = DoS sem credencial O endpoint anônimo executa 2 queries por CTO (2.514 statements com 10.001 estruturas). O backend roda 1 processo <code>uvicorn</code> e pool de 10+20 conexões, sem <code>statement_timeout</code> nem <code>rate limit</code> . Vinte requisições concorrentes levaram 16,9 s para concluir (<code>p50=16,8 s</code>), e o <code>readiness</code> (que precisa de conexão do pool) oscilou. <i>Condições:</i> Combina SEC-01 + PERF-01 + PERF-12. Basta alcançar <code>/api/v1/dashboard/summary</code> (Caddy publica <code>/api/*</code>). Prova: medido
MÉDIA	backend/app/core/ config.py:60 compose.yaml:49 backend/app/api/v1/ metrics.py:30 (+1 âncoras em findings.json)	SEC-02 — <code>/metrics</code> protegido por token padrão público e a variável <code>do compose</code> é ignorada <code>config.py</code> define um default fixo para <code>METRICS_SECRET_TOKEN</code> (dev-..., mascarado). O <code>compose.yaml</code> injeta <code>METRICS_TOKEN</code> (nome diferente), que <code>Settings</code> ignora (<code>extra="ignore"</code>), de modo que o default permanece ativo mesmo com <code>ENVIRONMENT=production</code> . <i>Condições:</i> Deploy via <code>compose.yaml</code> (padrão) ou qualquer deploy que não defina <code>METRICS_SECRET_TOKEN</code> . O Caddy encaminha <code>/api/*</code> (inclui <code>/api/v1/metrics</code>) para o backend. Prova: medido
MÉDIA	backend/app/api/v1/ attachments.py:84-87 backend/app/core/ permissions.py:18 backend/app/modules/attachments/ service.py:142 (+2 âncoras em findings.json)	SEC-04 — Anexos e trilha de auditoria ignoram a permissão da entidade dona (PII via technician) Anexos podem ser vinculados a <code>customer</code> e <code>service_link</code> ; GET <code>/attachments/{id}</code> e <code>/api/v1/attachments/{id}/download</code> só exigem <code>attachments:read</code> , sem consultar a permissão da entidade referenciada. GET <code>/audit-events</code> (<code>audit:read</code>) devolve <code>changes</code> — para <code>customer:updated</code> o JSON traz <code>phone</code> , <code>email</code> e <code>address</code> em claro. <i>Condições:</i> Papel <code>technician</code> (possui <code>attachments:read</code> e <code>audit:read</code> , mas não <code>customers:read</code>). Prova: verificado por leitura, não medido
MÉDIA	backend/app/core/ config.py:27-30 backend/app/core/ config.py:32 compose.yaml:8 (+3 âncoras em findings.json)	SEC-05 — Defaults de segredo públicos no código, no <code>compose</code> e no <code>.env.example</code>, sem validação de startup <code>SECRET_KEY</code> , <code>CSRF_SECRET</code> e <code>METRICS_SECRET_TOKEN</code> têm default em <code>config.py</code> ; <code>compose.yaml</code> repete <code>\${SECRET_KEY:-...}</code> , <code>\${CSRF_SECRET:-...}</code> e <code>\${POSTGRES_PASSWORD:-ftth...}</code> (senha padrão do banco). Não existe <code>model_validator</code> que rejeite defaults quando <code>is_production</code> . <i>Condições:</i> Deploy sem sobrescrever variáveis. O <code>compose</code> define <code>ENVIRONMENT default production</code> , mas <code>Settings</code> aceita os defaults mesmo assim. Prova: medido

Severid.	Arquivo:linha	Descrição
MÉDIA	backend/app/modules/identity/ service.py:56 backend/app/modules/identity/ service.py:62 backend/app/modules/identity/ service.py:98	SEC-06 — Rate limit de login permite travar qualquer conta (DoS de conta) e penaliza IP compartilhado check_login_rate_limit conta tentativas falhas nos últimos 15 min por IP ou e-mail ([]) e bloqueia com 429 antes de verificar a senha. Nenhum sucesso zera o contador. Atrás de NAT/CGNAT/LB (todos com o mesmo IP) 5 erros de qualquer usuário bloqueiam todos. Condições: Sempre. Basta conhecer o e-mail da vítima (ex.: admin). Prova: verificado por leitura, não medido
MÉDIA	backend/app/modules/identity/ service.py:174 backend/app/cli/ bootstrap_admin.py:79	SEC-08 — Troca de senha (e reset via CLI) não revoga as demais sessões change_user_password só grava o novo hash. As linhas de user_sessions permanecem válidas. O mesmo vale para bootstrap_admin --reset-password. Condições: Sessão roubada/vazada antes da troca de senha; TTL absoluto 7 dias, inatividade 24 h. Prova: verificado por leitura, não medido
MÉDIA	backend/app/core/ backup_restore.py:302 backend/app/core/ backup_restore.py:139 backend/app/core/ backup_restore.py:365 (+2 âncoras em findings.json)	SEC-10 — Restauração de backup extrai tar sem filtro e monta SQL com nomes vindos do arquivo tar.extractall(path=...) é chamado sem filter='data' em três pontos (path traversal / escrita arbitrária de arquivo). No fallback binário, f.stem de cada .bin extraído é interpolado em TRUNCATE TABLE {table_name} CASCADE (injeção de SQL). Condições: Operador restaura um arquivo de backup adulterado ou de origem não confiável (CLI scripts/restore.py). Os checksums estão dentro do próprio arquivo, portanto não autenticam. Prova: verificado por leitura, não medido
MÉDIA	Caddyfile:15 Caddyfile:3 SECURITY.md:46 (+2 âncoras em findings.json)	SEC-11 — CSP com 'unsafe-inline'/'unsafe-eval', proxy só em HTTP com cookie Secure e sem HSTS; documentação afirma o contrário O CSP do Caddy permite 'unsafe-inline' 'unsafe-eval' em script-src, o que anula a proteção contra XSS; SECURITY.md promete política 'estrita, bloqueando scripts inline'. O Caddy escuta só em :80 com auto_https off, mas o compose publica 443:443 e o runbook diz 'terminação TLS'. Condições: Deploy pelo compose.yaml + Caddyfile fornecidos. Prova: verificado por leitura, não medido
MÉDIA	backend/app/modules/exports/ service.py:62 backend/app/modules/exports/ service.py:68 backend/app/modules/exports/ service.py:169 (+3 âncoras em findings.json)	SEC-13 — Exportação de dados pessoais não é auditada, não expira e o download não revalida o papel (contradiz a UI) A UI afirma que a exportação de clientes é 'registrada permanentemente na trilha de auditoria', mas create_export_request não chama record_audit_event. O download exige só exports:read (engineer também), não confere o papel do solicitante nem a camada. Condições: Camada customers exportada por admin; arquivo permanece em disco; qualquer engineer/admin com o job_id baixa. Prova: verificado por leitura, não medido
MÉDIA	backend/app/modules/identity/ service.py:259 backend/app/modules/identity/ service.py:182 backend/app/api/v1/ reports.py:160	SEC-17 — Cadeia: sessão não revogada + ausência de auditoria de auth/usuários = persistência invisível Mudanças de papel, criação/desativação de usuários, login, logout e troca de senha não geram AuditEvent (só login_attempts). Sem revogação na troca de senha (SEC-08), o invasor mantém a sessão e pode, com papel admin, criar contas ou elevar papéis (PATCH /users/{id}) sem deixar trilha na auditoria da aplicação. Condições: Combina SEC-08 + EST-10. Requer uma sessão comprometida. Prova: verificado por leitura, não medido
BAIXA	backend/app/modules/identity/ service.py:89-91	SEC-07 — Login revela contas existentes porém desativadas (403 antes de verificar a senha) authenticate_user retorna 403 user_deactivated para conta inativa antes de comparar a senha; contas inexistentes/ativas retornam 401 invalid_credentials. Condições: Sempre; requer conhecer/adivinhar e-mails. Prova: verificado por leitura, não medido

Severid.	Arquivo:linha	Descrição
BAIXA	backend/app/core/ dependencies.py:87 backend/app/core/ security.py:58-60	SEC-09 — Validação de Origin do CSRF usa startswith (bypass por sufixo) e o token CSRF não é vinculado à sessão validate_csrf aceita a origem se origin.startswith(allowed) para qualquer item de CORS_ORIGINS. Além disso o CSRF é double-submit sem assinatura (o CSRF_SECRET não é usado): quem controlar um cookie no domínio (subdomínio vizinho) escolhe o par válido. Condições: Origem http://localhost:3000.evil.example (ou qualquer domínio que comece com uma origem permitida). Mitigado por SameSite=Lax do cookie de sessão e pela exigência do par cookie/cabeçalho. Prova: medido
BAIXA	backend/scripts/ seed_demo.py:186 backend/scripts/ seed_demo.py:176	SEC-12 — seed_demo cria administrador com credencial fixa pública, sem guarda de ambiente O seed cria admin@provedor... com senha literal do repositório (mascarada: Admi...). Não há checagem de is_production/URL do banco. Condições: Operador executa scripts/seed_demo.py contra um banco de produção (o script é opt-in, mas não verifica ENVIRONMENT). Prova: verificado por leitura, não medido
BAIXA	backend/app/modules/jobs/ service.py:325 backend/app/modules/jobs/ service.py:48 backend/app/api/v1/ imports_exports.py:188	SEC-14 — GET /jobs/{id} expõe error_message cru (str(exception)) a qualquer usuário autenticado O worker grava job.error_message = str(e) e get_job_by_id o devolve. Exceções SQLAlchemy incluem SQL, parâmetros e caminhos de arquivo. A rota só exige get_current_user (viewer incluso). Condições: Job falha (ex.: IntegrityError na importação); qualquer papel autenticado que conheça o UUID do job. Prova: verificado por leitura, não medido
BAIXA	frontend/ package.json:22 frontend/ pnpm-lock.yaml:2031	SEC-15 — Dependências do frontend com advisories (maplibre-gl, postcss via next, vitest) maplibre-gl@5.24.0 é vulnerável a bypass do sanitizador de HTML (DOM.sanitize), usado por Popup.setHTML; grep -rE 'Popup setHTML setText' frontend/src → 0 usos. As de postcss vivem no toolchain do Next (build), e as de vitest são dev-only. Condições: pnpm audit (2026-09-18): 7 advisories — 1 crítica (maplibre-gl ≤6.4.0, sanitizador do Popup), 2 altas (postcss ≤8.5.17/8.5.11, em build via next), 4 moderadas. Prova: medido
BAIXA	backend/app/ main.py:61 backend/app/core/ dependencies.py:23 backend/app/modules/identity/ models.py:64	SEC-18 — Cabeçalho X-Forwarded-For confiável de qualquer origem (ProxyHeaders '*' + get_client_ip) ProxyHeadersMiddleware(trusted_hosts=["*"]) e get_client_ip usam o primeiro item de X-Forwarded-For sem allowlist de proxies. O valor vai para login_attempts.ip_address (VARCHAR(45)) e para user_sessions. Condições: Backend alcançável sem passar pelo Caddy (ex.: compose.override.yaml.example publica 127.0.0.1:8000; ou LB à frente que não sobrescreve o cabeçalho). Prova: verificado por leitura, não medido

6.2 Estrutura (EST)

Severid.	Arquivo:linha	Descrição
ALTA	backend/scripts/ run_worker.py:20 backend/scripts/ run_worker.py:59 backend/app/modules/jobs/ service.py:281 (+2 âncoras em findings.json)	EST-01 — O worker de jobs não inicia: ImportError em scripts/run_worker.py (importações e exportações nunca são processadas) run_worker.py importa process_claimed_job de app.modules.jobs.service, mas a função existente chama-se process_next_job (e faz o claim internamente). O processo termina com ImportError e o compose o reinicia em loop (restart: unless-stopped) sem healthcheck. Condições: Sempre que o serviço worker do compose sobe (comando python scripts/run_worker.py). Os testes passam porque usam process_next_job, não o script. Prova: medido

Severid.	Arquivo:linha	Descrição
ALTA	backend/app/modules/jobs/ service.py:239 backend/app/modules/jobs/ service.py:241-247 backend/app/modules/cables/ models.py:24 (+2 âncoras em findings.json)	EST-05 — Importação descarta a geometria dos cabos (cria Cable sem segmento) e o CSV inventa coordenadas execute_import_commit valida coords do cabo mas cria apenas Cable(...): a geometria vive em CableSegment.geometry (a tabela cables não tem geometria) e nenhum segmento é criado. O contador created_cables informa sucesso. Condições: Todo import_commit com entidades cable (GeoJSON/KML/CSV) após corrigir o worker (EST-01). Prova: verificado por leitura, não medido
ALTA	backend/app/schemas/ cables.py:22 backend/app/modules/cables/ service.py:136 backend/app/api/v1/ attachments.py:66 (+3 âncoras em findings.json)	EST-11 — Sem tetos de tamanho e sem rate limit fora do login: fiber_count ilimitado derruba o processo com 1 request CableCreate.fiber_count tem ge=1 e nenhum le: create_cable instancia uma linha ORM Fiber por fibra numa única transação. Também: await file.read() lê todo o upload em memória antes de checar o limite; não há request_body max_size no Caddy nem limite no uvicorn; Condições: Papel engineer/admin (network:write) — ou sessão comprometida. Body sem limite no Caddy/uvicorn; uploads lidos por inteiro. Prova: medido
ALTA	frontend/ Dockerfile:25	EST-15 — Build do frontend falha: Dockerfile copia /app/public, diretório que não existe O estágio runner faz COPY --from=builder /app/public ./public, mas frontend/public não existe e não há arquivos rastreados nele (git ls-files frontend grep -c '^frontend/public' → 0). O Next.js não cria public. Condições: docker compose build frontend / up --build a partir do repositório. Prova: reproduzido
MÉDIA	backend/app/api/v1/ splitters.py:9 backend/app/api/v1/ settings.py:8 backend/app/api/v1/ inventory.py:269 (+2 âncoras em findings.json)	EST-02 — Rotas-stub 501 sem autenticação (allow-by-default): splitters, settings e occupancy 9 handlers ficam sem Depends(get_current_user/require_permission) e sem validate_csrf: 5 de /splitters, 2 de /settings, structures/{id}/occupancy (+ dashboard, SEC-01). Ao trocar pending_endpoint() pela implementação, a rota nasce pública. Condições: Hoje respondem 501. O risco materializa-se quando a implementação for preenchida (o contrato OpenAPI já as expõe sem security). Prova: medido
MÉDIA	backend/app/modules/inventory/ service.py:38 backend/app/modules/inventory/ service.py:145 backend/app/modules/inventory/ service.py:160 (+2 âncoras em findings.json)	EST-03 — Concorrência otimista (If-Match) é check-then-act: dois PATCH com a mesma versão passam (lost update) _validate_if_match compara current_version lida em Python e só depois o handler altera e faz version += 1; o UPDATE não tem WHERE version = :esperada e a linha não é bloqueada (with_for_update). VersionedModelMixin.version é uma coluna comum (sem version_id_col). Condições: Edições concorrentes do mesmo recurso (usuário, site, estrutura, dispositivo, porta, cabo, cliente, medição, perfil óptico). Prova: verificado por leitura, não medido
MÉDIA	backend/app/modules/cables/ service.py:576 backend/app/modules/cables/ service.py:591 backend/app/modules/cables/ service.py:788 (+2 âncoras em findings.json)	EST-04 — Divisão de segmento de cabo sem lock, sem If-Match e sem revisão topológica esperada split_cable_segment lê o segmento sem FOR UPDATE, cria dois novos segmentos com todos os terminais e conexões, e ao final db.delete(segment). O handler não recebe If-Match nem expected_topology_revison (ao contrário do editor de fusão, que trava a estrutura e valida a revisão). Condições: Dois usuários dividem o mesmo segmento ao mesmo tempo (ou duplo clique/reenvio). Prova: verificado por leitura, não medido
MÉDIA	backend/app/modules/jobs/ service.py:131 backend/app/modules/jobs/ service.py:96 backend/app/modules/jobs/ service.py:145 (+1 âncoras em findings.json)	EST-07 — Lease do job nunca é renovada: heartbeat() é código morto e importações longas podem ser reexecutadas por outro worker claim_next_job cria lease de 60 s; heartbeat() está definido mas nunca é chamado (grep -rn 'heartbeat(' backend → só a definição), e execute_import_commit não renova a lease. Qualquer outro worker reivindica jobs running com lease_expires_at < now e incrementa retry_count. Condições: 2+ workers e job com duração > 60 s (importação de arquivos grandes: ~2 round-trips por entidade, ver PERF-05). Prova: verificado por leitura, não medido

Severid.	Arquivo:linha	Descrição
MÉDIA	backend/app/modules/imports/ service.py:33 backend/app/modules/exports/ service.py:28 backend/app/core/ config.py:51 (+1 âncoras em findings.json)	EST-08 — Importações/exportações ignoram STORAGE_PATH: lêem settings.STORAGE_DIR, atributo que não existe getattr(settings, "STORAGE_DIR", "storage") sempre devolve o fallback relativo storage (a config define STORAGE_PATH). result_path e file_storage_path gravados no banco ficam relativos ao CWD do processo. Condições: Sempre; funciona no compose apenas porque WORKDIR=/app faz o caminho relativo storage/... coincidir com o volume /app/storage. Prova: verificado por leitura, não medido
MÉDIA	backend/app/modules/attachments/ service.py:215 backend/app/modules/attachments/ service.py:263 backend/app/modules/attachments/ service.py:433 (+1 âncoras em findings.json)	EST-09 — Upload grava em disco antes do commit; falhas geram arquivos órfãos e o reconciliador pode apagar upload em andamento save_attachment escreve o original (e a miniatura) antes de db.commit(). Qualquer exceção posterior deixa o arquivo sem registro. reconcile_storage_orphans remove todo arquivo não presente no snapshot lido do banco, incluindo o de um upload que ainda não commitou. Condições: Exceção após a escrita (ex.: DecompressionBombError, falha no commit) ou reconcile-orphans executado durante um upload. Prova: medido
MÉDIA	backend/app/modules/audit/ service.py:45 backend/app/api/v1/ reports.py:160 backend/app/modules/identity/ service.py:204 (+1 âncoras em findings.json)	EST-10 — Trilha de auditoria cobre só anexos, conectividade, clientes e importação; usuários, inventário, cabos, óptico e exportações ficam de fora record_audit_event é usado apenas em attachments, connectivity, customers e jobs. Não há eventos para criar/alterar/desativar usuários e papéis, login/logout/troca de senha, CRUD de sites/estruturas/dispositivos/portas/cabos, perfis/medições ópticas, settings e exportações. Condições: Sempre. GET /audit-events é apresentada como 'histórico de mutações e ações de usuários no sistema'. Prova: medido
MÉDIA	backend/app/core/ middleware.py:37 backend/app/core/ metrics.py:30 backend/app/core/ metrics.py:74 (+1 âncoras em findings.json)	EST-13 — Métricas em memória com cardinalidade ilimitada: paths 404 distintos crescem o dicionário sem limite (e são por-processo) RequestIDMiddleware registra request.url.path quando a rota não foi resolvida (route_format=None); normalize_route_path só troca UUIDs e números por {id}. Cada path 404 diferente cria chaves novas em _http_requests e _http_durations que nunca são removidas. Condições: Qualquer cliente (não requer autenticação): requisições a caminhos inexistentes. Prova: medido
MÉDIA	backend/ Dockerfile:59 compose.yaml:39 compose.yaml:61 (+1 âncoras em findings.json)	EST-14 — SPOF e estado local: 1 processo uvicorn, 1 backend com container_name fixo, storage em volume local, métricas por processo CMD uvicorn sem --workers; container_name: ftth_backend impede --scale; anexos/exportações vivem em volume Docker local (backend_storage) compartilhado apenas no mesmo host; métricas em dicionário do processo (EST-13); PostgreSQL único sem réplica/backup automático (há scripts/backup.sh manual); Condições: Ao tentar escalar (2+ réplicas/hosts) ou em falha de um único container. Prova: verificado por leitura, não medido
MÉDIA	.github/workflows/ ci.yml:76 .github/workflows/ ci.yml:115 .github/workflows/ ci.yml:45 (+1 âncoras em findings.json)	EST-16 — CI sem varredura de dependências, segredos ou SAST, sem build de imagens e sem permissions: mínimas O workflow roda ruff, mypy, pytest, restore drill, eslint, tsc e vitest, mas nenhum pip-audit/pnpm audit, secret scan, CodeQL/Semgrep, docker build nem trivy. Actions são fixadas por tag (@v4), não por SHA, e o workflow não declara permissions:. Condições: Todo push/PR. Prova: verificado por leitura, não medido
MÉDIA	backend/app/core/ backup_restore.py:102 backend/app/core/ backup_restore.py:118 backend/app/core/ backup_restore.py:270	EST-19 — Backup por fallback psycopg não é snapshot consistente e o arquivo é gravado sem criptografia O fallback copia tabela a tabela com COPY ... TO STDOUT numa conexão em READ COMMITTED (sem REPEATABLE READ/pg_export_snapshot), então tabelas ficam em instantes diferentes; os anexos são arquivados depois, sem congelamento. O .tar.gz final contém hashes de senha e PII de clientes em texto claro (sem criptografia). Condições: Ambiente sem pg_dump no PATH (fallback), ou operação com escritas concorrentes; backups armazenados fora do host. Prova: verificado por leitura, não medido

Severid.	Arquivo:linha	Descrição
BAIXA	backend/app/modules/imports/ service.py:602 backend/app/modules/imports/ models.py:43	EST-06 — Idempotência do commit de importação: check-then-insert e chave global sem escopo O handler consulta AsyncJob.idempotency_key e só depois insere; a corrida cai na UNIQUE como IntegrityError não tratado (HTTP 500). A chave não é escapada por usuário/import_id: reutilizá-la devolve o job_id de outra importação. Sem limite de tamanho (coluna VARCHAR(255) → 500). <i>Condições:</i> Duplo clique/retry concorrente com a mesma Idempotency-Key; ou chave reutilizada em outra importação. Prova: verificado por leitura, não medido
BAIXA	backend/app/api/v1/ customers.py:82 backend/app/modules/topology/ service.py:105 backend/app/modules/customers/ service.py:285 (+2 âncoras em findings.json)	EST-12 — UUID inválido em path/query/body resulta em HTTP 500 (ValueError não tratado) e strings sem max_length quebram no banco Nos routers de clientes, atendimentos e conectividade, uuid.UUID(param) é chamado sem try/except (17 pontos, 14 rotas). topology/trace e create_service_link fazem o mesmo. CustomerUpdate.phone/email/address não têm max_length enquanto as colunas são VARCHAR(30/100/255) → DataError → 500. <i>Condições:</i> Qualquer chamada autenticada com customer_id=abc etc. Prova: medido
BAIXA	backend/app/core/ config.py:61 backend/app/core/ config.py:59 backend/app/schemas/ topology.py:34	EST-17 — Configurações declaradas e nunca usadas: SECRET_KEY, CSRF_SECRET, METRICS_ENABLED, DEBUG, MAX_TRACE_HOPS grep mostra que SECRET_KEY, CSRF_SECRET, METRICS_ENABLED e MAX_TRACE_HOPS não são lidos por nenhum código (apenas DEBUG vai a um log). O teto do rastreio óptico vem do cliente (TraceRequest.max_hops ≤ 500), não da config. <i>Condições:</i> Sempre. Prova: medido
BAIXA	backend/scripts/ run_worker.py:25 backend/app/core/ metrics.py:92	EST-18 — Worker sem logs JSON/correlation-id, sem healthcheck e sem métricas de jobs run_worker.py usa logging.basicConfig com formato pseudo-JSON (quebra com aspas na mensagem), sem request_id/job_id de contexto; metrics_collector.record_job nunca é invocado; worker não tem healthcheck no compose; erros do job viram texto em error_message. <i>Condições:</i> Sempre. Prova: medido
BAIXA	backend/app/core/ permissions.py:4 frontend/src/lib/permissions/ rbac.ts:35 frontend/src/lib/permissions/ rbac.ts:49 (+1 âncoras em findings.json)	EST-20 — Matriz de permissões duplicada em backend e frontend (com divergência) e gates de UI inexistentes A matriz existe em permissions.py e em rbac.ts (cópias manuais; telemetry.write só no frontend). AuthGuard, PermissionGate e o campo permission da navegação nunca são usados: a UI mostra ações que o servidor recusará (UX) e não há fonte única. <i>Condições:</i> Sempre. Prova: medido
BAIXA	backend/app/modules/inventory/ service.py:38 backend/app/modules/customers/ service.py:31 backend/app/modules/connectivity/ service.py:42 (+4 âncoras em findings.json)	EST-21 — Lógica crítica duplicada: 6 cópias da validação de If-Match e regra/consulta dentro de handlers _validate_if_match existe copiado em inventory, customers, connectivity, measurements e optical, e _check_optimistic_lock em cables (mensagens diferentes). Além disso o handler download_export consulta o banco, valida estado do job, resolve content-type e caminho de arquivo; <i>Condições:</i> Sempre; amplifica EST-03 (a correção precisa ser replicada em 6 módulos). Prova: medido

6.3 Performance (PERF)

Severid.	Arquivo:linha	Descrição
ALTA	backend/app/modules/reports/ service.py:48 backend/app/modules/reports/ service.py:49 backend/app/modules/reports/ service.py:57	PERF-01 — Dashboard: 2 queries por CTO (N+1) — 2.514 statements e ~1 s com 10 mil estruturas, sem autenticação calculate_dashboard_summary carrega todas as CTOs e, dentro do for cto in ctos, executa SELECT ports e SELECT service_links por CTO; as contagens poderiam ser um único GROUP BY. O endpoint não exige autenticação (SEC-01). <i>Condições:</i> Cada requisição a GET /api/v1/dashboard/summary; cresce com o nº de CTOs não aposentadas. <i>Gatilho de escala:</i> Dói desde ~1 s/request em 10 mil estruturas (medido); a partir de dezenas de requisições/s (dashboard abre a cada carga de UI) satura 1 worker. Prova: medido

Severid.	Arquivo:linha	Descrição
ALTA	backend/app/modules/topology/ service.py:526 backend/app/modules/topology/ service.py:541 backend/app/modules/topology/ service.py:587 (+1 âncoras em findings.json)	PERF-02 — Análise de impacto: carrega todos os clientes e roda um rastreo óptico completo por vínculo ativo (62 statements por vínculo) analyze_cable_impact faz select(Customer) de toda a base, e para cada vínculo ativo executa consultas de CTO/ONU/terminal e chama trace_optical_path (BFS com várias queries por salto). cable_segment_ids não tem max_length. Condições: POST /api/v1/topology/impact (papel viewer basta). Cresce com nº de clientes com vínculo ativo x saltos. Gatilho de escala: Custo proporcional a C (clientes ativos): medido 62 statements e ~80-104 ms para 1 vínculo; com C=1.000 ≈ 62.000 statements (extrapolação linear, não medida). Prova: medido
ALTA	backend/app/api/v1/ attachments.py:46 backend/app/api/v1/ attachments.py:66 backend/app/modules/attachments/ service.py:121 (+1 âncoras em findings.json)	PERF-04 — Upload de anexo: handler async executa I/O e Pillow síncronos no event loop e uma PNG de 100 KiB consome +339 MiB upload_attachment é async def, lê o corpo inteiro (await file.read()) e chama save_attachment sincronamente (Pillow convert('RGB') + thumbnail(LANCZOS) + WEBP method=6, escrita em disco e commit) — bloqueando o event loop do único processo. O limite de 10 MiB é do arquivo comprimido; Condições: POST /api/v1/attachments por qualquer technician/engineer/admin (ou sessão comprometida); PNG grande com poucos KiB. Gatilho de escala: 1 upload de 100 KiB já causa +339 MiB (medido); 4-5 simultâneos ≈ 1,5 GiB (extrapolação linear). Prova: medido
MÉDIA	backend/app/modules/connectivity/ models.py:46 backend/app/modules/connectivity/ models.py:57 backend/migrations/versions/ 0006_connectivity_engine.py:31 (+1 âncoras em findings.json)	PERF-03 — Falta índice em terminals(entity_type, entity_id): buscas do rastreo e do impacto fazem Seq Scan em 206 mil linhas A coluna entity_id foi adicionada na migração 0006 sem índice e o modelo declara apenas idx_terminals_kind_structure e idx_terminals_occupancy. O EXPLAIN ANALYZE mostra Parallel Seq Scan on terminals. Condições: Todas as consultas Terminal.entity_type == 'port' AND Terminal.entity_id = ... (rastreo, impacto, orçamento óptico, medições). Gatilho de escala: Custo cresce com o nº de terminais (2 por fibra); medido 12-14 ms/lookup com 206.428 linhas. Prova: medido
MÉDIA	backend/app/api/v1/ imports_exports.py:42 backend/app/modules/imports/ service.py:457 backend/app/modules/jobs/ service.py:81 (+1 âncoras em findings.json)	PERF-05 — Importação: parse síncrono de até 20 MB dentro de handler async, tudo em memória, e 1 SELECT de cancelamento por entidade preview_import é async def e chama create_import_preview sincronamente (json.loads de até 20 MB, lista de dicts com todas as feições, hash, escrita em disco, commit) no event loop. No worker, execute_import_commit faz db.refresh(job) (1 SELECT) e flush() (1 INSERT) por entidade e relê o arquivo inteiro. Condições: POST /imports/preview com arquivos grandes; commit no worker. Gatilho de escala: Arquivos > alguns MB (limite 20 MB); commit com dezenas de milhares de entidades. Prova: verificado por leitura, não medido
MÉDIA	backend/app/modules/reports/ service.py:205 backend/app/modules/inventory/ service.py:85 backend/app/modules/customers/ service.py:85 (+1 âncoras em findings.json)	PERF-06 — Buscas por ILIKE '%q%' sem pg_trgm/GIN em sites, estruturas, cabos, dispositivos, clientes, usuários e notas de portas Todas as consultas de texto usam %termo%, que não usa índices B-tree; as migrações não criam pg_trgm nem índices GIN. O termo também não escapa %/_ (curinga). Condições: Busca global, listagens com q e relatórios (Port.notes ILIKE '%danificad%'); cresce com o tamanho das tabelas. Gatilho de escala: Medido 4,6 ms em 10.001 estruturas (Seq Scan, 10.001 linhas removidas); dor esperada em ordens de 10 ⁵ linhas (linear; não medido). Prova: medido

Severid.	Arquivo:linha	Descrição
MÉDIA	backend/app/modules/exports/ service.py:96 backend/app/modules/exports/ service.py:140 backend/app/modules/exports/ service.py:337	PERF-08 — Exportações carregam camadas inteiras em memória (.all()) e serializam JSON com indent generate_*_export faz db.scalars(select(Site)).all(), select(Structure), select(CableSegment), select(Customer) sem streaming e monta a lista completa de features antes de json.dump(..., indent=2)/ET.tostring. Condições: export_* em redes grandes (após corrigir o worker — EST-01). Gatilho de escala: Dezenas de milhares de segmentos/estruturas (não medido). Prova: verificado por leitura, não medido
MÉDIA	backend/app/db/ session.py:21 backend/app/core/ config.py:40 backend/ Dockerfile:59	PERF-12 — Um único processo uvicorn, pool de 10+20 conexões e nenhum statement_timeout/timeout de request CMD uvicorn sem --workers; handlers síncronos rodam na threadpool padrão (40 threads) sobre um pool de 30 conexões; create_engine não define connect_args (sem statement_timeout/connect_timeout) e não há timeout de requisição. Um endpoint lento retém conexões por segundos. Condições: Carga concorrente moderada ou 1 endpoint lento. Gatilho de escala: Medido: 20 requisições concorrentes ao dashboard → todas concluem ~17 s; readiness saltou para 1,1 s. Prova: medido
MÉDIA	backend/app/modules/topology/ service.py:181 backend/app/modules/topology/ service.py:224 backend/app/modules/topology/ service.py:268 (+2 âncoras em findings.json)	PERF-14 — Rastreo óptico: ≥ 5 queries por salto e cópia de listas por ramo; max_hops até 500 e max_results até 200 vindos do cliente Cada iteração do BFS executa db.get(Terminal), db.scalars(FiberSegment), db.scalars(Connection), db.scalars(InternalEdge), splitters e checagem de endpoint; ramos copiam current_steps + [next_step] (O(H²) por caminho). O teto configurado MAX_TRACE_HOPS=300 não é aplicado (EST-17). Condições: POST /topology/trace, /optical/budgets, /optical/simulations (papel viewer) e o impacto (PERF-02). Gatilho de escala: Topologias com splitters em cascata e milhares de ONUs por PON (não medido). Prova: verificado por leitura, não medido
BAIXA	backend/app/modules/inventory/ service.py:90 backend/app/modules/inventory/ service.py:94 backend/app/db/ base.py:18	PERF-07 — Listagens paginadas com COUNT(*) + OFFSET + ORDER BY created_at sem índice de suporte As listagens fazem count(*) completo a cada página e ORDER BY created_at DESC, id OFFSET n; created_at não é indexado nas tabelas de inventário (só em auditoria/login). OFFSET profundo descarta linhas ordenadas. Condições: Tabelas grandes (estruturas, cabos, medições, auditoria) e páginas profundas. Gatilho de escala: Não dói hoje (5,9 ms @10.001); gatilho esperado > 10⁵ linhas. Prova: medido
BAIXA	backend/app/modules/gis/ service.py:36 backend/app/modules/cables/ service.py:790	PERF-09 — Linha única network_topology_state atualizada em toda escrita de topologia serializa transações concorrentes bump_topology_revision executa UPDATE network_topology_state ... WHERE id = 1 dentro de cada transação de topologia; o lock de linha só é liberado no COMMIT. É correto para o contador monotônico (ponto forte), mas serializa os commits. Condições: Edição concorrente por vários usuários (fusão, divisão, importação, conexões). Gatilho de escala: Não medido; relevante com muitos editores simultâneos. Prova: verificado por leitura, não medido
BAIXA	backend/app/modules/identity/ service.py:124 backend/app/core/ dependencies.py:149 backend/app/modules/identity/ service.py:154	PERF-10 — Cada requisição autenticada faz 2 SELECTs (sessão e usuário por lazy load) e pode gravar last_activity_at get_active_session_by_token faz select(UserSession).join(User) só para filtrar; user_session.user dispara um segundo SELECT (lazy). A cada 60 s de atividade há UPDATE + COMMIT dentro de um GET. Condições: Todas as rotas autenticadas. Gatilho de escala: Proporcional ao tráfego total. Prova: verificado por leitura, não medido

Severid.	Arquivo:linha	Descrição
BAIXA	backend/app/modules/jobs/ service.py:332 backend/app/modules/identity/ models.py:58 backend/app/modules/identity/ models.py:27	PERF-11 — Sem retenção: login_attempts, user_sessions expiradas, audit_events e arquivos de exportação crescem indefinidamente A única rotina de limpeza (clean_expired_previews_and_exports) remove apenas ImportPreview; login_attempts (consultada a cada login com OR), sessões revogadas/expiradas e arquivos de exportação nunca são removidos. Condições: Operação prolongada. Gatilho de escala: Meses de operação/alto volume de logins. Prova: verificado por leitura, não medido
BAIXA	backend/app/db/ health.py:46 backend/app/api/v1/ health.py:39 compose.yaml:55	PERF-13 — /health/ready lê o diretório do Alembic do disco e usa uma conexão do pool a cada sonda (10 s) check_database_migrations chama ScriptDirectory.from_config(...) (parse de arquivos de migração) a cada readiness; a rota depende de get_db (checkout no pool). Condições: Sempre; agrava-se sob saturação do pool (PERF-12). Gatilho de escala: Sondas a cada 10 s (compose); ruim sob pool esgotado. Prova: medido

7. Top 10 gargalos de performance

Ordenados por impacto × probabilidade (rank definido pela combinação de severidade, frequência de uso do caminho e evidência medida).

#	Severid.	Gargalo	Gatilho de escala	Crescimento	Prova
1	ALTA	PERF-01 Dashboard: 2 queries por CTO (N+1) — 2.514 statements e ~1 s com 10 mil estruturas, sem autenticação	Dói desde ~1 s/request em 10 mil estruturas (medido); a partir de dezenas de requisições/s (dashboard abre a cada carga de UI) satura 1 worker.	O(C) queries por request (C = nº de CTOs); 2 statements por CTO.	medido
2	ALTA	PERF-02 Análise de impacto: carrega todos os clientes e roda um rastreio óptico completo por vínculo ativo (62 statements por vínculo)	Custo proporcional a C (clientes ativos): medido 62 statements e ~80-104 ms para 1 vínculo; com C=1.000 ≈ 62.000 statements (extrapolação linear, não medida).	O(C × H × q): C clientes ativos, H saltos do trajeto, q≈5 queries/salto.	medido
3	ALTA	PERF-04 Upload de anexo: handler async executa I/O e Pillow síncronos no event loop e uma PNG de 100 KiB consome +339 MiB	1 upload de 100 KiB já causa +339 MiB (medido); 4-5 simultâneos ≈ 1,5 GiB (extrapolação linear).	O(W×H) memória e CPU por upload, independente do tamanho do arquivo comprimido.	medido
4	MÉDIA	PERF-12 Um único processo uvicorn, pool de 10+20 conexões e nenhum statement_timeout/timeout de request	Medido: 20 requisições concorrentes ao dashboard → todas concluem ~17 s; readiness saltou para 1,1 s.	Fila: latência ≈ N × tempo de serviço em 1 worker (GIL).	medido
5	MÉDIA	PERF-03 Falta índice em terminals(entity_type, entity_id): buscas do rastreio e do impacto fazem Seq Scan em 206 mil linhas	Custo cresce com o nº de terminais (2 por fibra); medido 12-14 ms/lookup com 206.428 linhas.	O(T) por lookup (T = terminais) → O(T·N) por request.	medido
6	MÉDIA	PERF-14 Rastreio óptico: ≥ 5 queries por salto e cópia de listas por ramo; max_hops até 500 e max_results até 200 vindos do cliente	Topologias com splitters em cascata e milhares de ONUs por PON (não medido).	O(P·H·q) queries; O(P·H²) memória de steps.	por leitura, não medido
7	MÉDIA	PERF-05 Importação: parse síncrono de até 20 MB dentro de handler async, tudo em memória, e 1 SELECT de cancelamento por entidade	Arquivos > alguns MB (limite 20 MB); commit com dezenas de milhares de entidades.	O(N) memória e O(2N) round-trips.	por leitura, não medido
8	MÉDIA	PERF-06 Buscas por ILIKE '%q%' sem pg_trgm/GIN em sites, estruturas, cabos, dispositivos, clientes, usuários e notas de portas	Medido 4,6 ms em 10.001 estruturas (Seq Scan, 10.001 linhas removidas); dor esperada em ordens de 10 ⁵ linhas (linear; não medido).	O(N) por tabela por busca.	medido
9	MÉDIA	PERF-08 Exportações carregam camadas inteiras em memória (.all()) e serializam JSON com indent	Dezenas de milhares de segmentos/estruturas (não medido).	O(N) memória.	por leitura, não medido
10	BAIXA	PERF-07 Listagens paginadas com COUNT(*) + OFFSET + ORDER BY created_at sem índice de suporte	Não dói hoje (5,9 ms @10.001); gatilho esperado > 10 ⁵ linhas.	O(N log N) por página (sort) e O(N) para COUNT.	medido

8. Matriz esforço × impacto (todos os achados)

Colunas: esforço de correção (baixo → alto). Linhas: ganho de risco/desempenho (alto → baixo). Chips coloridos pela severidade.

	Esforço baixo	Esforço médio	Esforço alto
Impacto alto	Ganhos rápidos (6) EST-01EST-15PERF-01PERF-04 SEC-01SEC-03	Projetos prioritários (3) EST-05EST-11SEC-16	Investimentos estratégicos (1) PERF-02
Impacto médio	Fazer junto (8) EST-02EST-08EST-13PERF-03 PERF-06SEC-02SEC-05SEC-08	Planejar (12) EST-03EST-04EST-07EST-16 PERF-05PERF-12SEC-04SEC-06 SEC-10SEC-11SEC-13SEC-17	Avaliar (3) EST-10EST-14PERF-14
Impacto baixo	Preencher (14) EST-06EST-12EST-17EST-18 EST-21PERF-07PERF-10PERF-11 PERF-13SEC-07SEC-09SEC-12 SEC-14SEC-18	Backlog (6) EST-09EST-19PERF-08EST-20 PERF-09SEC-15	Evitar/rever (0)

9. Recomendações priorizadas

Onda 1 — imediato (dias)

Fechar acessos indevidos e quebras de deploy/operação de baixo esforço.

ALTA	EST-01	O worker de jobs não inicia: ImportError em scripts/run_worker.py (importações e exportações nunca são processadas) Esforço baixo · impacto alto — Alinhar a API (<code>process_claimed_job(db, job, worker_id)</code> extraída de <code>process_next_job</code>) ou ajustar o script; adicionar teste que importa <code>scripts.run_worker</code> e <code>healthcheck</code> do worker (<code>heartbeat</code> em tabela/arquivo).
ALTA	EST-11	Sem tetos de tamanho e sem rate limit fora do login: <code>fiber_count</code> ilimitado derruba o processo com 1 request Esforço médio · impacto alto — <code>le=</code> em <code>fiber_count/tube_count</code> (ex.: 1728/144), <code>max_length</code> em listas e textos; <code>request_body { max_size }</code> no Caddy e streaming com limite no upload; rate limit por IP/usuário nos endpoints caros.
ALTA	EST-15	Build do frontend falha: Dockerfile copia <code>/app/public</code> , diretório que não existe Esforço baixo · impacto alto — Criar <code>frontend/public/.gitkeep</code> ou remover a linha <code>COPY ... /app/public</code> (ou torná-la condicional). Adicionar <code>docker build</code> ao CI.
ALTA	PERF-01	Dashboard: 2 queries por CTO (N+1) — 2.514 statements e ~1 s com 10 mil estruturas, sem autenticação Esforço baixo · impacto alto — Uma consulta agregada (<code>COUNT ... GROUP BY structure_id</code> com <code>LEFT JOIN</code> de vínculos ativos) e cache de 15-30 s (invalidação pela <code>topology_revision</code>).
ALTA	PERF-04	Upload de anexo: handler <code>async</code> executa I/O e Pillow síncronos no event loop e uma PNG de 100 KiB consome +339 MiB Esforço baixo · impacto alto — Declarar o handler <code>def (threadpool)</code> ou <code>run_in_threadpool</code> ; validar dimensões antes de decodificar (<code>Image.open</code> → checar <code>size</code>) com teto ~25 Mpx e capturar <code>DecompressionBombError</code> ; ler o corpo em streaming com limite.
ALTA	SEC-01	Busca global e resumo do painel acessíveis sem autenticação Esforço baixo · impacto alto — Exigir <code>Depends(require_permission("reports:read"))</code> (dashboard) e <code>map:read/network:read</code> (busca); remover <code>get_optional_current_user</code> de <code>/search</code> e decidir por permissão granular quais grupos retornar.
ALTA	SEC-03	Papel viewer lê dados pessoais de clientes: permissão <code>customers:*</code> nunca é exigida Esforço baixo · impacto alto — Trocar <code>network:read/write</code> por <code>customers:read/write</code> em todas as rotas de clientes e vínculos; adicionar teste de matriz papel×rota.
ALTA	SEC-16	Cadeia: painel/busca anônimos + N+1 de 2.514 queries + pool pequeno = DoS sem credencial Esforço médio · impacto alto — Corrigir SEC-01 (exigir auth), PERF-01 (agregação em SQL único + cache curto), rate limit no Caddy para <code>/api/*</code> , <code>statement_timeout</code> e múltiplos workers.
MÉDIA	PERF-03	Falta índice em <code>terminals(entity_type, entity_id)</code> : buscas do rastreo e do impacto fazem Seq Scan em 206 mil linhas Esforço baixo · impacto médio — <code>CREATE INDEX CONCURRENTLY idx_terminals_entity ON terminals (entity_type, entity_id)</code> ; via migração.
MÉDIA	SEC-02	<code>/metrics</code> protegido por token padrão público e a variável do compose é ignorada Esforço baixo · impacto médio — Renomear no compose para <code>METRICS_SECRET_TOKEN</code> (ou aceitar <code>METRICS_TOKEN</code> via <code>validation_alias</code>); sem default (obrigatório quando <code>METRICS_ENABLED</code>); comparar com <code>hmac.compare_digest</code> ; não expor <code>/metrics</code> no Caddy (rede interna/porta separada).
MÉDIA	SEC-05	Defaults de segredo públicos no código, no compose e no <code>.env.example</code> , sem validação de startup Esforço baixo · impacto médio — Remover defaults do compose para segredos (<code>\${VAR:?defina VAR}</code>); <code>model_validator</code> em <code>Settings</code> que falha em <code>ENVIRONMENT=production</code> se <code>SECRET_KEY/CSRF_SECRET/METRICS_SECRET_TOKEN</code> /senha do banco forem os defaults conhecidos ou curtos.

Onda 2 — curto prazo (2-4 semanas)

Autorização por padrão, integridade de dados, limites e observabilidade mínima.

ALTA	EST-05	Importação descarta a geometria dos cabos (cria Cable sem segmento) e o CSV inventa coordenadas Esforço médio · impacto alto — Criar <code>CableSegment</code> (com estruturas de origem/destino resolvidas) ou rejeitar cabos sem estruturas; nunca preencher coordenadas por default (marcar como erro de validação).
ALTA	PERF-02	Análise de impacto: carrega todos os clientes e roda um rastreo óptico completo por vínculo ativo (62 statements por vínculo) Esforço alto · impacto alto — Resolver o impacto em SQL/grafos em memória: carregar terminais/conexões/arestas uma vez (ou CTE recursiva) e fazer BFS único a partir dos segmentos rompidos; mover para job assíncrono para bases grandes; limitar <code>cable_segment_ids</code> .

MÉDIA	EST-02	Rotas-stub 501 sem autenticação (allow-by-default): splitters, settings e occupancy Esforço baixo · impacto médio — Aplicar <code>dependencies=[Depends(get_current_user)]</code> no <code>APIRouter</code> raiz (<code>api_v1_router</code>) com <code>allowlist</code> explícita para <code>login/csrf/health</code> ; teste que percorre <code>app.routes</code> e falha para rotas sem auth.
MÉDIA	EST-03	Concorrência otimista (If-Match) é check-then-act: dois PATCH com a mesma versão passam (lost update) Esforço médio · impacto médio — Usar <code>version_id_col</code> do SQLAlchemy (ou <code>UPDATE ... WHERE id=:id AND version=:v</code> verificando <code>rowcount</code>) e/ou <code>SELECT ... FOR UPDATE</code> antes da validação.
MÉDIA	EST-04	Divisão de segmento de cabo sem lock, sem If-Match e sem revisão topológica esperada Esforço médio · impacto médio — <code>SELECT ... FOR UPDATE</code> no segmento (e nas fibras), exigir <code>If-Match/expected_topology_revision</code> , e falhar se o segmento já não existir.
MÉDIA	EST-08	Importações/exportações ignoram <code>STORAGE_PATH</code> : lêem <code>settings.STORAGE_DIR</code> , atributo que não existe Esforço baixo · impacto médio — Usar <code>Path(settings.STORAGE_PATH)/'imports' 'exports'</code> e persistir caminhos relativos ao <code>storage root</code> .
MÉDIA	EST-13	Métricas em memória com cardinalidade ilimitada: paths 404 distintos crescem o dicionário sem limite (e são por-processo) Esforço baixo · impacto médio — Usar um rótulo fixo (<code>__unmatched__</code>) quando não houver rota; limitar o número de chaves; exportar métricas por processo com agregação (prometheus <code>multiprocess</code>) ou instrumentar o worker.
MÉDIA	EST-16	CI sem varredura de dependências, segredos ou SAST, sem build de imagens e sem <code>permissions</code> : mínimas Esforço médio · impacto médio — Adicionar <code>pip-audit</code> , <code>pnpm audit --audit-level high</code> , <code>gitleaks</code> , <code>CodeQL</code> , <code>docker compose build</code> , <code>trivy</code> ; <code>permissions: contents: read</code> ; fixar actions por SHA.
MÉDIA	PERF-05	Importação: parse síncrono de até 20 MB dentro de handler <code>async</code> , tudo em memória, e 1 <code>SELECT</code> de cancelamento por entidade Esforço médio · impacto médio — Handler <code>def/threadpool</code> com streaming; limites de feições; verificar cancelamento a cada N entidades e usar <code>bulk_insert_mappings</code> .
MÉDIA	PERF-06	Buscas por <code>ILIKE '%q%'</code> sem <code>pg_trgm/GIN</code> em sites, estruturas, cabos, dispositivos, clientes, usuários e notas de portas Esforço baixo · impacto médio — <code>CREATE EXTENSION pg_trgm + GIN (code gin_trgm_ops)</code> (e <code>name/serial/notes</code>); escapar curingas do termo; para q curto exigir mínimo de 3 caracteres.
MÉDIA	PERF-12	Um único processo <code>uvicorn</code> , pool de 10+20 conexões e nenhum <code>statement_timeout/timeout</code> de request Esforço médio · impacto médio — <code>--workers N</code> (ou réplicas), <code>statement_timeout</code> (ex.: 15 s) no <code>connect_args</code> , <code>timeout</code> de request no proxy, dimensionar pool por worker e separar rotas pesadas (fila).
MÉDIA	SEC-04	Anexos e trilha de auditoria ignoram a permissão da entidade dona (PII via technician) Esforço médio · impacto médio — Autorizar por entidade: filtrar <code>entity_type in ('customer', 'service_link')</code> quando o papel não tem <code>customers:read</code> ; mascarar PII em <code>changes</code> ou restringir <code>audit:read</code> .
MÉDIA	SEC-06	Rate limit de login permite travar qualquer conta (DoS de conta) e penaliza IP compartilhado Esforço médio · impacto médio — Limitar por (IP, e-mail) combinados com backoff exponencial, sem bloquear login com senha correta de origem conhecida; limpar contador em sucesso; alertar em picos.
MÉDIA	SEC-08	Troca de senha (e reset via CLI) não revoga as demais sessões Esforço baixo · impacto médio — Ao trocar/resetar senha, marcar <code>is_revoked=True</code> em todas as sessões do usuário exceto a atual; expor 'sair de todos os dispositivos'.
MÉDIA	SEC-10	Restauração de backup extrai tar sem filtro e monta SQL com nomes vindos do arquivo Esforço médio · impacto médio — <code>tar.extractall(..., filter='data')</code> e validação de nomes de membros; validar <code>table_name</code> contra <code>information_schema/allowlist</code> e citar com <code>psycpg.sql.Identifier</code> ; assinar o manifesto (HMAC/GPG) e verificar antes de extrair.
MÉDIA	SEC-11	CSP com 'unsafe-inline'/'unsafe-eval', proxy só em HTTP com cookie Secure e sem HSTS; documentação afirma o contrário Esforço médio · impacto médio — Usar nonces/hashes do Next.js e remover <code>unsafe-*</code> ; habilitar TLS no Caddy (ou documentar e testar o LB) e enviar <code>Strict-Transport-Security</code> ; alinhar <code>SECURITY.md/runbook</code> .
MÉDIA	SEC-13	Exportação de dados pessoais não é auditada, não expira e o download não revalida o papel (contradiz a UI) Esforço médio · impacto médio — Auditar criação e download de exportações (ator, camadas, IP); checar no download que o solicitante é o dono ou admin; TTL configurável e job de limpeza dos arquivos.
MÉDIA	SEC-17	Cadeia: sessão não revogada + ausência de auditoria de auth/usuários = persistência invisível Esforço médio · impacto médio — Auditar todos os eventos de identidade (login ok/falha, logout, criação/edição/desativação, troca de senha, mudança de papel) e revogar sessões na troca de senha.

Onda 3 — médio prazo (1-3 meses)

Cobertura de auditoria, escala horizontal, backup e otimizações estruturais.

MÉDIA	EST-07	Lease do job nunca é renovada: <code>heartbeat()</code> é código morto e importações longas podem ser reexecutadas por outro worker Esforço médio · impacto médio — Renovar a lease a cada N entidades (thread de heartbeat ou chamadas dentro do loop) e verificar <code>lease_owner</code> antes de gravar o status final.
MÉDIA	EST-09	Upload grava em disco antes do commit; falhas geram arquivos órfãos e o reconciliador pode apagar upload em andamento Esforço médio · impacto baixo — Gravar em arquivo temporário e <code>os.replace</code> após o commit (ou registrar no banco primeiro e fazer GC por idade); reconciliador ignora arquivos com mtime recente.
MÉDIA	EST-10	Trilha de auditoria cobre só anexos, conectividade, clientes e importação; usuários, inventário, cabos, óptico e exportações ficam de fora Esforço alto · impacto médio — Centralizar em um decorador/dependência (ou event listener do SQLAlchemy) que audite toda mutação com ator, <code>request_id</code> e diff sanitizado; tornar a tabela append-only no banco (REVOKE UPDATE/DELETE ou trigger).
MÉDIA	EST-14	SPOF e estado local: 1 processo uvicorn, 1 backend com <code>container_name</code> fixo, storage em volume local, métricas por processo Esforço alto · impacto médio — Documentar/implementar: <code>--workers N</code> ou múltiplas réplicas sem <code>container_name</code> ; storage em objeto (S3/MinIO) ou NFS; backup agendado + teste de restauração; réplica de leitura/HA do banco; healthcheck do worker.
MÉDIA	EST-19	Backup por fallback <code>psycopy</code> não é snapshot consistente e o arquivo é gravado sem criptografia Esforço médio · impacto baixo — Abrir transação <code>REPEATABLE READ ONLY</code> , ou exigir <code>pg_dump -Fc</code> ; criptografar (age/gpg) e assinar o pacote.
MÉDIA	PERF-08	Exportações carregam camadas inteiras em memória (<code>.all()</code>) e serializam JSON com indent Esforço médio · impacto baixo — <code>yield_per/cursor</code> server-side e escrita incremental (NDJSON/ <code>json.dump</code> por feature); limite por camada.
MÉDIA	PERF-14	Rastreo óptico: ≥ 5 queries por salto e cópia de listas por ramo; <code>max_hops</code> até 500 e <code>max_results</code> até 200 vindos do cliente Esforço alto · impacto médio — Pré-carregar o grafo do subconjunto (terminais/conexões/arestas) em 3-4 queries; aplicar <code>min(max_hops, MAX_TRACE_HOPS)</code> ; memoizar; rate limit.
BAIXA	EST-06	Idempotência do commit de importação: check-then-insert e chave global sem escopo Esforço baixo · impacto baixo — <code>INSERT ... ON CONFLICT (idempotency_key) DO NOTHING RETURNING</code> e comparar <code>payload.import_id</code> ; escopar a chave por (<code>user_id</code> , <code>import_id</code>); validar tamanho.
BAIXA	EST-12	UUID inválido em path/query/body resulta em HTTP 500 (ValueError não tratado) e strings sem <code>max_length</code> quebram no banco Esforço baixo · impacto baixo — Tipar parâmetros como <code>uuid.UUID</code> nos handlers (FastAPI valida e responde 422) e adicionar <code>max_length</code> alinhado às colunas.
BAIXA	EST-17	Configurações declaradas e nunca usadas: <code>SECRET_KEY</code> , <code>CSRF_SECRET</code> , <code>METRICS_ENABLED</code> , <code>DEBUG</code> , <code>MAX_TRACE_HOPS</code> Esforço baixo · impacto baixo — Remover ou implementar (CSRF assinado com <code>CSRF_SECRET</code> , /metrics condicionado a <code>METRICS_ENABLED</code> , <code>min(request.max_hops, MAX_TRACE_HOPS)</code>).
BAIXA	EST-18	Worker sem logs JSON/correlation-id, sem healthcheck e sem métricas de jobs Esforço baixo · impacto baixo — Reutilizar <code>setup_logging</code> , incluir <code>job_id</code> , incrementar <code>record_job</code> e expor heartbeat/healthcheck.
BAIXA	SEC-07	Login revela contas existentes porém desativadas (403 antes de verificar a senha) Esforço baixo · impacto baixo — Verificar a senha primeiro e só então informar desativação (ou responder 401 genérico).
BAIXA	SEC-09	Validação de Origin do CSRF usa startswith (bypass por sufixo) e o token CSRF não é vinculado à sessão Esforço baixo · impacto baixo — Comparar origens por igualdade exata de (esquema, host, porta); assinar o token CSRF com <code>HMAC(CSRF_SECRET, session_id)</code> .
BAIXA	SEC-12	<code>seed_demo</code> cria administrador com credencial fixa pública, sem guarda de ambiente Esforço baixo · impacto baixo — Abortar quando <code>ENVIRONMENT=production</code> ; gerar senha aleatória e imprimir uma vez; exigir <code>--i-know-this-is-not-prod</code> .
BAIXA	SEC-14	<code>GET /jobs/{id}</code> expõe <code>error_message</code> cru (<code>str(exception)</code>) a qualquer usuário autenticado Esforço baixo · impacto baixo — Persistir mensagem sanitizada/código de erro para o cliente e o detalhe apenas em log; exigir <code>imports:read/exports:read</code> conforme o tipo do job.
BAIXA	SEC-15	Dependências do frontend com advisories (maplibre-gl, postcss via next, vitest) Esforço médio · impacto baixo — Atualizar maplibre-gl para $\geq 6.4.1$ (major), next/postcss e vitest; ativar <code>Dependabot/pnpm audit</code> no CI.

BAIXA	SEC-18	Cabeçalho X-Forwarded-For confiável de qualquer origem (ProxyHeaders '*' + get_client_ip) Esforço baixo · impacto baixo — Confiar apenas nos IPs dos proxies reais (forwarded_allow_ips/trusted_hosts), validar IP com ipaddress, truncar/rejeitar valores inválidos.
--------------	---------------	---

Backlog

Melhorias de baixo risco/retorno.

BAIXA	EST-20	Matriz de permissões duplicada em backend e frontend (com divergência) e gates de UI inexistentes Esforço médio · impacto baixo — Gerar rbac.ts a partir do backend (endpoint /auth/me já devolve permissions) e aplicar PermissionGate nas ações de escrita; teste de paridade.
BAIXA	EST-21	Lógica crítica duplicada: 6 cópias da validação de If-Match e regra/consulta dentro de handlers Esforço baixo · impacto baixo — Extrair core/concurrency.py (assert_version(current, if_match) + atualização atômica) e mover a lógica de download para modules/exports/service.py.
BAIXA	PERF-07	Listagens paginadas com COUNT(*) + OFFSET + ORDER BY created_at sem índice de suporte Esforço baixo · impacto baixo — Índice (created_at DESC, id); paginação keyset; contagem aproximada/cache.
BAIXA	PERF-09	Linha única network_topology_state atualizada em toda escrita de topologia serializa transações concorrentes Esforço médio · impacto baixo — Manter o bump como último passo antes do commit (já é) e evitar transações longas; considerar sequence/advisory lock curto.
BAIXA	PERF-10	Cada requisição autenticada faz 2 SELECTs (sessão e usuário por lazy load) e pode gravar last_activity_at Esforço baixo · impacto baixo — options(joinedload(UserSession.user))/contains_eager e atualizar last_activity_at de forma assíncrona/amostrada.
BAIXA	PERF-11	Sem retenção: login_attempts, user_sessions expiradas, audit_events e arquivos de exportação crescem indefinidamente Esforço baixo · impacto baixo — Job de retenção (ex.: 30 dias para login_attempts, 7 dias para sessões inválidas, TTL de exportações) e índice composto (email, attempted_at).
BAIXA	PERF-13	/health/ready lê o diretório do Alembic do disco e usa uma conexão do pool a cada sonda (10 s) Esforço baixo · impacto baixo — Cachear o head esperado em memória no startup; usar conexão dedicada/curta (engine.connect() com timeout).

10. Limitações do trabalho

- **Não executado:** build real do frontend e da stack docker compose completa; restauração de backup; concorrência real de PATCH/split; carga sustentada em hardware de produção; testes de penetração.
- **Medições** foram feitas in-process (TestClient) contra PostgreSQL/PostGIS descartável com o dataset sintético do repositório (10.001 estruturas, 103.200 fibras, 1 cliente): informam ordem de grandeza e proporções, não capacidade de produção. Extrapolações estão marcadas como tal.
- **Não verificável sem acesso à operação:** configuração real de produção (variáveis, TLS, LB à frente, política de rede), conteúdo do banco, logs reais e se o repositório é público (afeta a severidade dos defaults de segredo).
- pnpm audit/pip-audit refletem as bases de advisories no dia da execução (18/09/2026, rede disponível). O frontend não foi buildado para inspeção do bundle; foi varrido o .next já existente.

11. ISSUES PARA O GITHUB

29 issues (achados triviais e relacionados agrupados). O mesmo conteúdo está em `issues.md` (Markdown puro — use-o para colar no GitHub; copiar deste PDF quebra linhas). Cada bloco contém: título, labels, descrição, evidência (arquivo:linha + trecho), impacto, correção, como validar (PERF: comando antes/depois) e critérios de aceite.

Issue 1 — [Segurança] Exigir autenticação em /search e /dashboard/summary (e fechar a cadeia de DoS anônimo)

--- ISSUE 1 ---

Título: [Segurança] Exigir autenticação em /search e /dashboard/summary (e fechar a cadeia de DoS anônimo)

Labels sugeridas: security, severity:alta

Achados cobertos: SEC-01, SEC-16

Descrição do problema e por que é explorável / degrada o sistema

SEC-01 – Busca global e resumo do painel acessíveis sem autenticação (severidade: alta)

`dashboard/summary` não declara nenhuma dependência de autenticação e `search` usa `get\_optional\_current\_user` (anônimo permitido). Anônimos recebem sites, estruturas, cabos e dispositivos (inclui `serial\_number`) por `ILIKE '%q%'`, além de contadores agregados (clientes, vínculos ativos, faixas de ocupação de CT0s). Só a camada `customers` da busca é protegida (`customers:read`). O teste `test\_reports\_dashboard\_search.py:308` **codifica** o acesso anônimo como esperado, e os docs (`docs/api-catalog.md`, ADR-0003) não o declaram como decisão de produto.

- **Por que é explorável / degrada:** Reconhecimento gratuito do inventário físico do provedor (localização de POPs, códigos de CT0/CE0, serials de OLT/ONU) e enumeração por prefixo. Como o mesmo endpoint é caro (PERF-01), também é amplificador de DoS sem credencial (cadeia SEC-16).

- **Condições de ocorrência:** Sempre (nenhuma feature flag). `GET /api/v1/search` e `GET /api/v1/dashboard/summary` respondem 200 a qualquer cliente de rede que alcance a API; o Caddy publica `/api/\*` (Caddyfile:19-21).

- **Prova:** medido: `docs/security-audit/tools/measure.py` (Postgres descartável, 10.001 estruturas): `GET /api/v1/dashboard/summary` sem auth → HTTP 200 (3/3 execuções); `GET /api/v1/search?q=OLT` sem auth → HTTP 200, grupos retornados: `['device']`.

SEC-16 – Cadeia: painel/busca anônimos + N+1 de 2.514 queries + pool pequeno = DoS sem credencial (severidade: alta)

0 endpoint anônimo executa 2 queries por CT0 (2.514 statements com 10.001 estruturas). O backend roda 1 processo uvicorn e pool de 10+20 conexões, sem `statement\_timeout` nem rate limit. Vinte requisições concorrentes levaram 16,9 s para concluir (p50=16,8 s), e o readiness (que precisa de conexão do pool) oscilou.

- **Por que é explorável / degrada:** Um cliente não autenticado a ~1 req/s já mantém o processo saturado; readiness e healthcheck do compose passam a falhar, derrubando dependências (`frontend depends\_on backend healthy`).

- **Condições de ocorrência:** Combina SEC-01 + PERF-01 + PERF-12. Basta alcançar `/api/v1/dashboard/summary` (Caddy publica `/api/\*`).

- **Prova:** medido: `tools/measure5.py` — baseline `/health/ready`=151 ms, dashboard sequencial=1.095 ms; 20 concorrentes: wall=16,9 s, p50=16,8 s, max=16,9 s, todas HTTP 200; `/health/ready` durante a carga: [1126, 228, 259, 216, 260, 219] ms.

Evidência

SEC-01 `backend/app/api/v1/reports.py:35`

...

def get_dashboard_summary(db: Session = Depends(get_db)) -> DashboardSummaryResponse:

...

SEC-01 `backend/app/api/v1/reports.py:51`

...

current_user: User | None = Depends(get_optional_current_user),

...

SEC-01 `backend/app/modules/reports/service.py:280`

...

Device.serial_number.ilike(f"%{clean_q}%"),

...

SEC-01 `backend/tests/integration/test\_reports\_dashboard\_search.py:308`

...

resp_anon = client.get("/api/v1/search?q=ALPHA")

...

SEC-01 `Caddyfile:19`

...

handle /api/* {

...

SEC-16 `backend/app/api/v1/reports.py:35`

...

def get_dashboard_summary(db: Session = Depends(get_db)) -> DashboardSummaryResponse:

...

SEC-16 `backend/app/modules/reports/service.py:49`

...

ports = db.scalars(select(Port).where(Port.structure_id == cto.id)).all()

...

SEC-16 `backend/app/db/session.py:21`

...

pool_size=settings.DB_POOL_SIZE,

...

SEC-16 `backend/Dockerfile:59`

...

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

...

Impacto

- **SEC-01:** Vazamento de dados operacionais de infraestrutura crítica a qualquer host que alcance o proxy; base para engenharia social e ataques físicos/lógicos direcionados.

- **SEC-16:** Indisponibilidade da API para todos os usuários por um único atacante anônimo.

Sugestão de correção

- **SEC-01:** Exigir `Depends(require\_permission("reports:read"))` (dashboard) e `map:read`/`network:read` (busca); remover

```
`get_optional_current_user` de `/search` e decidir por permissão granular quais grupos retornar. Ajustar o teste que hoje afirma o comportamento anônimo.
- **SEC-16:** Corrigir SEC-01 (exigir auth), PERF-01 (agregação em SQL único + cache curto), rate limit no Caddy para `/api/*`, `statement_timeout` e múltiplos workers.

## Como validar
- **SEC-01:** `curl -s -o /dev/null -w '%{http_code}' http://HOST/api/v1/dashboard/summary` e `.../search?q=0LT` sem cookie: deve retornar 401. Com viewer: 200.
- **SEC-16:** `tools/measure5.py`: 20 GETs concorrentes ao dashboard; alvo p95 < 1 s e 401 sem sessão.

## Critérios de aceite
- [ ] SEC-01: Requisição sem sessão a `/api/v1/dashboard/summary` e `/api/v1/search` retorna 401 (`authentication_required`).
- [ ] SEC-01: Teste automatizado parametrizado sobre TODAS as rotas de `app` falha se algum handler fora de uma allowlist explícita (health/auth) não depender de `get_current_user` (falha hoje: 9+2 rotas; passa depois).
- [ ] SEC-01: `test_reports_dashboard_search.py` atualizado: anônimo → 401.
- [ ] SEC-16: Sem sessão → 401 imediato (custo 0(1)).
- [ ] SEC-16: Com sessão, `/dashboard/summary` executa ≤ 10 queries com 10k estruturas (teste com contador de statements, falha hoje: 2.514).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 1 ---
```

Issue 2 — [Segurança] /metrics com token padrão público e variável do compose ignorada

--- ISSUE 2 ---

Título: [Segurança] /metrics com token padrão público e variável do compose ignorada
Labels sugeridas: security, severity:média
Achados cobertos: SEC-02

```
## Descrição do problema e por que é explorável / degrada o sistema
### SEC-02 – /metrics protegido por token padrão público e a variável do compose é ignorada (severidade: média)
`config.py` define um default fixo para `METRICS_SECRET_TOKEN` (`dev-...`, mascarado). O `compose.yaml` injeta `METRICS_TOKEN` (nome diferente), que `Settings` ignora (`extra="ignore"`), de modo que o default permanece ativo mesmo com `ENVIRONMENT=production`. A comparação usa `==` (não é tempo constante) e não há validação de startup que rejeite o default.
- **Por que é explorável / degrada:** Quem conhece o default (está no repositório) faz `GET /api/v1/metrics` com `X-Metrics-Token` e obtém rotas, contagem/latência por rota, erros 5xx, estatísticas do pool de conexões e revisão topológica.
- **Condições de ocorrência:** Deploy via `compose.yaml` (padrão) ou qualquer deploy que não defina `METRICS_SECRET_TOKEN`. O Caddy encaminha `/api/*` (inclui `/api/v1/metrics`) para o backend.
- **Prova:** medido: `tools/measure3.py` – com `METRICS_TOKEN` definido no ambiente, `Settings().METRICS_SECRET_TOKEN` continua igual ao default do código (True); `tools/measure2.py` – `GET /api/v1/metrics` com o default → HTTP 200.
```

```
## Evidência
**SEC-02** `backend/app/core/config.py:60` – valor mascarado
...
    METRICS_SECRET_TOKEN: str = "dev-..."
...
**SEC-02** `compose.yaml:49`
...
    METRICS_TOKEN: ${METRICS_TOKEN:-}
...
**SEC-02** `backend/app/api/v1/metrics.py:30`
...
    if x_metrics_token and x_metrics_token == settings.METRICS_SECRET_TOKEN:
...
**SEC-02** `Caddyfile:19`
...
    handle /api/* {
```

```
## Impacto
- **SEC-02:** Divulgação de informações operacionais e do mapa de rotas; facilita reconhecimento e calibração de ataques de DoS.
```

```
## Sugestão de correção
- **SEC-02:** Renomear no compose para `METRICS_SECRET_TOKEN` (ou aceitar `METRICS_TOKEN` via `validation_alias`); sem default (obrigatório quando `METRICS_ENABLED`); comparar com `hmac.compare_digest`; não expor `/metrics` no Caddy (rede interna/porta separada).
```

```
## Como validar
- **SEC-02:** Subir stack com `METRICS_TOKEN=abc...` e chamar `/api/v1/metrics` com o token default do código: deve retornar 401/403.
```

```
## Critérios de aceite
- [ ] SEC-02: Com `ENVIRONMENT=production` e sem `METRICS_SECRET_TOKEN`, a aplicação recusa iniciar.
- [ ] SEC-02: Teste automatizado: `Settings(METRICS_TOKEN='x')` (nome do compose) → token efetivo == `x` (falha hoje; passa depois).
- [ ] SEC-02: Requisição com o default → 401/403.
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
```

--- FIM ISSUE 2 ---

Issue 3 — [Segurança] Viewer e technician acessam PII de clientes: aplicar customers:*, anexos e auditoria por entidade

--- ISSUE 3 ---

Título: [Segurança] Viewer e technician acessam PII de clientes: aplicar customers:*, anexos e auditoria por entidade

Labels sugeridas: security, severity:alta

Achados cobertos: SEC-03, SEC-04

Descrição do problema e por que é explorável / degrada o sistema

SEC-03 – Papel viewer lê dados pessoais de clientes: permissão `customers:\*` nunca é exigida (severidade: alta)
A matriz de permissões atribui `customers:read/write` apenas a engineer/admin, e a busca global usa `customers:read` para decidir se mostra clientes. Porém todas as rotas `/customers` e `/service-links` exigem apenas `network:read` (leitura) ou `network:write` (escrita); `customers:\*` não é exigido por nenhuma rota. `CustomerRead` devolve nome, telefone, e-mail, endereço e notas.

- **Por que é explorável / degrada:** Um viewer (ou uma sessão de viewer roubada) exporta a base de assinantes paginando `GET /customers?page\_size=200`.

- **Condições de ocorrência:** Qualquer conta autenticada (inclusive `viewer`, o papel mínimo). Sem feature flag.

- **Prova:** medido: `tools/measure2.py` – sessão de viewer: `GET /api/v1/customers` → HTTP 200 com campos id, code, name, phone, email, address, notes.

SEC-04 – Anexos e trilha de auditoria ignoram a permissão da entidade dona (PII via technician) (severidade: média)

Anexos podem ser vinculados a `customer` e `service\_link`; `GET /attachments`, `/{id}` e `/{id}/download` só exigem `attachments:read`, sem consultar a permissão da entidade referenciada. `GET /audit-events` (`audit:read`) devolve `changes` para `customer:updated` o JSON traz phone, email e address em claro.

- **Por que é explorável / degrada:** Um technician lê documentos anexados a clientes (contratos/fotos) e o histórico de alterações com PII, contornando o bloqueio a `customers:read`.

- **Condições de ocorrência:** Papel technician (possui `attachments:read` e `audit:read`, mas não `customers:read`).

- **Prova:** verificado por leitura, não medido (medido apenas que viewer recebe 403 em `/attachments` e `/audit-events`; technician não foi exercitado). Como medir: criar technician, anexo em customer e chamar as duas rotas.

Evidência

SEC-03 `backend/app/api/v1/customers.py:30-31`

```
...
    summary="Listar clientes",
    dependencies=[Depends(require_permission("network:read"))],
...
```

SEC-03 `backend/app/core/permissions.py:33`

```
...
    "customers:read",
...
```

SEC-03 `backend/app/modules/reports/service.py:305`

```
...
    user_can_read_customers = has_permission(role_enum, "customers:read")
...
```

SEC-03 `backend/app/schemas/customers.py:34`

```
...
class CustomerRead(BaseModel):
...
```

SEC-04 `backend/app/api/v1/attachments.py:84-87`

```
...
    summary="Listar anexos com paginação e filtros",
    description="Retorna lista paginada de anexos cadastrados com filtro opcional por entidade.",
    dependencies=[Depends(require_permission("attachments:read"))],
)
...
```

SEC-04 `backend/app/core/permissions.py:18`

```
...
    "attachments:read",
...
```

SEC-04 `backend/app/modules/attachments/service.py:142`

```
...
    "customer": Customer,
...
```

SEC-04 `backend/app/api/v1/reports.py:161`

```
...
    dependencies=[Depends(require_permission("audit:read"))],
...
```

SEC-04 `backend/app/modules/customers/service.py:168`

```
...
    changes["phone"] = customer.phone
...
```

Impacto

- **SEC-03:** Violação de LGPD (dados pessoais a papel sem necessidade) e contradição entre política declarada (`/search`) e política aplicada (`/customers`).

- **SEC-04:** Segundo e terceiro caminhos para o mesmo dado pessoal de SEC-03; quebra o modelo de menor privilégio.

Sugestão de correção

- **SEC-03:** Trocar `network:read/write` por `customers:read/write` em todas as rotas de clientes e vínculos; adicionar teste de matriz papel×rota.

- **SEC-04:** Autorizar por entidade: filtrar `entity\_type in ('customer','service\_link')` quando o papel não tem `customers:read`; mascarar PII em `changes` ou restringir `audit:read`.

Como validar

- **SEC-03:** Login como viewer e `GET /api/v1/customers`: deve retornar 403 `insufficient\_permissions`.

- **SEC-04:** Como technician: `GET /api/v1/attachments?entity\_type=customer` e `GET /api/v1/audit-events?entity\_type=customer` devem retornar 403 ou lista vazia.

```
## Critérios de aceite
- [ ] SEC-03: viewer e technician recebem 403 em `GET /customers`, `GET /customers/{id}`, `GET /service-links`.
- [ ] SEC-03: Teste automatizado de matriz: para cada permissão declarada, ao menos uma rota a exige; e viewer → 403 nas rotas de clientes (falha hoje; passa depois).
- [ ] SEC-04: technician não lista/baixa anexos de `customer`/`service_link`.
- [ ] SEC-04: `GET /audit-events` para technician não retorna `phone/email/address` (teste de contrato falha hoje, passa depois).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 3 ---
```

Issue 4 — [Segurança] Segredos default públicos e configurações mortas: validar em startup, sem defaults no compose

--- ISSUE 4 ---

Título: [Segurança] Segredos default públicos e configurações mortas: validar em startup, sem defaults no compose

Labels sugeridas: security, severity:média

Achados cobertos: SEC-05, EST-17

Descrição do problema e por que é explorável / degrada o sistema

SEC-05 – Defaults de segredo públicos no código, no compose e no .env.example, sem validação de startup (severidade: média)

`SECRET\_KEY`, `CSRF\_SECRET` e `METRICS\_SECRET\_TOKEN` têm default em `config.py`; `compose.yaml` repete `\${SECRET\_KEY:-...}`, `\${CSRF\_SECRET:-...}` e `\${POSTGRES\_PASSWORD:-ftth...}` (senha padrão do banco). Não existe `model\_validator` que rejeite defaults quando `is\_production`. (`SECRET\_KEY` e `CSRF\_SECRET` hoje não são lidos por nenhum código – ver EST-17 – mas a senha do banco e o token de métricas são.)

- ****Por que é explorável / degrada:**** Um operador que suba `docker compose up` sem `.env` obtém um sistema 'em produção' com credenciais conhecidas do repositório; quem alcançar a rede interna do Docker entra no PostgreSQL com o usuário `ftth\_user`.

- ****Condições de ocorrência:**** Deploy sem sobrescrever variáveis. O compose define `ENVIRONMENT` default `production`, mas `Settings` aceita os defaults mesmo assim.

- ****Prova:**** medido: `tools/measure3.py` – `ENVIRONMENT=production` → `Settings()` instancia com `SECRET\_KEY` = default (prefixo `dev-...`), `is\_production=True`.

EST-17 – Configurações declaradas e nunca usadas: SECRET_KEY, CSRF_SECRET, METRICS_ENABLED, DEBUG, MAX_TRACE_HOPS (severidade: baixa)

`grep` mostra que `SECRET\_KEY`, `CSRF\_SECRET`, `METRICS\_ENABLED` e `MAX\_TRACE\_HOPS` não são lidos por nenhum código (apenas `DEBUG` vai a um log). O teto do rastreo óptico vem do cliente (`TraceRequest.max\_hops ≤ 500`), não da config.

- ****Por que é explorável / degrada:**** Operadores acreditam controlar CSRF/métricas/teto de saltos por variáveis que não têm efeito.

- ****Condições de ocorrência:**** Sempre.

- ****Prova:**** medido (grep): `grep -rn 'MAX\_TRACE\_HOPS|METRICS\_ENABLED|CSRF\_SECRET|SECRET\_KEY' backend/app` → só `config.py`.

Evidência

SEC-05 `backend/app/core/config.py:27-30` – valor mascarado

```
...
SECRET_KEY: str = Field(
    default="dev-...",
    min_length=32,
...
)
```

SEC-05 `backend/app/core/config.py:32` – valor mascarado

```
...
CSRF_SECRET: str = "dev-..."
...
```

SEC-05 `compose.yaml:8` – valor mascarado

```
...
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-ftth...}
...
```

SEC-05 `compose.yaml:43` – valor mascarado

```
...
SECRET_KEY: ${SECRET_KEY:-dev-...}
...
```

SEC-05 `backend/app/core/config.py:63`

```
...
@field_validator("ENVIRONMENT", mode="before")
...
```

SEC-05 `.env.example:15` – valor mascarado

```
...
SECRET_KEY=dev-...
...
```

EST-17 `backend/app/core/config.py:61`

```
...
MAX_TRACE_HOPS: int = 300
...
```

EST-17 `backend/app/core/config.py:59`

```
...
METRICS_ENABLED: bool = True
...
```

EST-17 `backend/app/schemas/topology.py:34`

```
...
le=500,
...
```

Impacto

- ****SEC-05:**** Comprometimento total do banco em ambientes mal configurados; falsa sensação de segurança (a aplicação inicia

```

normalmente).
- **EST-17:** Falsa sensação de segurança; parte do problema de SEC-02/SEC-05/SEC-09.

## Sugestão de correção
- **SEC-05:** Remover defaults do compose para segredos (`${VAR:?defina VAR}`); `model_validator` em `Settings` que falha em `ENVIRONMENT=production` se `SECRET_KEY`/`CSRF_SECRET`/`METRICS_SECRET_TOKEN`/senha do banco forem os defaults conhecidos ou curtos.
- **EST-17:** Remover ou implementar (CSRF assinado com `CSRF_SECRET`, `/metrics` condicionado a `METRICS_ENABLED`, `min(request.max_hops, MAX_TRACE_HOPS)`).

## Como validar
- **SEC-05:** `ENVIRONMENT=production python -c 'from app.core.config import Settings; Settings()'` deve levantar erro.
- **EST-17:** `grep -rn 'MAX_TRACE_HOPS|METRICS_ENABLED|CSRF_SECRET' backend/app`.

## Critérios de aceite
- [ ] SEC-05: Aplicação recusa iniciar em produção com qualquer default conhecido (teste unitário em `test_config.py`, falha hoje).
- [ ] SEC-05: `docker compose config` falha se `POSTGRES_PASSWORD` não estiver definido.
- [ ] EST-17: Nenhuma variável de `Settings` sem leitor (teste que varre `Settings.model_fields` x `grep` no código; falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 4 ---

```

Issue 5 — [Segurança] Endurecer login e sessões: logout de conta, enumeração, revogação na troca de senha, X-Forwarded-For

--- ISSUE 5 ---

Título: [Segurança] Endurecer login e sessões: logout de conta, enumeração, revogação na troca de senha, X-Forwarded-For

Labels sugeridas: security, severity:média

Achados cobertos: SEC-06, SEC-07, SEC-08, SEC-18

```

## Descrição do problema e por que é explorável / degrada o sistema
### SEC-06 – Rate limit de login permite travar qualquer conta (DoS de conta) e penaliza IP compartilhado (severidade: média)
`check_login_rate_limit` conta tentativas falhas nos últimos 15 min por IP **ou** e-mail (`|`) e bloqueia com 429 antes de verificar a senha. Nenhum sucesso zera o contador. Atrás de NAT/CGNAT/LB (todos com o mesmo IP) 5 erros de qualquer usuário bloqueiam todos.
- **Por que é explorável / degrada:** 5 POSTs com senha errada para `admin@provedor...` bloqueiam o administrador legítimo por 15 min, repetível indefinidamente; sem CAPTCHA/backoff progressivo.
- **Condições de ocorrência:** Sempre. Basta conhecer o e-mail da vítima (ex.: admin).
- **Prova:** verificado por leitura, não medido. Como medir: script com `requests` contra instância local, 5 POST /auth/login inválidos e 1 válido.

### SEC-07 – Login revela contas existentes porém desativadas (403 antes de verificar a senha) (severidade: baixa)
`authenticate_user` retorna 403 `user_deactivated` para conta inativa **antes** de comparar a senha; contas inexistentes/ativas retornam 401 `invalid_credentials`.
- **Por que é explorável / degrada:** Diferença observável de resposta permite enumerar ex-funcionários/contas desativadas sem conhecer senha.
- **Condições de ocorrência:** Sempre; requer conhecer/adivinhar e-mails.
- **Prova:** verificado por leitura, não medido.

### SEC-08 – Troca de senha (e reset via CLI) não revoga as demais sessões (severidade: média)
`change_user_password` só grava o novo hash. As linhas de `user_sessions` permanecem válidas. O mesmo vale para `bootstrap_admin --reset-password`.
- **Por que é explorável / degrada:** A vítima troca a senha após suspeitar de comprometimento, mas o atacante mantém a sessão por até 7 dias (renovável enquanto houver atividade <24 h).
- **Condições de ocorrência:** Sessão roubada/vazada antes da troca de senha; TTL absoluto 7 dias, inatividade 24 h.
- **Prova:** verificado por leitura, não medido.

### SEC-18 – Cabeçalho X-Forwarded-For confiável de qualquer origem (ProxyHeaders '*' + get_client_ip) (severidade: baixa)
`ProxyHeadersMiddleware(trusted_hosts=["*"])` e `get_client_ip` usam o primeiro item de `X-Forwarded-For` sem allowlist de proxies. O valor vai para `login_attempts.ip_address` (VARCHAR(45)) e para `user_sessions`.
- **Por que é explorável / degrada:** 0 invasor escolhe o 'IP' registrado/limitado; um valor > 45 caracteres provoca erro de banco (HTTP 500) no login.
- **Condições de ocorrência:** Backend alcançável sem passar pelo Caddy (ex.: `compose.override.yaml.example` publica 127.0.0.1:8000; ou LB à frente que não sobrescreve o cabeçalho).
- **Prova:** verificado por leitura, não medido.

## Evidência
**SEC-06** `backend/app/modules/identity/service.py:56`
...
        (LoginAttempt.ip_address == ip_address) | (LoginAttempt.email == clean_email),
...
**SEC-06** `backend/app/modules/identity/service.py:62`
...
    if failed_attempts_count >= RATE_LIMIT_MAX_ATTEMPTS:
...
**SEC-06** `backend/app/modules/identity/service.py:98`
...
    record_login_attempt(session, ip_address, clean_email, success=True)
...
**SEC-07** `backend/app/modules/identity/service.py:89-91`
...
    if not user.is_active:
        record_login_attempt(session, ip_address, clean_email, success=False)
        raise ForbiddenError("Esta conta de usuário está desativada.", code="user_deactivated")

```

```

...
**SEC-08** `backend/app/modules/identity/service.py:174`
...
    user.password_hash = hash_password(new_password)
...
**SEC-08** `backend/app/cli/bootstrap_admin.py:79`
...
    user.password_hash = hash_password(password)
...
**SEC-18** `backend/app/main.py:61`
...
    app.add_middleware(ProxyHeadersMiddleware, trusted_hosts=["*"])
...
**SEC-18** `backend/app/core/dependencies.py:23`
...
    forwarded = request.headers.get("x-forwarded-for")
...
**SEC-18** `backend/app/modules/identity/models.py:64`
...
    ip_address: Mapped[str] = mapped_column(String(45), index=True, nullable=False)
...

## Impacto
- **SEC-06:** Negação de acesso administrativo direcionada; em operação de campo atrás de CGNAT, bloqueio coletivo.
- **SEC-07:** Enumeração de usuários (baixo impacto isolado; útil em phishing/spraying).
- **SEC-08:** Persistência do invasor após a remediação padrão; enfraquece o valor da troca de senha.
- **SEC-18:** Evasão do limite por IP e poluição de trilhas de auditoria/sessão; 500 evitável.

## Sugestão de correção
- **SEC-06:** Limitar por (IP, e-mail) combinados com backoff exponencial, sem bloquear login com senha correta de origem conhecida; limpar contador em sucesso; alertar em picos.
- **SEC-07:** Verificar a senha primeiro e só então informar desativação (ou responder 401 genérico).
- **SEC-08:** Ao trocar/resetar senha, marcar `is_revoked=True` em todas as sessões do usuário exceto a atual; expor 'sair de todos os dispositivos'.
- **SEC-18:** Confiar apenas nos IPs dos proxies reais (`forwarded_allow_ips`/`trusted_hosts`), validar IP com `ipaddress`, truncar/rejeitar valores inválidos.

## Como validar
- **SEC-06:** Enviar 5 logins inválidos para um e-mail e depois um login válido de outro IP: hoje 429; deve autenticar.
- **SEC-07:** Desativar um usuário e comparar respostas a `login` (senha errada) vs usuário inexistente.
- **SEC-08:** Duas sessões do mesmo usuário; trocar senha na A; chamar `/auth/me` na B: hoje 200; deve ser 401.
- **SEC-18:** `curl -H 'X-Forwarded-For: <60 chars>' -X POST .../auth/login` direto no backend: hoje 500.

## Critérios de aceite
- [ ] SEC-06: Teste: 5 falhas de IP A para `u@x` não impedem login correto de `u@x` a partir do IP B.
- [ ] SEC-06: Teste: sucesso reinicia a janela.
- [ ] SEC-07: Respostas de login para inexistente, desativado e senha incorreta são indistinguíveis (status e corpo) – teste de contrato.
- [ ] SEC-08: Teste de integração: após `POST /auth/change-password`, a outra sessão recebe 401 (falha hoje).
- [ ] SEC-18: Teste: XFF inválido é ignorado e `request.client.host` é usado (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

```

--- FIM ISSUE 5 ---

Issue 6 — [Segurança] Validação de Origin do CSRF por startswith e token não vinculado à sessão

--- ISSUE 6 ---

Título: [Segurança] Validação de Origin do CSRF por startswith e token não vinculado à sessão

Labels sugeridas: security, severity:baixa

Achados cobertos: SEC-09

```

## Descrição do problema e por que é explorável / degrada o sistema
### SEC-09 – Validação de Origin do CSRF usa startswith (bypass por sufixo) e o token CSRF não é vinculado à sessão (severidade: baixa)
`validate_csrf` aceita a origem se `origin.startswith(allowed)` para qualquer item de `CORS_ORIGINS`. Além disso o CSRF é double-submit **sem assinatura** (o `CSRF_SECRET` não é usado): quem controlar um cookie no domínio (subdomínio vizinho) escolhe o par válido.
- **Por que é explorável / degrada:** Defesa em profundidade quebrada: a checagem de Origin, que deveria bloquear origens estranhas, deixa passar hosts que apenas 'começam com' a origem permitida.
- **Condições de ocorrência:** Origem `http://localhost:3000.evil.example` (ou qualquer domínio que comece com uma origem permitida). Mitigado por SameSite=Lax do cookie de sessão e pela exigência do par cookie/cabeçalho.
- **Prova:** medido: `tools/measure3.py` – `Origin: http://localhost:3000` → 401 (passou na checagem); `https://evil.example` → 403; `http://localhost:3000.evil.example` → **401 (passou)**; `http://127.0.0.1:3000.attacker.test` → **401 (passou)**.

## Evidência
**SEC-09** `backend/app/core/dependencies.py:87`
...
    or any(origin.startswith(allowed) for allowed in settings.CORS_ORIGINS)
...
**SEC-09** `backend/app/core/security.py:58-60`
...

def generate_csrf_token() -> str:
    """Gera um token CSRF seguro e aleatório."""
    return secrets.token_urlsafe(32)

```

```

...

## Impacto
- **SEC-09:** Baixo isolado (SameSite=Lax protege o cookie de sessão), mas remove uma camada e cria falsa confiança.

## Sugestão de correção
- **SEC-09:** Comparar origens por igualdade exata de (esquema, host, porta); assinar o token CSRF com HMAC(`CSRF_SECRET`, session_id).

## Como validar
- **SEC-09:** `Origin: http://localhost:3000.evil.example` em POST /auth/login deve retornar 403 `csrf_origin_mismatch`.

## Critérios de aceite
- [ ] SEC-09: Teste unitário: Origens `http://localhost:3000.evil.example` e `http://127.0.0.1:3000.attacker.test` → 403 (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 6 ---

```

Issue 7 — [Segurança] Restauração de backup: extractall sem filtro e SQL a partir de nomes do arquivo; backup consistente e cifrado

--- ISSUE 7 ---

Título: [Segurança] Restauração de backup: extractall sem filtro e SQL a partir de nomes do arquivo; backup consistente e cifrado

Labels sugeridas: security, severity:média

Achados cobertos: SEC-10, EST-19

```

## Descrição do problema e por que é explorável / degrada o sistema
### SEC-10 — Restauração de backup extrai tar sem filtro e monta SQL com nomes vindos do arquivo (severidade: média)
`tar.extractall(path=...)` é chamado sem `filter='data'` em três pontos (path traversal / escrita arbitrária de arquivo). No fallback binário, `f.stem` de cada `.bin` extraído é interpolado em `TRUNCATE TABLE {table_name} CASCADE` (injeção de SQL). O manifesto (`manifest.json`) e os SHA-256 vêm do mesmo tar, permitindo recalculá-los tudo.
- **Por que é explorável / degrada:** Um arquivo de backup malicioso executa SQL com o usuário do banco e/ou grava fora do diretório de destino com as permissões do operador.
- **Condições de ocorrência:** Operador restaura um arquivo de backup adulterado ou de origem não confiável (CLI `scripts/restore.py`). Os checksums estão dentro do próprio arquivo, portanto não autenticam.
- **Prova:** verificado por leitura, não medido (não executei restore). Como medir: PoC em banco/diretório descartáveis.

### EST-19 — Backup por fallback pycpg não é snapshot consistente e o arquivo é gravado sem criptografia (severidade: média)
O fallback copia tabela a tabela com `COPY ... TO STDOUT` numa conexão em READ COMMITTED (sem `REPEATABLE READ`/`pg_export_snapshot`), então tabelas ficam em instantes diferentes; os anexos são arquivados depois, sem congelamento. O `.tar.gz` final contém hashes de senha e PII de clientes em texto claro (sem criptografia).
- **Por que é explorável / degrada:** Restauração pode violar FKs/invariantes (ex.: conexões sem terminais) e o backup vaza tudo se o arquivo escapar.
- **Condições de ocorrência:** Ambiente sem `pg_dump` no PATH (fallback), ou operação com escritas concorrentes; backups armazenados fora do host.
- **Prova:** verificado por leitura, não medido.

```

```

## Evidência
**SEC-10** `backend/app/core/backup_restore.py:302`
...
        tar.extractall(path=tmp_dir)
...
**SEC-10** `backend/app/core/backup_restore.py:139`
...
        tar.extractall(path=extract_dir)
...
**SEC-10** `backend/app/core/backup_restore.py:365`
...
        tar.extractall(path=target_storage_path)
...
**SEC-10** `backend/app/core/backup_restore.py:151`
...
        cur.execute(f"TRUNCATE TABLE {table_name} CASCADE;")
...
**SEC-10** `backend/app/core/backup_restore.py:318`
...
        if calc_db_sha != manifest.database_checksum_sha256:
...
**EST-19** `backend/app/core/backup_restore.py:102`
...
        with pycpg.connect(raw_url) as conn, conn.cursor() as cur:
...
**EST-19** `backend/app/core/backup_restore.py:118`
...
        with open(t_file, "wb") as f, cur.copy(f"COPY {t} TO STDOUT (FORMAT binary)") as copy:
...
**EST-19** `backend/app/core/backup_restore.py:270`
...
        tar.add(db_dump_file, arcname="database.dump")
...

```

```

## Impacto
- **SEC-10:** Execução de SQL arbitrário (incl. `TRUNCATE` de tabelas) e sobrescrita de arquivos no host durante a

```

```

restauração.
- **EST-19:** Risco de backup inconsistente e de vazamento de dados de backup.

## Sugestão de correção
- **SEC-10:** `tar.extractall(..., filter='data')` e validação de nomes de membros; validar `table_name` contra `information_schema/allowlist` e citar com `psycopg.sql.Identifier`; assinar o manifesto (HMAC/GPG) e verificar antes de extrair.
- **EST-19:** Abrir transação `REPEATABLE READ READ ONLY`, ou exigir `pg_dump -Fc`; criptografar (age/gpg) e assinar o pacote.

## Como validar
- **SEC-10:** Criar tar com membro `../x` e `bin` de nome `a;drop table users.bin` em ambiente descartável e chamar `restore_backup`: deve recusar.
- **EST-19:** Executar backup durante carga de escrita e restaurar em base descartável; rodar checagens de FK.

## Critérios de aceite
- [ ] SEC-10: Teste: tar com `../evil` → `ValueError`/`tarfile.FilterError` (falha hoje).
- [ ] SEC-10: Teste: nome de tabela fora da allowlist → recusado antes de qualquer SQL.
- [ ] EST-19: Teste: backup com escritas concorrentes restaura sem violação de FK (hoje sem cobertura).
- [ ] EST-19: Arquivo de backup é criptografado (teste verifica que `tar` não abre sem chave).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 7 ---

```

Issue 8 — [Segurança] CSP com unsafe-inline/eval, sem TLS/HSTS no proxy e documentação divergente

--- ISSUE 8 ---

Título: [Segurança] CSP com unsafe-inline/eval, sem TLS/HSTS no proxy e documentação divergente

Labels sugeridas: security, severity:média

Achados cobertos: SEC-11

```

## Descrição do problema e por que é explorável / degrada o sistema
### SEC-11 — CSP com 'unsafe-inline'/'unsafe-eval', proxy só em HTTP com cookie Secure e sem HSTS; documentação afirma o contrário (severidade: média)
O CSP do Caddy permite `unsafe-inline` `unsafe-eval` em `script-src`, o que anula a proteção contra XSS; `SECURITY.md` promete política 'estrita, bloqueando scripts inline'. O Caddy escuta só em `:80` com `auto_https off`, mas o compose publica `443:443` e o runbook diz 'terminação TLS'. O backend marca cookies `Secure` em produção — sem TLS à frente o login falha; com TLS externo não há HSTS.
- **Por que é explorável / degrada:** Qualquer XSS futuro executa sem barreira do CSP; tráfego HTTP em claro se o operador não colocar TLS externo; declaração de controle inexistente induz a erro de auditoria.
- **Condições de ocorrência:** Deploy pelo `compose.yaml` + `Caddyfile` fornecidos.
- **Prova:** verificado por leitura, não medido.

```

Evidência

```

**SEC-11** `Caddyfile:15`
...
Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval' blob:; worker-src 'self' blob:; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob: https://*.tile.openstreetmap.org https://*.basemaps.cartocdn.com; connect-src 'self' blob: https://*.tile.openstreetmap.org https://*.basemaps.cartocdn.com; font-src 'self' data:; frame-ancestors 'self';"
...
**SEC-11** `Caddyfile:3`
...
auto_https off
...
**SEC-11** `SECURITY.md:46`
...
- **Content-Security-Policy (CSP):** Política estrita aplicada no Caddy/Reverse Proxy, bloqueando scripts inline e permitindo exclusivamente conexões autorizadas a servidores de tiles cartográficos.
...
**SEC-11** `compose.yaml:108`
...
- "${HTTPS_PORT:-443}:443"
...
**SEC-11** `docs/runbooks/deployment-and-maintenance.md:18`
...
| `caddy` | `docker.io/caddy:2.8.4-alpine` | `public` | `80:80`, `443:443` | Reverse proxy com compressão, terminação TLS e CSP estrito |
...

```

Impacto

- **SEC-11:** Redução das defesas em profundidade e risco de exposição de sessão em rede não confiável.

Sugestão de correção

- **SEC-11:** Usar nonces/hashes do Next.js e remover `unsafe-\*`; habilitar TLS no Caddy (ou documentar e testar o LB) e enviar `Strict-Transport-Security`; alinhar SECURITY.md/runbook.

Como validar

- **SEC-11:** `curl -sI http://HOST/ | grep -i content-security-policy` e checar ausência de `unsafe-`; `curl -sI https://HOST/ | grep -i strict-transport`.

Critérios de aceite

- [] SEC-11: CSP sem `unsafe-inline`/`unsafe-eval` em `script-src`; teste E2E (Playwright) confirma que a UI funciona.
- [] SEC-11: Resposta HTTPS inclui HSTS; `docker compose up` sobe TLS ou documenta explicitamente o requisito.
- [] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado

no CI.
 - [] Nenhum segredo em claro em logs, respostas ou testes adicionados.
 --- FIM ISSUE 8 ---

Issue 9 — [Segurança] seed_demo cria admin com credencial pública sem guarda de ambiente

--- ISSUE 9 ---

Título: [Segurança] seed_demo cria admin com credencial pública sem guarda de ambiente

Labels sugeridas: security, severity:baixa

Achados cobertos: SEC-12

```
## Descrição do problema e por que é explorável / degrada o sistema
### SEC-12 – seed_demo cria administrador com credencial fixa pública, sem guarda de ambiente (severidade: baixa)
0 seed cria `admin@provedor...` com senha literal do repositório (mascarada: `Admi...`). Não há checagem de `is_production`/URL
do banco.
- **Por que é explorável / degrada:** Se executado por engano em produção, cria uma conta admin com senha conhecida por
qualquer leitor do repositório.
- **Condições de ocorrência:** Operador executa `scripts/seed_demo.py` contra um banco de produção (o script é opt-in, mas
não verifica `ENVIRONMENT`).
- **Prova:** verificado por leitura, não medido.

## Evidência
**SEC-12** `backend/scripts/seed_demo.py:186` – valor mascarado
...
    password_hash=hash_password("Admi..."),
...
**SEC-12** `backend/scripts/seed_demo.py:176`
...
def seed_demo_scenario(session: Session) -> dict[str, Any]:
...

## Impacto
- **SEC-12:** Tomada total do sistema nesse cenário.

## Sugestão de correção
- **SEC-12:** Abortar quando `ENVIRONMENT=production`; gerar senha aleatória e imprimir-la uma vez; exigir
`--i-know-this-is-not-prod`.

## Como validar
- **SEC-12:** `ENVIRONMENT=production python scripts/seed_demo.py` deve sair com erro.

## Critérios de aceite
- [ ] SEC-12: Teste unitário: `seed_demo` com `ENVIRONMENT=production` retorna código ≠ 0 sem tocar o banco (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado
no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 9 ---
```

Issue 10 — [Segurança] Exportações de PII sem auditoria/retenção e vazamento de error_message em /jobs

--- ISSUE 10 ---

Título: [Segurança] Exportações de PII sem auditoria/retenção e vazamento de error_message em /jobs

Labels sugeridas: security, severity:média

Achados cobertos: SEC-13, SEC-14

```
## Descrição do problema e por que é explorável / degrada o sistema
### SEC-13 – Exportação de dados pessoais não é auditada, não expira e o download não revalida o papel (contradiz a UI)
(severidade: média)
A UI afirma que a exportação de clientes é 'registrada permanentemente na trilha de auditoria', mas `create_export_request`
não chama `record_audit_event`. O download exige só `exports:read` (engineer também), não confere o papel do solicitante nem
a camada. Arquivos em `storage/exports` nunca são removidos: a rotina de retenção só limpa `ImportPreview`, apesar do nome.
- **Por que é explorável / degrada:** LGPD: exfiltração em massa (nome, telefone, endereço) sem rastro, com cópia
persistente em disco por prazo indefinido.
- **Condições de ocorrência:** Camada `customers` exportada por admin; arquivo permanece em disco; qualquer engineer/admin
com o `job_id` baixa.
- **Prova:** verificado por leitura, não medido (`grep -n 'record_audit_event'|audit'
backend/app/modules/exports/service.py` → 0 linhas).

### SEC-14 – GET /jobs/{id} expõe error_message cru (str(exception)) a qualquer usuário autenticado (severidade: baixa)
O worker grava `job.error_message = str(e)` e `get_job_by_id` o devolve. Exceções SQLAlchemy incluem SQL, parâmetros e
caminhos de arquivo. A rota só exige `get_current_user` (viewer incluso).
- **Por que é explorável / degrada:** Vaza detalhes internos (esquema, valores, caminhos) a papéis sem necessidade.
- **Condições de ocorrência:** Job falha (ex.: IntegrityError na importação); qualquer papel autenticado que conheça o UUID
do job.
- **Prova:** verificado por leitura, não medido.

## Evidência
**SEC-13** `backend/app/modules/exports/service.py:62`
...
    if "customers" in payload.layers and (not user or user.role != "admin"):
...
**SEC-13** `backend/app/modules/exports/service.py:68`
```

```

...
    job_type = f"export_{payload.format.value}"
...
**SEC-13** `backend/app/modules/exports/service.py:169`
...
        "phone": c.phone,
...
**SEC-13** `backend/app/api/v1/imports_exports.py:131-132`
...
    summary="Download do arquivo exportado",
    dependencies=[Depends(require_permission("exports:read"))],
...
**SEC-13** `backend/app/modules/jobs/service.py:335`
...
    expired_previews = db.scalars(select(ImportPreview).where(ImportPreview.expires_at < now)).all()
...
**SEC-13** `frontend/src/features/imports_exports/components/export-wizard.tsx:286`
...
    A exportação da camada de clientes contém dados sensíveis (nomes, telefones e endereços). Esta ação é
    restrita ao perfil de administrador e é registrada permanentemente na trilha de auditoria do sistema.
...
**SEC-14** `backend/app/modules/jobs/service.py:325`
...
    job.error_message = str(e)
...
**SEC-14** `backend/app/modules/jobs/service.py:48`
...
    error_message=job.error_message,
...
**SEC-14** `backend/app/api/v1/imports_exports.py:188`
...
    dependencies=[Depends(get_current_user)],
...

## Impacto
- **SEC-13:** Não-conformidade e impossibilidade de investigação forense; dados pessoais retidos além da necessidade.
- **SEC-14:** Divulgação de informação que auxilia outros ataques.

## Sugestão de correção
- **SEC-13:** Auditar criação e download de exportações (ator, camadas, IP); checar no download que o solicitante é o dono
ou admin; TTL configurável e job de limpeza dos arquivos.
- **SEC-14:** Persistir mensagem sanitizada/código de erro para o cliente e o detalhe apenas em log; exigir
`imports:read`/`exports:read` conforme o tipo do job.

## Como validar
- **SEC-13:** Exportar camada `customers` como admin e consultar `GET /audit-events`: hoje sem evento `export_*`.
- **SEC-14:** Provocar colisão de código em importação e ler `GET /jobs/{id}` como viewer.

## Critérios de aceite
- [ ] SEC-13: Teste: criar exportação gera `AuditEvent(action='export_requested')` (falha hoje).
- [ ] SEC-13: Teste: arquivo com mais de N dias é removido pelo worker; download retorna 410.
- [ ] SEC-14: `error_message` retornado não contém `SQL`, `psycopg` nem caminhos (`/app/`); viewer recebe 403.
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado
no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 10 ---

```

Issue 11 — [Segurança] Supply chain: advisories no frontend e CI sem scanners/gates

--- ISSUE 11 ---

Título: [Segurança] Supply chain: advisories no frontend e CI sem scanners/gates

Labels sugeridas: security, severity:média

Achados cobertos: SEC-15, EST-16

Descrição do problema e por que é explorável / degrada o sistema

SEC-15 – Dependências do frontend com advisories (maplibre-gl, postcss via next, vitest) (severidade: baixa)
`maplibre-gl@5.24.0` é vulnerável a bypass do sanitizador de HTML (`DOM.sanitize`), usado por `Popup.setHTML`; `grep -rNE`
`Popup|setHTML|setText` frontend/src → 0 usos. As de `postcss` vivem no toolchain do Next (build), e as de `vitest` são
dev-only.

- **Por que é explorável / degrada:** Se alguém introduzir `Popup.setHTML` com dados do inventário, o bypass viabiliza XSS;
as demais exigem controle de entradas de build.

- **Condições de ocorrência:** `pnpm audit` (2026-09-18): 7 advisories – 1 crítica (maplibre-gl ≤6.4.0, sanitizador do
`Popup`), 2 altas (postcss ≤8.5.17/8.5.11, em build via next), 4 moderadas. O código não usa `Popup`/`setHTML`, então a
crítica não é alcançável hoje.

- **Prova:** medido: `pnpm audit --json` em 2026-09-18 → `docs/security-audit/evidencias/pnpm-audit-frontend.json` (7
advisories). `pip-audit` no backend: 0 vulnerabilidades em 55 dependências resolvidas.

EST-16 – CI sem varredura de dependências, segredos ou SAST, sem build de imagens e sem `permissions:` mínimas
(severidade: média)

0 workflow roda ruff, mypy, pytest, restore drill, eslint, tsc e vitest, mas nenhum `pip-audit`/`pnpm audit`, secret scan,
CodeQL/Semgrep, `docker build` nem `trivy`. Actions são fixadas por tag (`@v4`), não por SHA, e o workflow não declara
`permissions:`. Resultado: o CI ficaria verde com os 7 advisories do frontend (SEC-15) e com o Dockerfile quebrado (EST-15).

- **Por que é explorável / degrada:** Regressões de segurança e de build chegam à branch principal sem sinal.

- **Condições de ocorrência:** Todo push/PR.

- **Prova:** verificado por leitura, não medido.

```
## Evidência
**SEC-15** `frontend/package.json:22`
...
    "maplibre-gl": "^5.1.0",
...
**SEC-15** `frontend/pnpm-lock.yaml:2031`
...
    maplibre-gl@5.24.0:
...
**EST-16** `.github/workflows/ci.yml:76`
...
    run: uv run pytest --cov=app --cov-report=xml -v
**EST-16** `.github/workflows/ci.yml:115`
...
    run: pnpm test
...
**EST-16** `.github/workflows/ci.yml:45`
...
    uses: actions/checkout@v4
...
**EST-16** `.github/workflows/ci.yml:9`
...
concurrency:
...
```

Impacto

- ****SEC-15:**** Baixo hoje; risco latente de supply chain (o CI não bloqueia advisories – EST-16).
- ****EST-16:**** Ausência de gates de qualidade de supply chain.

Sugestão de correção

- ****SEC-15:**** Atualizar maplibre-gl para ≥6.4.1 (major), `next`/`postcss` e `vitest`; ativar Dependabot/`pnpm audit` no CI.
- ****EST-16:**** Adicionar `pip-audit`, `pnpm audit --audit-level high`, gitleaks, CodeQL, `docker compose build`, trivy; `permissions: contents: read`; fixar actions por SHA.

Como validar

- ****SEC-15:**** `pnpm audit --prod` deve retornar 0 high/critical.
- ****EST-16:**** Executar os scanners localmente (feito neste relatório: `pip-audit` limpo; `pnpm audit` → 7).

Critérios de aceite

- [] SEC-15: `pnpm audit --audit-level high` retorna 0 no CI (falha hoje).
- [] EST-16: Job `security` falha o CI quando existe advisory high/critical (hoje inexistente).
- [] Existe ao menos um teste automatizado que ****falha antes**** da correção e ****passa depois**** (indicado acima), executado no CI.
- [] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 11 ---

Issue 12 — [Arquitetura] Cobertura de auditoria: eventos de identidade, inventário, óptico e exportações

--- ISSUE 12 ---

Título: [Arquitetura] Cobertura de auditoria: eventos de identidade, inventário, óptico e exportações

Labels sugeridas: architecture, severity:média

Achados cobertos: EST-10, SEC-17

Descrição do problema e por que é explorável / degrada o sistema

EST-10 – Trilha de auditoria cobre só anexos, conectividade, clientes e importação; usuários, inventário, cabos, óptico e exportações ficam de fora (severidade: média)

`record\_audit\_event` é usado apenas em `attachments`, `connectivity`, `customers` e `jobs`. Não há eventos para criar/alterar/desativar usuários e papéis, login/logout/troca de senha, CRUD de sites/estruturas/dispositivos/portas/cabos, perfis/medições ópticas, settings e exportações. Vários chamadores de anexos não propagam `request\_id`.

- ****Por que é explorável / degrada:**** Ações administrativas e alterações de inventário não são rastreáveis; a descrição da rota cria expectativa de cobertura total.

- ****Condições de ocorrência:**** Sempre. `GET /audit-events` é apresentada como 'histórico de mutações e ações de usuários no sistema'.

- ****Prova:**** medido (contagem): `grep -rn record\_audit\_event backend/app` → attachments 3, connectivity 7, customers 7, jobs 2 (uso incl. import); zero nos demais módulos.

SEC-17 – Cadeia: sessão não revogada + ausência de auditoria de auth/usuários = persistência invisível (severidade: média)

Mudanças de papel, criação/desativação de usuários, login, logout e troca de senha não geram `AuditEvent` (só `login\_attempts`). Sem revogação na troca de senha (SEC-08), o invasor mantém a sessão e pode, com papel admin, criar contas ou elevar papéis (`PATCH /users/{id}`) sem deixar trilha na auditoria da aplicação.

- ****Por que é explorável / degrada:**** Comprometer uma conta admin e criar um segundo admin não deixa evidência em `GET /audit-events`.

- ****Condições de ocorrência:**** Combina SEC-08 + EST-10. Requer uma sessão comprometida.

- ****Prova:**** verificado por leitura, não medido (`grep -rn record\_audit\_event backend/app` → apenas attachments, connectivity, customers, jobs).

Evidência

****EST-10**** `backend/app/modules/audit/service.py:45`

...

def record_audit_event(

...

****EST-10**** `backend/app/api/v1/reports.py:160`

...

description="Retorna histórico ordenado de mutações e ações de usuários no sistema.",

```

...
**EST-10** `backend/app/modules/identity/service.py:204`
...
def update_user_by_admin(
...
**EST-10** `backend/app/modules/inventory/service.py:112`
...
def create_site(session: Session, payload: SiteCreate) -> Site:
...
**SEC-17** `backend/app/modules/identity/service.py:259`
...
    user.role = payload.role.value
...
**SEC-17** `backend/app/modules/identity/service.py:182`
...
def create_user_by_admin(session: Session, payload: UserCreate) -> User:
...
**SEC-17** `backend/app/api/v1/reports.py:160`
...
    description="Retorna histórico ordenado de mutações e ações de usuários no sistema.",
...

## Impacto
- **EST-10:** Impossibilidade de responder 'quem alterou o quê' e de investigar incidentes (vide SEC-13, SEC-17).
- **SEC-17:** Persistência furtiva e investigação forense inviável.

## Sugestão de correção
- **EST-10:** Centralizar em um decorador/dependência (ou event listener do SQLAlchemy) que audite toda mutação com ator, request_id e diff sanitizado; tornar a tabela append-only no banco (REVOKE UPDATE/DELETE ou trigger).
- **SEC-17:** Auditar todos os eventos de identidade (login ok/falha, logout, criação/edição/desativação, troca de senha, mudança de papel) e revogar sessões na troca de senha.

## Como validar
- **EST-10:** `grep -rn record_audit_event backend/app | awk -F: '{print $1}' | sort | uniq -c`.
- **SEC-17:** Criar usuário via API e consultar `GET /audit-events?entity_type=user`.

## Critérios de aceite
- [ ] EST-10: Teste parametrizado: cada rota mutante gera exatamente 1 AuditEvent (falha hoje para ≥ 30 rotas).
- [ ] EST-10: Migração cria trigger que impede UPDATE/DELETE em `audit_events`.
- [ ] SEC-17: Teste: `POST /users` e `PATCH /users/{id}` (role) produzem AuditEvent (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

```

--- FIM ISSUE 12 ---

Issue 13 — [Arquitetura] O worker de jobs não inicia (ImportError) — importar/exportar está indisponível

--- ISSUE 13 ---

Título: [Arquitetura] O worker de jobs não inicia (ImportError) – importar/exportar está indisponível

Labels sugeridas: architecture, severity:alta

Achados cobertos: EST-01, EST-18

Descrição do problema e por que é explorável / degrada o sistema

EST-01 – O worker de jobs não inicia: ImportError em scripts/run_worker.py (importações e exportações nunca são processadas) (severidade: alta)

`run\_worker.py` importa `process\_claimed\_job` de `app.modules.jobs.service`, mas a função existente chama-se `process\_next\_job` (e faz o claim internamente). O processo termina com `ImportError` e o compose o reinicia em loop (`restart: unless-stopped`) sem healthcheck.

- **Por que é explorável / degrada:** Todos os jobs (`import\_commit`, `export\_\*`) ficam em `queued` para sempre; a UI de importação/exportação aguarda indefinidamente.

- **Condições de ocorrência:** Sempre que o serviço `worker` do compose sobe (comando `python scripts/run\_worker.py`). Os testes passam porque usam `process\_next\_job`, não o script.

- **Prova:** medido: `python -c "import scripts.run\_worker"` (importação, sem efeitos colaterais) → `ImportError: cannot import name 'process\_claimed\_job' from 'app.modules.jobs.service'`.

EST-18 – Worker sem logs JSON/correlation-id, sem healthcheck e sem métricas de jobs (severidade: baixa)

`run\_worker.py` usa `logging.basicConfig` com formato pseudo-JSON (quebra com aspas na mensagem), sem `request\_id`/`job\_id` de contexto; `metrics\_collector.record\_job` nunca é invocado; `worker` não tem `healthcheck` no compose; erros do job viram texto em `error\_message`.

- **Por que é explorável / degrada:** Falhas do worker (vide EST-01) passam despercebidas.

- **Condições de ocorrência:** Sempre.

- **Prova:** medido (grep): `grep -rn 'record\_job(' backend` → só a definição.

Evidência

```

**EST-01** `backend/scripts/run_worker.py:20`
...
    process_claimed_job,
...
**EST-01** `backend/scripts/run_worker.py:59`
...
    success = process_claimed_job(db, job, worker_id=worker_id)
...
**EST-01** `backend/app/modules/jobs/service.py:281`
...
def process_next_job(db: Session, worker_id: str = "worker-default") -> bool:
...

```

```

**EST-01** `compose.yaml:72`
...
... command: ["python", "scripts/run_worker.py"]
**EST-01** `backend/tests/integration/test_imports_exports_jobs.py:29`
...
from app.modules.jobs.service import claim_next_job, process_next_job
...
**EST-18** `backend/scripts/run_worker.py:25`
...
... format='{ "timestamp": "%(asctime)s", "level": "%(levelname)s", "logger": "%(name)s", "message": "%(message)s" }',
...
**EST-18** `backend/app/core/metrics.py:92`
...
... def record_job(self, job_type: str, status: str) -> None:
...

## Impacto
- **EST-01:** Funcionalidades de importação e exportação totalmente indisponíveis em produção; falha silenciosa (crash-loop sem alerta).
- **EST-18:** Detecção tardia de incidentes em filas.

## Sugestão de correção
- **EST-01:** Alinhar a API (`process_claimed_job(db, job, worker_id)` extraída de `process_next_job`) ou ajustar o script; adicionar teste que importa `scripts.run_worker` e healthcheck do worker (heartbeat em tabela/arquivo).
- **EST-18:** Reutilizar `setup_logging`, incluir `job_id`, incrementar `record_job` e expor heartbeat/healthcheck.

## Como validar
- **EST-01:** `docker compose logs worker` mostra `ImportError`; após a correção: criar export e observar `succeeded`.
- **EST-18:** `docker compose ps` (worker sem health) e `grep -rn 'record_job(' backend`.

## Critérios de aceite
- [ ] EST-01: Teste automatizado: `import scripts.run_worker` não levanta exceção (falha hoje).
- [ ] EST-01: Teste E2E: `POST /exports` → job chega a `succeeded` com o worker real.
- [ ] EST-01: Serviço `worker` com `healthcheck` no compose.
- [ ] EST-18: Log do worker é JSON válido por linha e contém `job_id` (teste de parsing).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 13 ---

```

Issue 14 — [Arquitetura] Importação: geometria de cabos descartada, lease sem heartbeat, storage fora do STORAGE_PATH e idempotência

--- ISSUE 14 ---

Título: [Arquitetura] Importação: geometria de cabos descartada, lease sem heartbeat, storage fora do STORAGE_PATH e idempotência

Labels sugeridas: architecture, severity:alta

Achados cobertos: EST-05, EST-06, EST-07, EST-08

Descrição do problema e por que é explorável / degrada o sistema

EST-05 – Importação descarta a geometria dos cabos (cria Cable sem segmento) e o CSV inventa coordenadas (severidade: alta)

`execute\_import\_commit` valida `coords` do cabo mas cria apenas `Cable(...)`; a geometria vive em `CableSegment.geometry` (a tabela `cables` não tem geometria) e nenhum segmento é criado. O contador `created\_cables` informa sucesso. No CSV, se `coordinates` faltar, `parse\_csv` injeta uma linha fictícia em São Paulo (`-46.6333,-23.5505`).

- **Por que é explorável / degrada:** Cabos importados nascem sem trecho/fibras ligadas a estruturas; relatórios de capacidade e rastreo os tratam como cabos 'sem rota'. Coordenadas inventadas podem ser validadas como reais.
- **Condições de ocorrência:** Todo `import\_commit` com entidades `cable` (GeoJSON/KML/CSV) após corrigir o worker (EST-01).
- **Prova:** verificado por leitura, não medido (o worker atual não roda – EST-01; a lógica foi lida e a ausência de `CableSegment` em `execute\_import\_commit` confirmada).

EST-06 – Idempotência do commit de importação: check-then-insert e chave global sem escopo (severidade: baixa)

O handler consulta `AsyncJob.idempotency\_key` e só depois insere; a corrida cai na `UNIQUE` como `IntegrityError` não tratado (HTTP 500). A chave não é escapada por usuário/`import\_id`: reutilizá-la devolve o `job\_id` de **outra** importação. Sem limite de tamanho (coluna VARCHAR(255) → 500).

- **Por que é explorável / degrada:** Retentativas legítimas geram 500; chaves iguais retornam o job alheio (vazamento de UUID e falso 'já processado').
- **Condições de ocorrência:** Duplo clique/retry concorrente com a mesma `Idempotency-Key`; ou chave reutilizada em outra importação.
- **Prova:** verificado por leitura, não medido.

EST-07 – Lease do job nunca é renovada: `heartbeat()` é código morto e importações longas podem ser reexecutadas por outro worker (severidade: média)

`claim\_next\_job` cria lease de 60 s; `heartbeat()` está definido mas nunca é chamado (`grep -rn 'heartbeat(' backend` → só a definição), e `execute\_import\_commit` não renova a lease. Qualquer outro worker reivindica jobs `running` com `lease\_expires\_at < now` e incrementa `retry\_count`.

- **Por que é explorável / degrada:** O job em execução é reprocessado em paralelo; ambos brigam pelos mesmos `code` (UNIQUE) e o status final é decidido pelo último `commit` (dados podem existir com status `failed`).
- **Condições de ocorrência:** 2+ workers e job com duração > 60 s (importação de arquivos grandes: ~2 round-trips por entidade, ver PERF-05).
- **Prova:** verificado por leitura, não medido.

EST-08 – Importações/exportações ignoram STORAGE_PATH: lêem `settings.STORAGE\_DIR`, atributo que não existe (severidade: média)

`getattr(settings, "STORAGE\_DIR", "storage")` sempre devolve o fallback relativo `storage` (a config define `STORAGE\_PATH`). `result\_path` e `file\_storage\_path` gravados no banco ficam relativos ao CWD do processo.

- ****Por que é explorável / degrada:**** Mudar `STORAGE\_PATH`, o `WORKDIR`, ou rodar o worker de outro diretório grava arquivos fora do volume persistente e quebra downloads/commits (`FileNotFoundError`).

- ****Condições de ocorrência:**** Sempre; funciona no compose apenas porque `WORKDIR=/app` faz o caminho relativo `storage/...` coincidir com o volume `/app/storage`.

- ****Prova:**** verificado por leitura, não medido.

Evidência

```

**EST-05** `backend/app/modules/jobs/service.py:239`
...
        coords = c_it["coords"]
...
**EST-05** `backend/app/modules/jobs/service.py:241-247`
...
        cable = Cable(
            code=c_it["code"],
            model=c_it.get("name") or "Standard Cable",
            fiber_count=int(c_it["props"].get("fiber_count") or 12),
            tube_count=int(c_it["props"].get("tube_count") or 2),
            status=c_it["props"].get("status", "installed"),
        )
...
**EST-05** `backend/app/modules/cables/models.py:24`
...
class Cable(Base, VersionedModelMixin):
...
**EST-05** `backend/app/modules/cables/models.py:139` - geometria existe apenas em CableSegment
...
    geometry: Mapped[Any] = mapped_column(
...
**EST-05** `backend/app/modules/imports/service.py:419-420`
...
        # Linha fictícia padrão se não informado
        coords = [[-46.6333, -23.5505], [-46.6334, -23.5506]]
...
**EST-06** `backend/app/modules/imports/service.py:602`
...
        existing_job = db.scalar(select(AsyncJob).where(AsyncJob.idempotency_key == idempotency_key))
...
**EST-06** `backend/app/modules/imports/models.py:43`
...
        idempotency_key: Mapped[str] = mapped_column(
...
**EST-07** `backend/app/modules/jobs/service.py:131`
...
def heartbeat(db: Session, job_id: uuid.UUID, worker_id: str, lease_seconds: int = 60) -> bool:
...
**EST-07** `backend/app/modules/jobs/service.py:96`
...
        AsyncJob.lease_expires_at < now,
...
**EST-07** `backend/app/modules/jobs/service.py:145`
...
def execute_import_commit(db: Session, job: AsyncJob) -> dict[str, Any]:
...
**EST-07** `backend/scripts/run_worker.py:55`
...
        job = claim_next_job(db, worker_id=worker_id, lease_seconds=60)
...
**EST-08** `backend/app/modules/imports/service.py:33`
...
        base_dir = os.path.join(getattr(settings, "STORAGE_DIR", "storage"), "imports")
...
**EST-08** `backend/app/modules/exports/service.py:28`
...
        base_dir = os.path.join(getattr(settings, "STORAGE_DIR", "storage"), "exports")
...
**EST-08** `backend/app/core/config.py:51`
...
        STORAGE_PATH: str = "./storage"
...
**EST-08** `compose.yaml:77`
...
        STORAGE_PATH: /app/storage
...

## Impacto
- **EST-05:** Perda silenciosa de dados de geometria e dados fabricados no inventário; falsa sensação de importação completa.
- **EST-06:** Falhas intermitentes e confusão de estado; baixo risco de duplicação (a UNIQUE protege).
- **EST-07:** Importações duplicadas/conflitantes e estados inconsistentes sob escala horizontal; hoje mascarado por haver 1 worker.
- **EST-08:** Perda de arquivos de importação/exportação em mudanças de configuração; comportamento diferente entre dev, CI e produção.

## Sugestão de correção
- **EST-05:** Criar `CableSegment` (com estruturas de origem/destino resolvidas) ou rejeitar cabos sem estruturas; nunca preencher coordenadas por default (marcar como erro de validação).
- **EST-06:** `INSERT ... ON CONFLICT (idempotency_key) DO NOTHING RETURNING` e comparar `payload.import_id`; escopar a chave por (user_id, import_id); validar tamanho.
- **EST-07:** Renovar a lease a cada N entidades (thread de heartbeat ou chamadas dentro do loop) e verificar `lease_owner`

```

antes de gravar o status final.

- **EST-08:** Usar `Path(settings.STORAGE\_PATH)/'imports'|'exports'` e persistir caminhos relativos ao storage root.

Como validar

- **EST-05:** Importar GeoJSON com um LineString e consultar `cable\_segments` e o relatório de inconsistências.
 - **EST-06:** Duas requisições simultâneas de commit com a mesma chave: hoje uma pode retornar 500.
 - **EST-07:** Dois workers + importação com atraso artificial > 60 s; observar duas execuções.
 - **EST-08:** `STORAGE\_PATH=/data python -c` → verificar que `get\_export\_storage\_path()` retorna `/data/exports`.

Critérios de aceite

- [] **EST-05:** Teste: após `import\_commit` com 1 cabo, existe ≥1 `CableSegment` com a geometria do arquivo (falha hoje).
 - [] **EST-05:** Teste: CSV de cabo sem `coordinates` → `validation\_status=error`.
 - [] **EST-06:** Teste concorrente: ambas retornam o mesmo `job\_id` e status 202/200 (falha hoje possível 500).
 - [] **EST-06:** Mesma chave com `import\_id` diferente → 409.
 - [] **EST-07:** Teste: job de 120 s simulados mantém `lease\_expires\_at` no futuro e é processado uma única vez (falha hoje).
 - [] **EST-08:** Teste unitário: com `STORAGE\_PATH=/tmp/x`, os dois helpers retornam subpastas de `/tmp/x` (falha hoje).
 - [] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
 - [] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- **FIM ISSUE 14** ---

Issue 15 — [Arquitetura] Autorização por padrão: rotas stub 501 sem auth e teste que varre todas as rotas

--- **ISSUE 15** ---

Título: [Arquitetura] Autorização por padrão: rotas stub 501 sem auth e teste que varre todas as rotas

Labels sugeridas: architecture, severity:média

Achados cobertos: EST-02, EST-20

Descrição do problema e por que é explorável / degrada o sistema

EST-02 – Rotas-stub 501 sem autenticação (allow-by-default): splitters, settings e occupancy (severidade: média)
 9 handlers ficam sem `Depends(get\_current\_user/require\_permission)` e sem `validate\_csrf`: 5 de `/splitters`, 2 de `/settings`, `structures/{id}/occupancy` (+ dashboard, SEC-01). Ao trocar `pending\_endpoint()` pela implementação, a rota nasce pública. Além disso, o frontend já possui páginas de settings que chamam essas rotas e recebem 501.
 - **Por que é explorável / degrada:** Padrão estrutural: autorização é opt-in por decorator; não existe camada única (router-level dependency / política padrão negar).
 - **Condições de ocorrência:** Hoje respondem 501. O risco materializa-se quando a implementação for preenchida (o contrato OpenAPI já as expõe sem `security`).
 - **Prova:** medido: `tools/enum\_routes.py` – 16 handlers sem dependência de auth (7 públicos por design + 9 desta classe).

EST-20 – Matriz de permissões duplicada em backend e frontend (com divergência) e gates de UI inexistentes (severidade: baixa)

A matriz existe em `permissions.py` e em `rbac.ts` (cópias manuais; `telemetry:write` só no frontend). `AuthGuard`, `PermissionGate` e o campo `permission` da navegação nunca são usados: a UI mostra ações que o servidor recusará (UX) e não há fonte única.

- **Por que é explorável / degrada:** Divergência silenciosa entre o que a UI acredita e o que o servidor impõe; sem teste que compare os dois.
 - **Condições de ocorrência:** Sempre.
 - **Prova:** medido (grep): `grep -rn 'PermissionGate|requiredPermission' frontend/src` fora de `components/auth/` → 0 usos.

Evidência

```
**EST-02** `backend/app/api/v1/splitters.py:9`
...

splitters_router = APIRouter(prefix="/splitters", tags=["Splitters"])
...

**EST-02** `backend/app/api/v1/settings.py:8`
...

settings_router = APIRouter(prefix="/settings", tags=["Configurações da Organização"])
...

**EST-02** `backend/app/api/v1/inventory.py:269`
...

def get_structure_occupancy(structure_id: str) -> StructureOccupancyResponse:
...

**EST-02** `backend/app/core/contracts.py:6`
...

def pending_endpoint(stage: str) -> NoReturn:
...

**EST-02** `backend/app/api/v1/router.py:21`
...

api_v1_router = APIRouter()
...

**EST-20** `backend/app/core/permissions.py:4`
...

ROLE_PERMISSIONS: dict[UserRole, set[str]] = {
...

**EST-20** `frontend/src/lib/permissions/rbac.ts:35`
...

export const ROLE_PERMISSIONS: Record<UserRole, readonly Permission[]> = {
...

**EST-20** `frontend/src/lib/permissions/rbac.ts:49`
...

... "telemetry:write",
...

**EST-20** `frontend/src/app/(app)/layout.tsx:6`
...

<AuthGuard>
...
```

```
## Impacto
- **EST-02:** Novos endpoints 'esquecem' o controle por padrão (o mesmo mecanismo gerou SEC-01).
- **EST-20:** Manutenção frágil; erros de UX; risco de decisões de segurança baseadas na UI.

## Sugestão de correção
- **EST-02:** Aplicar `dependencies=[Depends(get_current_user)]` no `APIRouter` raiz (`api_v1_router`) com allowlist explícita para login/csrf/health; teste que percorre `app.routes` e falha para rotas sem auth.
- **EST-20:** Gerar `rbac.ts` a partir do backend (endpoint `/auth/me` já devolve `permissions`) e aplicar `PermissionGate` nas ações de escrita; teste de paridade.

## Como validar
- **EST-02:** `python docs/security-audit/tools/enum_routes.py` e listar handlers sem dependência de auth.
- **EST-20:** Comparar chaves de `ROLE_PERMISSIONS` das duas linguagens.

## Critérios de aceite
- [ ] EST-02: Teste automatizado (percorre `effective_route_contexts`) falha hoje com 11 rotas e passa depois.
- [ ] EST-02: Rotas stub retornam 401 sem sessão.
- [ ] EST-20: Teste de paridade backend×frontend passa; UI oculta ações sem permissão.
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 15 ---
```

Issue 16 — [Arquitetura] Concorrência: If-Match atômico e divisão de segmento com lock/revisão

--- ISSUE 16 ---

Título: [Arquitetura] Concorrência: If-Match atômico e divisão de segmento com lock/revisão

Labels sugeridas: architecture, severity:média

Achados cobertos: EST-03, EST-04, EST-21

```
## Descrição do problema e por que é explorável / degrada o sistema
### EST-03 – Concorrência otimista (If-Match) é check-then-act: dois PATCH com a mesma versão passam (lost update)
(severidade: média)
`_validate_if_match` compara `current_version` lida em Python e só depois o handler altera e faz `version += 1`; o `UPDATE` não tem `WHERE version = :esperada` e a linha não é bloqueada (`with_for_update`). `VersionedModelMixin.version` é uma coluna comum (sem `version_id_col`).
- **Por que é explorável / degrada:** Duas requisições simultâneas com `If-Match: 3` leem versão 3, ambas passam e a última grava sobrescrevendo a primeira sem 412.
- **Condições de ocorrência:** Edições concorrentes do mesmo recurso (usuário, site, estrutura, dispositivo, porta, cabo, cliente, medição, perfil óptico).
- **Prova:** verificado por leitura, não medido. Como medir: `concurrent.futures` com dois PATCH simultâneos contra Postgres descartável.

### EST-04 – Divisão de segmento de cabo sem lock, sem If-Match e sem revisão topológica esperada (severidade: média)
`split_cable_segment` lê o segmento sem `FOR UPDATE`, cria dois novos segmentos com todos os terminais e conexões, e ao final `db.delete(segment)`. O handler não recebe `If-Match` nem `expected_topology_revision` (ao contrário do editor de fusão, que trava a estrutura e valida a revisão). `bump_topology_revision` só ocorre no fim.
- **Por que é explorável / degrada:** Duas transações concorrentes criam 2+2 segmentos e 2× terminais/fibras para o mesmo trecho; o segundo `DELETE` afeta 0 linhas (apenas aviso do SQLAlchemy) e ambos comitam.
- **Condições de ocorrência:** Dois usuários dividem o mesmo segmento ao mesmo tempo (ou duplo clique/reenvio).
- **Prova:** verificado por leitura, não medido (a corrida depende de duas transações reais).

### EST-21 – Lógica crítica duplicada: 6 cópias da validação de If-Match e regra/consulta dentro de handlers (severidade: baixa)
`_validate_if_match` existe copiado em inventory, customers, connectivity, measurements e optical, e `_check_optimistic_lock` em cables (mensagens diferentes). Além disso o handler `download_export` consulta o banco, valida estado do job, resolve content-type e caminho de arquivo; parte da lógica de auditoria também é importada dentro do handler (`reports.py`).
- **Por que é explorável / degrada:** Cada cópia pode divergir (já divergem nas mensagens/tratamento de aspas); corrigir a concorrência otimista exige editar 6 lugares; regra no handler dificulta testes unitários.
- **Condições de ocorrência:** Sempre; amplifica EST-03 (a correção precisa ser replicada em 6 módulos).
- **Prova:** medido (grep + AST): 6 definições duplicadas. Verificação de ciclos: análise AST do grafo de imports de `backend/app` → **nenhum ciclo em nível de módulo** (há acoplamento entre pacotes gis→inventory e 5 imports locais em `optical/service.py:220-231`, sem ciclo).
```

```
## Evidência
**EST-03** `backend/app/modules/inventory/service.py:38`
...
def _validate_if_match(if_match: str | None, current_version: int) -> None:
...
**EST-03** `backend/app/modules/inventory/service.py:145`
...
    _validate_if_match(if_match, site.version)
...
**EST-03** `backend/app/modules/inventory/service.py:160`
...
    site.version += 1
...
**EST-03** `backend/app/db/base.py:39`
...
    version: Mapped[int] = mapped_column(
...
**EST-03** `backend/app/modules/identity/service.py:224`
...
    if user.version != expected_version:
```

```

...
**EST-04** `backend/app/modules/cables/service.py:576`
...
def split_cable_segment(
...
**EST-04** `backend/app/modules/cables/service.py:591`
...
    segment = get_cable_segment_by_id(db, segment_id)
...
**EST-04** `backend/app/modules/cables/service.py:788`
...
    db.delete(segment)
...
**EST-04** `backend/app/api/v1/cables.py:246-251`
...
def split_segment(
    segment_id: str,
    payload: SegmentSplitRequest,
    current_user: User = Depends(require_permission("network:write")),
    db: Session = Depends(get_db),
) -> Any:
...
**EST-04** `backend/app/modules/connectivity/service.py:439` – contraste: o editor de fusão trava a estrutura
...
    select(Structure).where(Structure.id == struct_id).with_for_update()
...
**EST-21** `backend/app/modules/inventory/service.py:38`
...
def _validate_if_match(if_match: str | None, current_version: int) -> None:
...
**EST-21** `backend/app/modules/customers/service.py:31`
...
def _validate_if_match(if_match: str | None, current_version: int) -> None:
...
**EST-21** `backend/app/modules/connectivity/service.py:42`
...
def _validate_if_match(if_match: str | None, current_version: int) -> None:
...
**EST-21** `backend/app/modules/measurements/service.py:29`
...
def _validate_if_match(if_match: str | None, current_version: int) -> None:
...
**EST-21** `backend/app/modules/optical/service.py:30`
...
def _validate_if_match(if_match: str | None, current_version: int) -> None:
...
**EST-21** `backend/app/modules/cables/service.py:47`
...
def _check_optimistic_lock(current_version: int, if_match: str | None) -> None:
...
**EST-21** `backend/app/api/v1/imports_exports.py:146`
...
    job = db.get(AsyncJob, uid)
...

## Impacto
- **EST-03:** Perda silenciosa de edições no inventário; o contrato de 412 dá falsa garantia.
- **EST-04:** Topologia duplicada/corrompida (fibras com terminais órfãos), afetando rastreamento e impacto.
- **EST-21:** Risco de correção parcial e de comportamento inconsistente entre recursos.

## Sugestão de correção
- **EST-03:** Usar `version_id_col` do SQLAlchemy (ou `UPDATE ... WHERE id=:id AND version=:v` verificando `rowcount`) e/ou `SELECT ... FOR UPDATE` antes da validação.
- **EST-04:** `SELECT ... FOR UPDATE` no segmento (e nas fibras), exigir `If-Match`/`expected_topology_revision`, e falhar se o segmento já não existir.
- **EST-21:** Extrair `core/concurrency.py` (`assert_version(current, if_match)` + atualização atômica) e mover a lógica de download para `modules/imports_exports/service.py`.

## Como validar
- **EST-03:** Teste com duas threads: dois `PATCH /sites/{id}` com o mesmo `If-Match`; hoje ambos 200.
- **EST-04:** Teste com dois `POST .../split` simultâneos no mesmo segmento em Postgres descartável; hoje ambos 200.
- **EST-21:** `grep -rn '_validate_if_match\|_check_optimistic_lock' backend/app`: 6 definições hoje; alvo 1.

## Critérios de aceite
- [ ] EST-03: Teste de concorrência: exatamente um dos dois PATCH retorna 200 e o outro 412 (falha hoje).
- [ ] EST-04: Teste de concorrência: somente uma divisão tem sucesso; a outra recebe 409/412; contagem de segmentos = 2 (falha hoje).
- [ ] EST-21: Uma única implementação de verificação de versão (teste que falha se houver definição duplicada; falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

```

--- FIM ISSUE 16 ---

Issue 17 — [Arquitetura] Uploads: consistência disco×banco e reconciliador seguro

--- ISSUE 17 ---

Título: [Arquitetura] Uploads: consistência disco×banco e reconciliador seguro

Labels sugeridas: architecture, severity:média

Achados cobertos: EST-09

Descrição do problema e por que é explorável / degrada o sistema

EST-09 – Upload grava em disco antes do commit; falhas geram arquivos órfãos e o reconciliador pode apagar upload em andamento (severidade: média)

`save\_attachment` escreve o original (e a miniatura) antes de `db.commit()`. Qualquer exceção posterior deixa o arquivo sem registro. `reconcile\_storage\_orphans` remove todo arquivo não presente no snapshot lido do banco, incluindo o de um upload que ainda não commitou.

- **Por que é explorável / degrada:** Uploads falhos acumulam lixo (até 10 MiB por tentativa – combinável com PERF-04); a reconciliação concorrente apaga o arquivo e o commit posterior aponta para um arquivo inexistente.
- **Condições de ocorrência:** Exceção após a escrita (ex.: `DecompressionBombError`, falha no commit) ou `reconcile-orphans` executado durante um upload.
- **Prova:** medido (parte): `tools/measure4.py` – PNG 14000×14000 (23,3 KiB) → `DecompressionBombError` não capturado em `generate\_thumbnail\_image`; a escrita do original ocorre antes (linha 215). Corrida com reconciliador: verificado por leitura, não medido.

Evidência

```

**EST-09** `backend/app/modules/attachments/service.py:215`
...
    original_target_path.write_bytes(raw_content)
...
**EST-09** `backend/app/modules/attachments/service.py:263`
...
    db.refresh(attachment)
...
**EST-09** `backend/app/modules/attachments/service.py:433`
...
    if rel not in known_relative_paths:
...
**EST-09** `backend/app/modules/attachments/service.py:127`
...
    except (UnidentifiedImageError, OSError, ValueError):
...

```

Impacto

- **EST-09:** Consumo de disco por tentativas maliciosas/falhas e, mais raramente, perda de anexo recém-enviado.

Sugestão de correção

- **EST-09:** Gravar em arquivo temporário e `os.replace` após o commit (ou registrar no banco primeiro e fazer GC por idade); reconciliador ignora arquivos com mtime recente.

Como validar

- **EST-09:** Enviar PNG que dispare `DecompressionBombError` (PERF-04) e listar `storage/attachments/originals`: arquivo permanece.

Critérios de aceite

- [] EST-09: Teste: falha na geração de miniatura não deixa arquivo em `originals/` (falha hoje).
- [] EST-09: Teste: reconciliador não remove arquivo com menos de N minutos.
- [] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 17 ---

Issue 18 — [Arquitetura] Tetos de entrada e rate limit: fiber_count ilimitado, corpo sem limite, endpoints caros

--- ISSUE 18 ---

Título: [Arquitetura] Tetos de entrada e rate limit: fiber_count ilimitado, corpo sem limite, endpoints caros

Labels sugeridas: architecture, severity:alta

Achados cobertos: EST-11

Descrição do problema e por que é explorável / degrada o sistema

EST-11 – Sem tetos de tamanho e sem rate limit fora do login: `fiber\_count` ilimitado derruba o processo com 1 request (severidade: alta)

`CableCreate.fiber\_count` tem `ge=1` e nenhum `le`: `create\_cable` instancia uma linha ORM `Fiber` por fibra numa única transação. Também: `await file.read()` lê todo o upload em memória antes de checar o limite; não há `request\_body max\_size` no Caddy nem limite no uvicorn; listas de entrada (`cable\_segment\_ids`, `cut\_fiber\_ids`) e campos de texto (`notes`) são ilimitados; nenhum endpoint além do login tem rate limit (inclusive os caros: trace, impact, budgets, simulations, search).

- **Por que é explorável / degrada:** `POST /cables` com `fiber\_count` de milhões consome CPU/RAM até OOM; uploads de vários GB são bufferizados na memória do processo.
- **Condições de ocorrência:** Papel engineer/admin (`network:write`) – ou sessão comprometida. Body sem limite no Caddy/uvicorn; uploads lidos por inteiro.
- **Prova:** medido: `tools/measure6.py` (Postgres descartável) – `fiber\_count=10.000`: 1,5 s, +50 MiB; `40.000`: 5,9 s, +125 MiB (≈ 0,15 ms e ≈ 2,5 KiB/fibra; extrapolação linear, não medida: 1 milhão ≈ 150 s e ≈ 2,5 GiB).

Evidência

```

**EST-11** `backend/app/schemas/cables.py:22`
...
    fiber_count: int = Field(..., ge=1, description="Quantidade total de fibras ópticas no cabo")
...

```

```

**EST-11** `backend/app/modules/cables/service.py:136`
...
    for pos in range(1, fibers_in_this_tube + 1):
**EST-11** `backend/app/api/v1/attachments.py:66`
...
    raw_content = await file.read()
...
**EST-11** `backend/app/api/v1/imports_exports.py:47`
...
    content = await file.read()
...
**EST-11** `backend/app/modules/identity/service.py:32` – único rate limit do sistema
...
RATE_LIMIT_MAX_ATTEMPTS = 5
...
**EST-11** `Caddyfile:20`
...
    reverse_proxy backend:8000
...

## Impacto
- **EST-11:** Negação de serviço por 1 requisição autenticada; sem defesa em camadas (proxy, app, banco).

## Sugestão de correção
- **EST-11:** `le=` em `fiber_count`/`tube_count` (ex.: 1728/144), `max_length` em listas e textos; `request_body { max_size }` no Caddy e streaming com limite no upload; rate limit por IP/usuário nos endpoints caros.

## Como validar
- **EST-11:** `tools/measure6.py`: tempo/memória por `fiber_count`; após a correção, 422 para valores acima do teto.

## Critérios de aceite
- [ ] EST-11: `POST /cables` com `fiber_count=1_000_000` → 422 (falha hoje).
- [ ] EST-11: Upload > `MAX_UPLOAD_SIZE_BYTES` é rejeitado com 413 sem ler o corpo inteiro (teste com stream).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 18 ---

```

Issue 19 — [Arquitetura] UUID inválido retorna 500 e campos sem max_length

--- ISSUE 19 ---

Título: [Arquitetura] UUID inválido retorna 500 e campos sem max_length

Labels sugeridas: architecture, severity:baixa

Achados cobertos: EST-12

```

## Descrição do problema e por que é explorável / degrada o sistema
### EST-12 – UUID inválido em path/query/body resulta em HTTP 500 (ValueError não tratado) e strings sem max_length quebram no banco (severidade: baixa)
Nos routers de clientes, atendimentos e conectividade, `uuid.UUID(param)` é chamado sem `try/except` (17 pontos, 14 rotas). `topology/trace` e `create_service_link` fazem o mesmo. `CustomerUpdate.phone/email/address` não têm `max_length` enquanto as colunas são VARCHAR(30/100/255) → `DataError` → 500.
- **Por que é explorável / degrada:** Entrada malformada produz 500 (ruído em métricas e alertas, log de stack trace) em vez de 422/404; mascara falhas reais.
- **Condições de ocorrência:** Qualquer chamada autenticada com `customer_id=abc` etc.
- **Prova:** medido: `tools/measure2.py` – `GET /api/v1/customers/not-a-uuid` (sessão viewer) → HTTP 500 (`application/problem+json`, `internal_server_error`).

```

Evidência

```

**EST-12** `backend/app/api/v1/customers.py:82`
...
    return service.get_customer_by_id(db, uuid.UUID(customer_id))
...
**EST-12** `backend/app/modules/topology/service.py:105`
...
    start_uuid = uuid.UUID(request.start_terminal_id)
...
**EST-12** `backend/app/modules/customers/service.py:285`
...
    customer_uuid = uuid.UUID(payload.customer_id)
...
**EST-12** `backend/app/schemas/customers.py:28`
...
    phone: str | None = None
...
**EST-12** `backend/app/modules/customers/models.py:25`
...
    phone: Mapped[str | None] = mapped_column(String(30), nullable=True)
...

```

Impacto

- **EST-12:** Observabilidade degradada e superfície de erro 5xx controlável pelo cliente.

Sugestão de correção

- **EST-12:** Tipar parâmetros como `uuid.UUID` nos handlers (FastAPI valida e responde 422) e adicionar `max\_length` alinhado às colunas.

```
## Como validar
- **EST-12:** `GET /api/v1/customers/not-a-uuid` com sessão: hoje 500.

## Critérios de aceite
- [ ] EST-12: Teste: `GET /customers/not-a-uuid` → 422 (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 19 ---
```

Issue 20 — [Arquitetura] Métricas em memória: cardinalidade ilimitada de paths 404 e visão por processo

--- ISSUE 20 ---

Título: [Arquitetura] Métricas em memória: cardinalidade ilimitada de paths 404 e visão por processo

Labels sugeridas: architecture, severity:média

Achados cobertos: EST-13

```
## Descrição do problema e por que é explorável / degrada o sistema
### EST-13 – Métricas em memória com cardinalidade ilimitada: paths 404 distintos crescem o dicionário sem limite (e são por-processo) (severidade: média)
`RequestIDMiddleware` registra `request.url.path` quando a rota não foi resolvida (`route_format=None`);
`normalize_route_path` só troca UUIDs e números por `{id}`. Cada path 404 diferente cria chaves novas em `_http_requests` e `_http_durations` que nunca são removidas. As métricas também são locais ao processo (o worker não aparece; `record_job` nunca é chamado).
- **Por que é explorável / degrada:** Um scanner (ou atacante) fazendo `GET /a1`, `/a2`... incha a memória do processo indefinidamente e o endpoint `/metrics`.
- **Condições de ocorrência:** Qualquer cliente (não requer autenticação): requisições a caminhos inexistentes.
- **Prova:** medido: `tools/measure2.py` – chaves de `_http_requests`: 8 antes → 308 depois de 300 GETs a paths 404 distintos (+300).
```

```
## Evidência
**EST-13** `backend/app/core/middleware.py:37`
...
    route_format = getattr(route, "path_format", None)
...
**EST-13** `backend/app/core/metrics.py:30`
...
    clean = UUID_PATTERN.sub("{id}", path)
...
**EST-13** `backend/app/core/metrics.py:74`
...
    self._http_requests[req_key] = self._http_requests.get(req_key, 0) + 1
...
**EST-13** `backend/app/core/metrics.py:92`
...
    def record_job(self, job_type: str, status: str) -> None:
...
```

```
## Impacto
- **EST-13:** Vazamento de memória controlável por anônimos; métricas incompletas sob múltiplos processos/réplicas.
```

```
## Sugestão de correção
- **EST-13:** Usar um rótulo fixo (`__unmatched__`) quando não houver rota; limitar o número de chaves; exportar métricas por processo com agregação (prometheus multiprocessing) ou instrumentar o worker.
```

```
## Como validar
- **EST-13:** `tools/measure2.py`: 300 GETs a paths 404 distintos.
```

```
## Critérios de aceite
- [ ] EST-13: Teste: 10.000 paths 404 distintos deixam `len(_http_requests)` constante (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
```

--- FIM ISSUE 20 ---

Issue 21 — [Arquitetura] SPOF e estado local: preparar escala horizontal (workers, storage, backup, HA)

--- ISSUE 21 ---

Título: [Arquitetura] SPOF e estado local: preparar escala horizontal (workers, storage, backup, HA)

Labels sugeridas: architecture, severity:média

Achados cobertos: EST-14

```
## Descrição do problema e por que é explorável / degrada o sistema
### EST-14 – SPOF e estado local: 1 processo uvicorn, 1 backend com `container_name` fixo, storage em volume local, métricas por processo (severidade: média)
`CMD uvicorn` sem `--workers`; `container_name: ftth_backend` impede `--scale`; anexos/exportações vivem em volume Docker local (`backend_storage`) compartilhado apenas no mesmo host; métricas em dicionário do processo (EST-13); PostgreSQL único sem réplica/backup automático (há `scripts/backup.sh` manual); worker sem healthcheck. Rate limit de login e sessões estão no banco (**correto**: escalam).
- **Por que é explorável / degrada:** Queda do container/host derruba tudo; réplicas em outro host não veem os arquivos; sem redundância.
```

- ****Condições de ocorrência:**** Ao tentar escalar (2+ réplicas/hosts) ou em falha de um único container.
- ****Prova:**** verificado por leitura, não medido.

Evidência

```

**EST-14** `backend/Dockerfile:59`
...
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
...
**EST-14** `compose.yaml:39`
...
    container_name: ftth_backend
...
**EST-14** `compose.yaml:61`
...
    - backend_storage:/app/storage
...
**EST-14** `backend/app/core/metrics.py:43`
...
    self._http_requests: dict[tuple[str, str, int], int] = {}
...

```

Impacto

- ****EST-14:**** Disponibilidade limitada a 1 nó; risco de perda de anexos por falha de disco.

Sugestão de correção

- ****EST-14:**** Documentar/implementar: `--workers N` ou múltiplas réplicas sem `container\_name`; storage em objeto (S3/MinIO) ou NFS; backup agendado + teste de restauração; réplica de leitura/HA do banco; healthcheck do worker.

Como validar

- ****EST-14:**** `docker compose up --scale backend=2` → falha por `container\_name`.

Critérios de aceite

- [] EST-14: `docker compose up --scale backend=2` sobe duas réplicas saudáveis atrás do Caddy.
- [] EST-14: Anexos acessíveis por qualquer réplica (teste de download após upload em outra réplica).
- [] Existe ao menos um teste automatizado que ****falha antes**** da correção e ****passa depois**** (indicado acima), executado no CI.
- [] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 21 ---

Issue 22 — [Arquitetura] Build do frontend quebrado: COPY de /app/public inexistente

--- ISSUE 22 ---

Título: [Arquitetura] Build do frontend quebrado: COPY de /app/public inexistente

Labels sugeridas: architecture, severity:alta

Achados cobertos: EST-15

Descrição do problema e por que é explorável / degrada o sistema

EST-15 – Build do frontend falha: Dockerfile copia /app/public, diretório que não existe (severidade: alta)
 O estágio `runner` faz `COPY --from=builder /app/public ./public`, mas `frontend/public` não existe e não há arquivos rastreados nele (`git ls-files frontend | grep -c '^frontend/public'` → 0). O Next.js não cria `public`.
 - ****Por que é explorável / degrada:**** BuildKit aborta com `"/app/public": not found`; a stack oficial não sobe do zero.
 - ****Condições de ocorrência:**** `docker compose build frontend` / `up --build` a partir do repositório.
 - ****Prova:**** reproduzido em Dockerfile equivalente isolado (`tools/repro-copy-missing-dir/Dockerfile`): `ERROR ... "/app/public": not found`. O build real do frontend não foi executado (pnpm install pesado); a ausência do diretório foi verificada no working tree e no git.

Evidência

```

**EST-15** `frontend/Dockerfile:25`
...
COPY --from=builder /app/public ./public
...

```

Impacto

- ****EST-15:**** Bloqueio total do deploy pelo caminho oficial (imagem do frontend não é gerada).

Sugestão de correção

- ****EST-15:**** Criar `frontend/public/.gitkeep` ou remover a linha `COPY ... /app/public` (ou torná-la condicional). Adicionar `docker build` ao CI.

Como validar

- ****EST-15:**** `docker build -f frontend/Dockerfile frontend` deve concluir.

Critérios de aceite

- [] EST-15: Job de CI `docker compose build` passa (falha hoje).
- [] Existe ao menos um teste automatizado que ****falha antes**** da correção e ****passa depois**** (indicado acima), executado no CI.
- [] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 22 ---

Issue 23 — [Performance] Dashboard com N+1 (2.514 queries) — agregar em SQL e cachear

--- ISSUE 23 ---

Título: [Performance] Dashboard com N+1 (2.514 queries) – agregar em SQL e cachear

Labels sugeridas: performance, severity:alta

Achados cobertos: PERF-01

Descrição do problema e por que é explorável / degrada o sistema

PERF-01 – Dashboard: 2 queries por CTO (N+1) – 2.514 statements e ~1 s com 10 mil estruturas, sem autenticação (severidade: alta)

`calculate\_dashboard\_summary` carrega todas as CTOs e, dentro do `for cto in ctos`, executa `SELECT ports` e `SELECT service\_links` por CTO; as contagens poderiam ser um único `GROUP BY`. O endpoint não exige autenticação (SEC-01).

- **Por que é explorável / degrada:** Custo por request O(CTOs) em round-trips; concorrência satura o único processo e o pool de conexões (SEC-16).

- **Condições de ocorrência:** Cada requisição a `GET /api/v1/dashboard/summary`; cresce com o nº de CTOs não aposentadas.

- **Gatilho de escala:** Dói desde ~1 s/request em 10 mil estruturas (medido); a partir de dezenas de requisições/s (dashboard abre a cada carga de UI) satura 1 worker.

- **Ordem de crescimento:** O(C) queries por request (C = nº de CTOs); 2 statements por CTO.

- **Prova:** medido: `tools/measure.py` (Postgres 16/PostGIS descartável, 10.001 estruturas, 5.050 cabos, 103.200 fibras, 206.428 terminais) – `GET /api/v1/dashboard/summary`: run1=1.524 ms, run2=1.018 ms, run3=1.129 ms; **2.514 SQL statements por requisição**.

Evidência

PERF-01 `backend/app/modules/reports/service.py:48`

```

for cto in ctos:

```

PERF-01 `backend/app/modules/reports/service.py:49`

```

ports = db.scalars(select(Port).where(Port.structure\_id == cto.id)).all()

```

PERF-01 `backend/app/modules/reports/service.py:57`

```

active\_links = db.scalars(

```

Impacto

- **PERF-01:** Latência ~1 s com 10 mil estruturas (medido), degradação linear e amplificação de DoS.

Sugestão de correção

- **PERF-01:** Uma consulta agregada (`COUNT ... GROUP BY structure\_id` com `LEFT JOIN` de vínculos ativos) e cache de 15–30 s (invalidação pela `topology\_revision`).

Como validar

- **PERF-01:** `tools/measure.py` (contador de statements) antes/depois; alvo ≤ 10 statements e p95 < 200 ms com 10k estruturas.

- Comando de medição (executar **antes** e **depois** da correção; em banco descartável):

```
DATABASE_URL=postgresql+psycopg://USUARIO:SENHA@127.0.0.1:55432/audit PYTHONPATH=backend python
docs/security-audit/tools/measure.py # contador de statements + tempo do dashboard (antes: 2.514 statements, 1,0–1,5 s)
DATABASE_URL=... python docs/security-audit/tools/measure5.py # 20 requisições concorrentes (antes: wall 16,9 s)
```

Critérios de aceite

- [] PERF-01: Teste com contador de statements: `/dashboard/summary` executa ≤ 10 queries com 10.001 estruturas (falha hoje: 2.514).

- [] PERF-01: Benchmark `tools/measure5.py`: 20 concorrentes concluem em < 3 s (hoje 16,9 s).

- [] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.

- [] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 23 ---

Issue 24 — [Performance] Análise de impacto e rastreo óptico: N+1 por cliente/salto

--- ISSUE 24 ---

Título: [Performance] Análise de impacto e rastreo óptico: N+1 por cliente/salto

Labels sugeridas: performance, severity:alta

Achados cobertos: PERF-02, PERF-14

Descrição do problema e por que é explorável / degrada o sistema

PERF-02 – Análise de impacto: carrega todos os clientes e roda um rastreo óptico completo por vínculo ativo (62 statements por vínculo) (severidade: alta)

`analyze\_cable\_impact` faz `select(Customer)` de toda a base, e para cada vínculo ativo executa consultas de CTO/ONU/terminal e chama `trace\_optical\_path` (BFS com várias queries por salto). `cable\_segment\_ids` não tem `max\_length`. - **Por que é explorável / degrada:** O custo total é O(clientes × saltos × queries/salto) numa única requisição síncrona de um papel de baixo privilégio, sem rate limit.

- **Condições de ocorrência:** `POST /api/v1/topology/impact` (papel viewer basta). Cresce com nº de clientes com vínculo ativo × saltos.

- **Gatilho de escala:** Custo proporcional a C (clientes ativos): medido 62 statements e ~80–104 ms para **1** vínculo; com C=1.000 ≈ 62.000 statements (extrapolação linear, não medida).

- **Ordem de crescimento:** O(C × H × q): C clientes ativos, H saltos do trajeto, q=5 queries/salto.

- **Prova:** medido: `tools/measure2.py` – viewer, 1 cliente / 1 vínculo: **62 SQL statements**, 104 ms e 80 ms (2 execuções). A partir daí, escala linear é extrapolação, rotulada como tal.

```

### PERF-14 – Rastreo óptico: ≥ 5 queries por salto e cópia de listas por ramo; max_hops até 500 e max_results até 200
vindos do cliente (severidade: média)
Cada iteração do BFS executa `db.get(Terminal)`, `db.scalars(FiberSegment)`, `db.scalars(Connection)`,
`db.scalars(InternalEdge)`, splitters e checagem de endpoint; ramos copiam `current_steps + [next_step]` ( $O(H^2)$  por
caminho). O teto configurado `MAX_TRACE_HOPS=300` não é aplicado (EST-17).
- **Por que é explorável / degrada:** Custo  $O(\text{caminhos} \times \text{saltos} \times \text{queries})$  controlado pelo cliente dentro do teto de 200×500.
- **Condições de ocorrência:** `POST /topology/trace`, `/optical/budgets`, `/optical/simulations` (papel viewer) e o impacto
(PERF-02).
- **Gatilho de escala:** Topologias com splitters em cascata e milhares de ONUs por PON (não medido).
- **Ordem de crescimento:**  $O(P \cdot H \cdot q)$  queries;  $O(P \cdot H^2)$  memória de steps.
- **Prova:** verificado por leitura, não medido (o dataset sintético do repositório tem só 5 conexões ativas, insuficiente
para medir o rastreo em escala).

## Evidência
**PERF-02** `backend/app/modules/topology/service.py:526`
...
    all_customers = db.scalars(
...
**PERF-02** `backend/app/modules/topology/service.py:541`
...
    for link in active_links:
**PERF-02** `backend/app/modules/topology/service.py:587`
...
        trace_res = trace_optical_path(db, trace_req)
**PERF-02** `backend/app/schemas/topology.py:98`
...
        cable_segment_ids: list[str] = Field(
...
**PERF-14** `backend/app/modules/topology/service.py:181`
...
        fiber_segs = db.scalars(
**PERF-14** `backend/app/modules/topology/service.py:224`
...
        connections = db.scalars(
**PERF-14** `backend/app/modules/topology/service.py:268`
...
        internal_edges = db.scalars(
**PERF-14** `backend/app/modules/topology/service.py:431`
...
        new_steps = current_steps + [next_step]
**PERF-14** `backend/app/schemas/topology.py:34`
...
        le=500,

## Impacto
- **PERF-02:** Requisições de minutos/horas em bases reais; pool esgotado; qualquer viewer pode derrubar a API.
- **PERF-14:** Requisições de vários segundos com splitters em cascata (fan-out 1:8 × 1:8...).

## Sugestão de correção
- **PERF-02:** Resolver o impacto em SQL/gráfo em memória: carregar terminais/conexões/arestas uma vez (ou CTE recursiva) e
fazer BFS único a partir dos segmentos rompidos; mover para job assíncrono para bases grandes; limitar `cable_segment_ids`.
- **PERF-14:** Pré-carregar o gráfico do subconjunto (terminais/conexões/arestas) em 3-4 queries; aplicar `min(max_hops,
MAX_TRACE_HOPS)`; memoizar; rate limit.

## Como validar
- **PERF-02:** `tools/measure2.py` (contador de statements) e `EXPLAIN ANALYZE`; repetir com N clientes gerados.
- Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
...
    DATABASE_URL=... PYTHONPATH=backend python docs/security-audit/tools/measure2.py # POST /topology/impact: statements e
tempo (antes: 62 statements/vínculo)
...
- **PERF-14:** `trace` a partir de terminal OLT com fan-out alto em dataset sintético; contar statements.
- Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
...
    Contador de statements em POST /api/v1/topology/trace com terminal OLT de fan-out alto (dataset sintético + splitters em
cascata)
...

## Critérios de aceite
- [ ] PERF-02: Teste: statements de `/topology/impact` independem do nº de clientes ( $\leq 30$ ) – falha hoje (62 por vínculo).
- [ ] PERF-02: `cable_segment_ids` limitado (`max_length`) com 422 acima do teto.
- [ ] PERF-14: Statements do rastreo independem de `max_hops` ( $\leq 10$  fixos) – falha hoje.
- [ ] PERF-14: `max_hops` efetivo  $\leq$  `MAX_TRACE_HOPS`.
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado
no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

```

--- FIM ISSUE 24 ---

Issue 25 — [Performance] Upload de imagem: loop bloqueado, amplificação de memória e DecompressionBomb

--- ISSUE 25 ---

Título: [Performance] Upload de imagem: loop bloqueado, amplificação de memória e DecompressionBomb

Labels sugeridas: performance, severity:alta

Achados cobertos: PERF-04

Descrição do problema e por que é explorável / degrada o sistema
 ### PERF-04 – Upload de anexo: handler async executa I/O e Pillow síncronos no event loop e uma PNG de 100 KiB consome +339 MiB (severidade: alta)
 `upload\_attachment` é `async def`, lê o corpo inteiro (`await file.read()`) e chama `save\_attachment` sincronamente (Pillow `convert('RGB')` + `thumbnail(LANCZOS)` + `WEBP method=6`, escrita em disco e commit) – bloqueando o event loop do único processo. O limite de 10 MiB é do arquivo comprimido; o Pillow só avisa acima de 89 Mpx e levanta `DecompressionBombError` (não capturado → 500) acima de 178 Mpx.
 - **Por que é explorável / degrada:** Um upload de 100 KiB (88 Mpx) aloca centenas de MiB e ocupa o loop ~0,7 s; poucos uploads paralelos causam OOM e travam **todas** as requisições (inclusive `async def` de import).
 - **Condições de ocorrência:** `POST /api/v1/attachments` por qualquer technician/engineer/admin (ou sessão comprometida); PNG grande com poucos KiB.
 - **Gatilho de escala:** 1 upload de 100 KiB já causa +339 MiB (medido); 4–5 simultâneos ≈ 1,5 GiB (extrapolação linear).
 - **Ordem de crescimento:** 0(WxH) memória e CPU por upload, independente do tamanho do arquivo comprimido.
 - **Prova:** medido: `tools/measure4.py` – PNG 9400×9400 (88,4 Mpx, **100 KiB**): passa `inspect\_file\_content`; `generate\_thumbnail\_image` = 0,67 s, RSS pico 516 MiB (antes 177 MiB) → **+339 MiB**. PNG 14000×14000 (23,3 KiB): `DecompressionBombError` não capturado.

Evidência

```

**PERF-04** `backend/app/api/v1/attachments.py:46`
...

async def upload_attachment(
...
**PERF-04** `backend/app/api/v1/attachments.py:66`
...
    raw_content = await file.read()
...
**PERF-04** `backend/app/modules/attachments/service.py:121`
...
    thumb_img = img.convert("RGB")
...
**PERF-04** `backend/app/modules/attachments/service.py:127`
...
except (UnidentifiedImageError, OSError, ValueError):
...

```

Impacto

- **PERF-04:** DoS por usuário de baixo privilégio; latência global durante uploads; 500 + arquivo órfão (EST-09).

Sugestão de correção

- **PERF-04:** Declarar o handler `def` (threadpool) ou `run\_in\_threadpool`; validar dimensões antes de decodificar (`Image.open` → checar `size`) com teto ~25 Mpx e capturar `DecompressionBombError`; ler o corpo em streaming com limite.

Como validar

- **PERF-04:** `tools/measure4.py`: RSS pico e tempo por upload; alvo < +50 MiB e rejeição 422 acima do teto.
 - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
 ...
 PYTHONPATH=backend python docs/security-audit/tools/measure4.py # RSS pico e tempo do thumbnail (antes: +339 MiB, 0,67 s; bomb 14000×14000 → exceção)
 ...

Critérios de aceite

- [] PERF-04: PNG 9400×9400 → 422 antes de decodificar; RSS do processo não cresce > 50 MiB (teste com `tracemalloc`, falha hoje).
 - [] PERF-04: PNG 14000×14000 → 422 (hoje 500 por exceção não tratada).
 - [] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
 - [] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 25 ---

Issue 26 — [Performance] Índices ausentes: terminals(entity_type, entity_id), trigram para ILIKE e created_at

--- ISSUE 26 ---

Título: [Performance] Índices ausentes: terminals(entity_type, entity_id), trigram para ILIKE e created_at

Labels sugeridas: performance, severity:média

Achados cobertos: PERF-03, PERF-06, PERF-07

Descrição do problema e por que é explorável / degrada o sistema
 ### PERF-03 – Falta índice em terminals(entity_type, entity_id): buscas do rastreo e do impacto fazem Seq Scan em 206 mil linhas (severidade: média)
 A coluna `entity\_id` foi adicionada na migração 0006 sem índice e o modelo declara apenas `idx\_terminals\_kind\_structure` e `idx\_terminals\_occupancy`. O `EXPLAIN ANALYZE` mostra `Parallel Seq Scan on terminals`.
 - **Por que é explorável / degrada:** Cada lookup varre a tabela inteira; o impacto (PERF-02) faz vários por cliente.
 - **Condições de ocorrência:** Todas as consultas `Terminal.entity\_type = 'port' AND Terminal.entity\_id = ...` (rastreo, impacto, orçamento óptico, medições).
 - **Gatilho de escala:** Custo cresce com o nº de terminais (2 por fibra); medido 12–14 ms/lookup com 206.428 linhas.
 - **Ordem de crescimento:** 0(T) por lookup (T = terminais) → 0(T·N) por request.
 - **Prova:** medido: `EXPLAIN (ANALYZE)` em `tools/measure2.py` – `Parallel Seq Scan on terminals ... Rows Removed by Filter:

68809 (×3 workers)\, Execution Time 12,5–14,4 ms (206.428 linhas).

PERF-06 – Buscas por ILIKE '%q%' sem pg_trgm/GIN em sites, estruturas, cabos, dispositivos, clientes, usuários e notas de portas (severidade: média)
 Todas as consultas de texto usam %termo%, que não usa índices B-tree; as migrações não criam pg_trgm nem índices GIN. O termo também não escapa %/_ (curinga).
 - **Por que é explorável / degrada:** Seq Scan por tabela e por requisição de busca (5 tabelas na busca global).
 - **Condições de ocorrência:** Busca global, listagens com 'q' e relatórios (Port.notes ILIKE '%danificad%'); cresce com o tamanho das tabelas.
 - **Gatilho de escala:** Medido 4,6 ms em 10.001 estruturas (Seq Scan, 10.001 linhas removidas); dor esperada em ordens de 10⁵ linhas (linear; não medido).
 - **Ordem de crescimento:** O(N) por tabela por busca.
 - **Prova:** medido (parcial): EXPLAIN (ANALYZE) em tools/measure.py – Seq Scan on structures ... Rows Removed by Filter: 10001, 4,588 ms. Ausência de pg_trgm/GIN: grep -rniE 'pg_trgm|gin' backend/migrations → 0.

PERF-07 – Listagens paginadas com COUNT(*) + OFFSET + ORDER BY created_at sem índice de suporte (severidade: baixa)
 As listagens fazem count(*) completo a cada página e ORDER BY created_at DESC, id OFFSET n; created_at não é indexado nas tabelas de inventário (só em auditoria/login). OFFSET profundo descarta linhas ordenadas.
 - **Por que é explorável / degrada:** Custo cresce com o tamanho da tabela e com a página (O(N log N) por página).
 - **Condições de ocorrência:** Tabelas grandes (estruturas, cabos, medições, auditoria) e páginas profundas.
 - **Gatilho de escala:** Não dói hoje (5,9 ms @10.001); gatilho esperado > 10⁵ linhas.
 - **Ordem de crescimento:** O(N log N) por página (sort) e O(N) para COUNT.
 - **Prova:** medido: EXPLAIN (ANALYZE) – Sort ... quicksort Memory: 2920kB sobre Seq Scan (10.001 linhas), 5,924 ms; count(structures.id) 1,415 ms.

Evidência

```

**PERF-03** `backend/app/modules/connectivity/models.py:46`
...
    entity_id: Mapped[uuid.UUID | None] = mapped_column(UUID(as_uuid=True), nullable=True)
...
**PERF-03** `backend/app/modules/connectivity/models.py:57`
...
    Index("idx_terminals_occupancy", "occupancy"),
...
**PERF-03** `backend/migrations/versions/0006_connectivity_engine.py:31`
...
    sa.Column("entity_id", postgresql.UUID(as_uuid=True), nullable=True),
...
**PERF-03** `backend/app/modules/topology/service.py:565`
...
        Terminal.entity_id.in_(onu_ports),
...
**PERF-06** `backend/app/modules/reports/service.py:205`
...
        or_(Site.code.ilike(f"%{clean_q}%"), Site.name.ilike(f"%{clean_q}%")),
...
**PERF-06** `backend/app/modules/inventory/service.py:85`
...
        term = f"%{q.strip()}"
...
**PERF-06** `backend/app/modules/customers/service.py:85`
...
        search = f"%{q.strip()}"
...
**PERF-06** `backend/app/modules/reports/service.py:164`
...
        Port.notes.ilike("%danificad%"),
...
**PERF-07** `backend/app/modules/inventory/service.py:90`
...
        total = session.scalar(count_query) or 0
...
**PERF-07** `backend/app/modules/inventory/service.py:94`
...
        query.order_by(Site.created_at.desc(), Site.id).offset(offset).limit(page_size)
...
**PERF-07** `backend/app/db/base.py:18`
...
        created_at: Mapped[datetime] = mapped_column(

```

Impacto

- **PERF-03:** Latência linear no tamanho de `terminals`, multiplicada por N chamadas por request.
 - **PERF-06:** Latência linear com o volume; combina-se com SEC-01 (busca anônima).
 - **PERF-07:** Hoje baixo (medido 5,9 ms para OFFSET 9000 em 10 mil linhas); relevante em ordens de 10⁵–10⁶.

Sugestão de correção

- **PERF-03:** CREATE INDEX CONCURRENTLY idx_terminals_entity ON terminals (entity_type, entity_id); via migração.
 - **PERF-06:** CREATE EXTENSION pg_trgm + GIN (code gin_trgm_ops) (e name/serial/notes); escapar curingas do termo; para 'q' curto exigir mínimo de 3 caracteres.
 - **PERF-07:** Índice (created_at DESC, id); paginação keyset; contagem aproximada/cache.

Como validar

- **PERF-03:** EXPLAIN (ANALYZE) SELECT * FROM terminals WHERE entity_type='port' AND entity_id=:id antes/depois: deve usar Index Scan (< 1 ms).
 - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
 ...
 psql ... -c "EXPLAIN (ANALYZE) SELECT * FROM terminals WHERE entity_type='port' AND entity_id='<uuid>' # antes:
 Parallel Seq Scan, 12–14 ms; depois: Index Scan

```

    ...
- **PERF-06:** `EXPLAIN (ANALYZE)` da busca em `structures`/`devices` com 100k+ linhas antes/depois.
- Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
    ...
    DATABASE_URL=... PYTHONPATH=backend python docs/security-audit/tools/measure.py # EXPLAIN (ANALYZE) das buscas ILIKE
(antes: Seq Scan 4,6 ms @10k)
    ...
- **PERF-07:** `EXPLAIN (ANALYZE)` de `ORDER BY created_at DESC OFFSET 9000` antes/depois.
- Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
    ...
    DATABASE_URL=... PYTHONPATH=backend python docs/security-audit/tools/measure.py # EXPLAIN de ORDER BY created_at DESC
OFFSET 9000 (antes: Sort 5,9 ms)
    ...

## Critérios de aceite
- [ ] PERF-03: Migração cria o índice; teste de plano (`EXPLAIN`) afirma `Index Scan` (falha hoje).
- [ ] PERF-06: Plano usa `Bitmap Index Scan on ...trgm...` nas buscas (teste de plano; falha hoje).
- [ ] PERF-07: Plano usa `Index Scan Backward` na listagem (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado
no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 26 ---

```

Issue 27 — [Performance] Importação/exportação em memória e por entidade (parse síncrono, N+1 de cancelamento)

--- ISSUE 27 ---

Título: [Performance] Importação/exportação em memória e por entidade (parse síncrono, N+1 de cancelamento)

Labels sugeridas: performance, severity:média

Achados cobertos: PERF-05, PERF-08

```

## Descrição do problema e por que é explorável / degrada o sistema
### PERF-05 — Importação: parse síncrono de até 20 MB dentro de handler async, tudo em memória, e 1 SELECT de cancelamento
por entidade (severidade: média)
`preview_import` é `async def` e chama `create_import_preview` sincronamente (`json.loads` de até 20 MB, lista de dicts com
todas as feições, hash, escrita em disco, commit) no event loop. No worker, `execute_import_commit` faz `db.refresh(job)` (1
SELECT) e `flush()` (1 INSERT) por entidade e relê o arquivo inteiro.
- **Por que é explorável / degrada:** Arquivos grandes bloqueiam o loop por segundos e ocupam centenas de MiB; o commit tem
~2 round-trips por entidade.
- **Condições de ocorrência:** `POST /imports/preview` com arquivos grandes; commit no worker.
- **Gatilho de escala:** Arquivos > alguns MB (limite 20 MB); commit com dezenas de milhares de entidades.
- **Ordem de crescimento:** O(N) memória e O(2N) round-trips.
- **Prova:** verificado por leitura, não medido. Como medir: gerar GeoJSON de 20 MB e cronometrar o handler +
`PYTHONASYNCIODEBUG=1`.

### PERF-08 — Exportações carregam camadas inteiras em memória (`all()`) e serializam JSON com indent (severidade: média)
`generate_export` faz `db.scalars(select(Site)).all()`, `select(Structure)`, `select(CableSegment)`, `select(Customer)`
sem streaming e monta a lista completa de features antes de `json.dump(..., indent=2)`/`ET.toString`.
- **Por que é explorável / degrada:** Memória proporcional ao total de geometrias; sem paginação nem `yield_per`.
- **Condições de ocorrência:** `export_*` em redes grandes (após corrigir o worker — EST-01).
- **Gatilho de escala:** Dezenas de milhares de segmentos/estruturas (não medido).
- **Ordem de crescimento:** O(N) memória.
- **Prova:** verificado por leitura, não medido.

## Evidência
**PERF-05** `backend/app/api/v1/imports_exports.py:42`
...
async def preview_import(
...
**PERF-05** `backend/app/modules/imports/service.py:457`
...
    if len(content) > 20 * 1024 * 1024:
...
**PERF-05** `backend/app/modules/jobs/service.py:81`
...
    db.refresh(job)
...
**PERF-05** `backend/app/modules/jobs/service.py:154`
...
    content = f.read()
...
**PERF-08** `backend/app/modules/exports/service.py:96`
...
    sites = db.scalars(select(Site)).all()
...
**PERF-08** `backend/app/modules/exports/service.py:140`
...
    segments = db.scalars(select(CableSegment)).all()
...
**PERF-08** `backend/app/modules/exports/service.py:337`
...
    json.dump(geojson_data, f, ensure_ascii=False, indent=2)
...

## Impacto
- **PERF-05:** Latência global durante previews e importações demoradas (que ainda exigem lease — EST-07).
- **PERF-08:** Picos de memória no worker e tempo alto para redes com dezenas de milhares de segmentos.

```

```
## Sugestão de correção
- **PERF-05:** Handler `def`/threadpool com streaming; limites de feições; verificar cancelamento a cada N entidades e usar `bulk_insert_mappings`.
- **PERF-08:** `yield_per`/cursor server-side e escrita incremental (NDJSON/'json.dump` por feature); limite por camada.

## Como validar
- **PERF-05:** Arquivo GeoJSON de 20 MB local; medir tempo do loop (asyncio debug slow-callback) e RSS.
  - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
    ```
 PYTHONASYNCIODEBUG=1 uvicorn app.main:app & curl -w '%{time_total}\n' -F file=@grande.geojson -H 'X-CSRF-Token: ...' -b cookies.txt http://127.0.0.1:8000/api/v1/imports/preview # medir tempo e `ps -o rss`; em paralelo `curl /health/live`
    ```
- **PERF-08:** Exportar dataset sintético (10k estruturas/5k segmentos) medindo RSS e tempo.
  - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
    ```
 /usr/bin/time -v python -c "from app.modules.exports.service import generate_geojson_export ..." # RSS pico antes/depois com o dataset sintético
    ```

## Critérios de aceite
- [ ] PERF-05: Teste: `/imports/preview` não bloqueia `/health/live` (latência < 100 ms durante o parse).
- [ ] PERF-05: Import de 50 mil pontos executa < 5 s de round-trips (contador de statements ≤ 2 por lote de 500).
- [ ] PERF-08: RSS do worker durante export ≤ 2x o tamanho do arquivo (teste com `tracemalloc`, falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.

--- FIM ISSUE 27 ---
```

Issue 28 — [Performance] Pool, workers e timeouts: statement_timeout, readiness, auth path e hotspot de revisão

--- ISSUE 28 ---

Título: [Performance] Pool, workers e timeouts: statement_timeout, readiness, auth path e hotspot de revisão

Labels sugeridas: performance, severity:média

Achados cobertos: PERF-12, PERF-13, PERF-10, PERF-09

Descrição do problema e por que é explorável / degrada o sistema

PERF-12 – Um único processo uvicorn, pool de 10+20 conexões e nenhum statement_timeout/timeout de request (severidade: média)

`CMD uvicorn` sem `--workers`; handlers síncronos rodam na threadpool padrão (40 threads) sobre um pool de 30 conexões; `create\_engine` não define `connect\_args` (sem `statement\_timeout`/`connect\_timeout`) e não há timeout de requisição. Um endpoint lento retém conexões por segundos.

- **Por que é explorável / degrada:** Poucas requisições lentas esgotam o pool (30) e as demais aguardam até `pool\_timeout=30 s` – inclusive `/health/ready`, que também precisa de conexão.
- **Condições de ocorrência:** Carga concorrente moderada ou 1 endpoint lento.
- **Gatilho de escala:** Medido: 20 requisições concorrentes ao dashboard → todas concluem ~17 s; readiness saltou para 1,1 s.

- **Ordem de crescimento:** Fila: latência ≈ N × tempo de serviço em 1 worker (GIL).
- **Prova:** medido: `tools/measure5.py` – 20 GETs concorrentes: wall 16,9 s (sequencial teórico 21,9 s), p50 16,8 s; `/health/ready`: 151 ms (baseline) → [1126, 228, 259, 216, 260, 219] ms. In-process (TestClient), 1 processo: indica ordem de grandeza, não capacidade de produção.

PERF-13 – /health/ready lê o diretório do Alembic do disco e usa uma conexão do pool a cada sonda (10 s) (severidade: baixa)

`check\_database\_migrations` chama `ScriptDirectory.from\_config(...)` (parse de arquivos de migração) a cada readiness; a rota depende de `get\_db` (checkout no pool).

- **Por que é explorável / degrada:** Custo desnecessário por sonda e acoplamento do readiness ao pool que ele deveria monitorar.

- **Condições de ocorrência:** Sempre; agrava-se sob saturação do pool (PERF-12).
- **Gatilho de escala:** Sondas a cada 10 s (compose); ruim sob pool esgotado.
- **Ordem de crescimento:** 0 (arquivos de migração) por sonda.
- **Prova:** medido (parcial): readiness baseline 151 ms; ver PERF-12 para o comportamento sob carga.

PERF-10 – Cada requisição autenticada faz 2 SELECTs (sessão e usuário por lazy load) e pode gravar last_activity_at (severidade: baixa)

`get\_active\_session\_by\_token` faz `select(UserSession).join(User)` só para filtrar; `user\_session.user` dispara um segundo SELECT (lazy). A cada 60 s de atividade há `UPDATE` + `COMMIT` dentro de um GET.

- **Por que é explorável / degrada:** Custo fixo de ~2 round-trips por request (mais o commit periódico).
- **Condições de ocorrência:** Todas as rotas autenticadas.
- **Gatilho de escala:** Proporcional ao tráfego total.
- **Ordem de crescimento:** 0(1) extra por request.
- **Prova:** verificado por leitura, não medido.

PERF-09 – Linha única network_topology_state atualizada em toda escrita de topologia serializa transações concorrentes (severidade: baixa)

`bump\_topology\_revision` executa `UPDATE network\_topology\_state ... WHERE id = 1` dentro de cada transação de topologia; o lock de linha só é liberado no `COMMIT`. É correto para o contador monotônico (**ponto forte**), mas serializa os commits.

- **Por que é explorável / degrada:** Transações longas (importação, split) seguram o lock da linha e bloqueiam as demais escritas de topologia.

- **Condições de ocorrência:** Edição concorrente por vários usuários (fusão, divisão, importação, conexões).
- **Gatilho de escala:** Não medido; relevante com muitos editores simultâneos.
- **Ordem de crescimento:** Serialização 0(1) por commit de topologia.
- **Prova:** verificado por leitura, não medido.

Evidência

PERF-12 `backend/app/db/session.py:21`

```

...
        pool_size=settings.DB_POOL_SIZE,
...
**PERF-12** `backend/app/core/config.py:40`
...
        DB_MAX_OVERFLOW: int = 20
...
**PERF-12** `backend/Dockerfile:59`
...
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
...
**PERF-13** `backend/app/db/health.py:46`
...
        script_dir = ScriptDirectory.from_config(alembic_cfg)
...
**PERF-13** `backend/app/api/v1/health.py:39`
...
def readiness(response: Response, db: Session = Depends(get_db)) -> dict[str, Any]:
...
**PERF-13** `compose.yaml:55`
...
        test: ["CMD", "curl", "-f", "http://localhost:8000/health/ready"]
...
**PERF-10** `backend/app/modules/identity/service.py:124`
...
        .join(User)
...
**PERF-10** `backend/app/core/dependencies.py:149`
...
        user = user_session.user
...
**PERF-10** `backend/app/modules/identity/service.py:154`
...
        user_session.last_activity_at = now
...
**PERF-09** `backend/app/modules/gis/service.py:36`
...
        "UPDATE network_topology_state "
...
**PERF-09** `backend/app/modules/cables/service.py:790`
...
        new_rev = bump_topology_revision(db)
...

## Impacto
- **PERF-12:** Falhas em cascata (readiness), latência global e impossibilidade de isolar endpoints caros.
- **PERF-13:** Sondas lentas/falhas sob carga → container 'unhealthy'.
- **PERF-10:** Baixo; soma-se a todas as demais latências.
- **PERF-09:** Contenção sob edição concorrente intensa; hoje baixa.

## Sugestão de correção
- **PERF-12:** `--workers N` (ou réplicas), `statement_timeout` (ex.: 15 s) no `connect_args`, timeout de request no proxy, dimensionar pool por worker e separar rotas pesadas (fila).
- **PERF-13:** Cachear o head esperado em memória no startup; usar conexão dedicada/curta (`engine.connect()` com timeout).
- **PERF-10:** `options(joinedload(UserSession.user))` `/contains_eager` e atualizar `last_activity_at` de forma assíncrona/amostrada.
- **PERF-09:** Manter o bump como último passo antes do commit (já é) e evitar transações longas; considerar sequence/advisory lock curto.

## Como validar
- **PERF-12:** `tools/measure5.py` com timeout configurado; observar `/health/ready` sob carga.
    - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
        ...
        DATABASE_URL=... PYTHONPATH=backend python docs/security-audit/tools/measure5.py # readiness sob carga (antes: pico 1,1 s; wall 16,9 s)
        ...
- **PERF-13:** Medir latência do readiness com e sem carga.
    - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
        ...
        for i in $(seq 20); do curl -s -o /dev/null -w '%{time_total}\n' http://127.0.0.1:8000/health/ready; done # com e sem carga
        ...
- **PERF-10:** Contador de statements de `GET /auth/me`: 2 hoje → 1.
    - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
        ...
        Contador de statements (event before_cursor_execute) em GET /api/v1/auth/me # por leitura, antes: 2 (não medido); alvo: 1
        ...
- **PERF-09:** `pg_locks` / `pg_stat_activity` durante 2 splits paralelos.
    - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
        ...
        psql -c "SELECT pid, wait_event_type, wait_event, query FROM pg_stat_activity WHERE wait_event_type='Lock'" # durante dois splits paralelos
        ...

## Critérios de aceite
- [ ] PERF-12: Consulta com `pg_sleep(60)` é cancelada em ≤ statement_timeout (teste de integração, falha hoje).
- [ ] PERF-12: Readiness responde < 500 ms com 20 requisições caras concorrentes.
- [ ] PERF-13: Readiness não faz I/O de arquivo por chamada (teste com mock de `ScriptDirectory`).

```

```
- [ ] PERF-10: Teste: `GET /auth/me` executa 1 SELECT (falha hoje).
- [ ] PERF-09: Duas divisões em segmentos distintos concluem sem esperar mais que o tempo de commit (teste de concorrência).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 28 ---
```

Issue 29 — [Performance] Política de retenção para login_attempts, sessões e exportações

--- ISSUE 29 ---

Título: [Performance] Política de retenção para login_attempts, sessões e exportações

Labels sugeridas: performance, severity:baixa

Achados cobertos: PERF-11

```
## Descrição do problema e por que é explorável / degrada o sistema
### PERF-11 – Sem retenção: login_attempts, user_sessions expiradas, audit_events e arquivos de exportação crescem indefinidamente (severidade: baixa)
A única rotina de limpeza (`clean_expired_previews_and_exports`) remove apenas `ImportPreview`; `login_attempts` (consultada a cada login com `OR`), sessões revogadas/expiradas e arquivos de exportação nunca são removidos.
- **Por que é explorável / degrada:** Tabelas e disco crescem sem teto; o `count(*)` do rate limit varre um histórico crescente.
- **Condições de ocorrência:** Operação prolongada.
- **Gatilho de escala:** Meses de operação/alto volume de logins.
- **Ordem de crescimento:** O(t) linear no tempo.
- **Prova:** verificado por leitura, não medido.

## Evidência
**PERF-11** `backend/app/modules/jobs/service.py:332`
```
def clean_expired_previews_and_exports(db: Session) -> dict[str, int]:
```
**PERF-11** `backend/app/modules/identity/models.py:58`
```
class LoginAttempt(Base):
```
**PERF-11** `backend/app/modules/identity/models.py:27`
```
class UserSession(Base):
```

## Impacto
- **PERF-11:** Degradação lenta e consumo de disco.

## Sugestão de correção
- **PERF-11:** Job de retenção (ex.: 30 dias para `login_attempts`, 7 dias para sessões inválidas, TTL de exportações) e índice composto `(email, attempted_at)`.

## Como validar
- **PERF-11:** `SELECT count(*)` por tabela após semanas de operação; `EXPLAIN` do rate limit.
  - Comando de medição (executar **antes** e **depois** da correção; em banco descartável):
    

```
psql -c "SELECT relname, n_live_tup FROM pg_stat_user_tables WHERE relname IN ('login_attempts', 'user_sessions', 'audit_events')" # semanas depois
```



## Critérios de aceite
- [ ] PERF-11: Teste: registros mais antigos que o TTL são removidos pelo worker (falha hoje).
- [ ] Existe ao menos um teste automatizado que **falha antes** da correção e **passa depois** (indicado acima), executado no CI.
- [ ] Nenhum segredo em claro em logs, respostas ou testes adicionados.
--- FIM ISSUE 29 ---
```